

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Enabling Co-Creation for Augmented Reality: A User-Friendly Narrative Editor for Cultural Heritage Experiences

Miguel Norberto Costa Freitas



Mestrado em Engenharia Informática e Computação

Supervisor: Maria van Zeller

Second Supervisor: António Coelho

July 23rd, 2024

Enabling Co-Creation for Augmented Reality: A User-Friendly Narrative Editor for Cultural Heritage Experiences

Miguel Norberto Costa Freitas

Mestrado em Engenharia Informática e Computação

Approved by:

President: Gilberto Bernardes de Almeida

Referee: Claudio Cleverson de Lima

Referee: Maria Van Zeller de Macedo de Oliveira e Sousa

July 23rd, 2024

Abstract

Augmented Reality (AR) technology has significantly permeated modern life, offering novel ways to interact with the physical world. The ever-expanding landscape of interactive storytelling has witnessed significant progress over the last decade, with a myriad of tools emerging to cater to the changing needs of users. They have been widely used in contexts such as education and video game development. This thesis capitalizes on these recent advancements, contributing to the ongoing discourse on story creation in AR environments. It presents a story editor developed to meet the evolving demands of the digital age, where storytelling and interactive content have gained unprecedented prominence. The proposed story editor reflects the culmination of the progress observed in developing interactive story editors over the past years. By leveraging and building upon these advancements, the goal is to further enhance the accessibility, creativity, and user experience associated with authoring tools for narrative creation.

The thesis documents the development of a user-friendly interactive story editor — a tool designed for individuals without programming backgrounds, including historians, curators, tour guides, and the general public. Operating within a web environment, this editor empowers users to craft interactive multimedia narratives encompassing elements such as 3D models, videos, images, and AR experiences. A systematic review of existing literature and technologies identified the current state of story editors, highlighting gaps and opportunities for improvement. Meetings with stakeholders refined the project goals to align with practical needs. The Design Science Research (DSR) methodology guided the development process through several phases. Requirements were defined based on the review and stakeholder input, focusing on usability, accessibility, and functionality for non-technical users. A solution was then proposed and developed using web technologies. The AR story editor and player were built with React, while the backend was developed with NodeJS. User testing and feedback sessions evaluated the solutions, allowing continuous refinement to ensure the final product met user needs and expectations.

The project's primary objective is to support the creation of AR-enabled visits to cultural heritage sites in co-creation environments, such as workshops. The results from this thesis will contribute towards the democratization of content creation by making it more accessible and thus unlocking new dimensions of self-expression, which will ultimately lead to a more inclusive approach to cultural heritage experiences.

Keywords: Interactive storytelling, Interactive Narrative, Authoring Tool, End-user, No code, Augmented Reality

ACM Classification: Human-centered computing → - Human computer interaction (HCI) → Interactive systems and tools → User interface management systems; Human-centered computing → Human computer interaction (HCI) → Interactive systems and tools → User interface programming; Human-centered computing → Human computer interaction (HCI) → Interaction paradigms → Mixed / augmented reality

Resumo

A tecnologia de Realidade Aumentada (AR) penetrou significativamente na vida moderna, oferecendo novas formas de interação com o mundo físico. O cenário em constante expansão da narração interativa de histórias testemunhou um progresso significativo na última década, com uma miríade de ferramentas a surgir para satisfazer as necessidades em transformação dos utilizadores. Estas ferramentas têm sido amplamente utilizadas em contextos como a educação e o desenvolvimento de jogos de vídeo. Esta tese aproveita estes avanços recentes, contribuindo para a atual discussão sobre a criação de histórias em ambientes de AR. Apresenta um editor de histórias desenvolvido para satisfazer as exigências em evolução da era digital, em que a narração de histórias e os conteúdos interativos ganharam uma proeminência sem precedentes. O editor de histórias proposto reflete o culminar dos progressos observados no desenvolvimento de editores de histórias interativos ao longo dos últimos anos. Aproveitando e construindo sobre estes avanços, o objetivo é melhorar ainda mais a acessibilidade, a criatividade e a experiência do utilizador associadas às ferramentas de autoria para a criação de narrativas.

Esta tese documenta o desenvolvimento de um editor de histórias interativo de fácil utilização - uma ferramenta concebida para pessoas sem conhecimentos de programação, incluindo historiadores, curadores, guias turísticos e o público em geral. Funcionando num ambiente Web, este editor permite aos utilizadores criar narrativas multimédia interativas que incluem elementos como modelos 3D, vídeos, imagens e experiências de AR. Uma revisão sistemática da literatura e das tecnologias existentes identificou o estado atual dos editores de histórias, destacando lacunas e oportunidades de melhoria. As reuniões com as partes interessadas refinaram os objetivos do projeto para os alinhar com as necessidades práticas. A metodologia Design Science Research (DSR) orientou o processo de desenvolvimento através de várias fases. Os requisitos foram definidos com base na revisão e nas contribuições das partes interessadas, centrando-se na usabilidade, acessibilidade e funcionalidade para utilizadores não técnicos. Foi então proposta e desenvolvida uma solução utilizando tecnologias Web. O editor e o leitor de histórias de realidade aumentada foram construídos com React, enquanto o backend foi desenvolvido com NodeJS. As sessões de teste e feedback dos utilizadores avaliaram as soluções, permitindo um aperfeiçoamento contínuo para garantir que o produto final satisfazia as necessidades e expectativas dos utilizadores.

O principal objetivo do projeto é apoiar a criação de visitas a sítios do património cultural com recurso à AR em ambientes de co-criação, como workshops. Os resultados desta tese contribuirão para a democratização da criação de conteúdos, tornando-a mais acessível e desbloqueando assim novas dimensões de auto-expressão, o que, em última análise, conduzirá a uma abordagem mais inclusiva das experiências do património cultural.

Acknowledgments

I would like to thank my parents for supporting me throughout my life. Without their support, I certainly would not have been able to enroll in university, and this dissertation would never exist. Thank you for providing me with opportunities all these years. I would like to thank my friends for giving me many great memories throughout my 5 years of university. A special thanks to my two great friends, Marta and Carol, who accompanied me on many afternoons spent at FEUP working on this dissertation. Thanks for letting me take my mind off the stressful dissertation work and giving me tons of opportunities to laugh. You two are very special people. A very special thanks to Pedro Nascimento, who made the beautiful logo for the application and for providing wonderful feedback about the application. Finally, I thank my friends for hanging out with me after long days of work, and helping me do some much needed unwinding. Special thanks to Diogo Pereira, who was always quick to show up. I would also like to thank my supervisors, Professor Maria van Zeller, and Professor António Coelho, for assisting me throughout the whole dissertation process. A special thanks to Mariana Sarmento for helping me validate my work.

Last but definitely not least, a thank you as big as the world to my wonderful girlfriend Gabi for constantly giving me some much needed encouragement and always being there for me when I needed and listening to me whine when I was frustrated and tired. Couldn't have done it without you!

Miguel Freitas

“Men make their own history, but they do not make it as they please; they do not make it under self-selected circumstances, but under circumstances existing already, given and transmitted from the past.”

Karl Marx

Contents

1	Introduction	1
1.1	Context	1
1.2	Motivation, Questions and Research Objectives	2
1.3	Methodological framework	3
1.4	Document structure	5
2	State of the Art	6
2.1	Literature review	6
2.2	Systematic review	13
2.3	Results	16
2.4	Discussion	28
2.5	Non-academic apps analysis	34
3	Development	41
3.1	Process employed	41
3.2	Previous iteration analysis	42
3.3	Requirements Elicitation and Objectives	49
3.4	Prototype design	51
4	Implementation	58
4.1	Architecture	58
4.2	Story Editor	60
4.3	Story Player	81
4.4	Backend	87
5	User testing	88
5.1	Procedure adopted	88
5.2	Results	90
5.3	Discussion	95
5.4	Improvements After User Testing	97
6	Conclusions	99
6.1	Summary	99
6.2	Contributions	100
6.3	Limitations	102
6.4	Future Work	103
	References	106

A	Informed Consent	109
A.1	Consentimento Informado para Participação em Teste de Aplicação	109
B	Low-fidelity Prototyping Example	111
C	User Testing Results	119

List of Figures

1.1	Design Science Research Methodology	3
2.1	Diagram representing the review process	7
2.2	Moirai's color-coded nodes	18
2.3	WEGAS's role-based access. The left side shows how users can switch between the 4 roles	19
2.4	TaleBlazer's block programming interface	23
2.5	ZapWorks Designer - User defining a new button for the AR scene. Note the possibility of anchoring the component to any edge of the screen and configuring the size and position.	35
2.6	ARWay - Adding an AR image to the floorplan.	37
2.7	ARWay - Heatmap displays the amount of traffic across different map zones. . . .	38
2.8	Metaverse Studio - Storyboard view with one Character Scene.	39
3.1	Previous Iteration's Unity Prototype Main Screen - © João Cardoso	43
3.2	Previous Iteration's Unity Prototype Display Story Contents - © João Cardoso . .	43
3.3	Previous Iteration's Unity Prototype Load Menu - © João Cardoso	44
3.4	Previous Iteration's Unity Prototype Save Menu - © João Cardoso	44
3.5	Previous Iteration's React Prototype Create Story Menu - © João Cardoso	45
3.6	Previous Iteration's React Prototype Main Menu - © João Cardoso	45
3.7	Previous Iteration's React Prototype Select Character Menu - © João Cardoso . .	46
3.8	Previous Iteration's React Prototype Create Character Menu - © João Cardoso . .	47
3.9	Previous Iteration's React Prototype Save Story Menu - © João Cardoso	47
3.10	Previous Iteration's React Prototype Load Story Menu - © João Cardoso	48
3.11	Previous Iteration's Player Main Menu - © João Cardoso	48
3.12	Editor Design Prototype's Flowchart tab with the Inspector displayed	51
3.13	Editor Design Prototype's Add Scene Menu	52
3.14	Editor Design Prototype's Map Menu Initial State	52
3.15	Editor Design Prototype's Map Tab with two anchors	53
3.16	Editor Design Prototype's Popup for naming a new map	53
3.17	Editor Design Prototype's Map Tab	53
3.18	Editor Design Prototype's Dialogue Editing Tab	54
3.19	Editor Design Prototype's Dialogue Add Dialogue Component Menu	54
3.20	Editor Design Prototype's Top App Bar	55
3.21	Editor Design Prototype's Export Story Menu	55
3.22	Player Design Prototype's Experiences tab	56
3.23	Player Design Prototype Playing Screen	56
3.24	Player Design Prototype Profile Tab	57

4.1	Project Overall Architecture	58
4.2	Editor Component Architecture	59
4.3	Player Component Architecture	60
4.4	Story Editor's layout	60
4.5	Popup allowing users to choose a new Scene to add to the Story	62
4.6	Audio Scene displayed on the Story graph	63
4.7	3D Scene without AR enabled vs. same scene with AR enabled. Users use the Inspector tab to toggle between the two modes	63
4.8	GPS Coordinates input fields. Users can choose the map, the place, and the tolerance distance.	64
4.9	QR-Code input fields	65
4.10	Image Tracking input fields. Users must input the image that they wish to be tracked by the camera.	65
4.11	Inspector tab on the Story tab	67
4.12	Preview AR button on the Inspector tab	67
4.13	Using the AR Preview feature to test out an Image Tracking scene	67
4.14	Map tab initial state	68
4.15	Placing anchors on a newly created map	69
4.16	Naming the new map	69
4.17	Final state of the map creation process	70
4.18	Popup prompting the user to select which type of location to add to the map	70
4.19	Dialogue tab with a Dialogue and Question Scene	72
4.20	Popup prompting the user to select which type of node to add to the dialogue	73
4.21	Dialogue tab's Choice node	74
4.22	Story Editor's Top Bar	76
4.23	Popup triggered upon selecting the "Load" option	77
4.24	Export Story Popup	78
4.25	Popup listing the characters in the project	78
4.26	Edit existing character popup	79
4.27	Create new character popup	79
4.28	Success message indicating that a project has been loaded	80
4.29	Error message displayed due to "Scene 2" lacking an outgoing edge	80
4.30	Profile tab view when user is logged out	81
4.31	Profile tab view when user is logged in	82
4.32	Experience Selection screen	83
4.33	Example Start Screen for experience on the Story Player	83
4.34	Quiz Scene on the Story Player	84
4.35	AR-enabled 3D Scene with Image Tracking	84
4.36	Path Scene when the player is far from the destination	85
4.37	Path Scene when the player's distance to the destination is smaller than the tolerance	85
4.38	Dialogue Scene on the Story Player, with the icon on the bottom left indicating that audio is playing	86
4.39	End Scene on the Story Player	86
5.1	Test participant Age Distribution	95
5.2	Test participant Educational Levels	95
5.3	Scene node selected for copying	98
B.1	All Screens in paper format	111

B.2	Story creation process beginning	112
B.3	First Scene in the Story Editor	112
B.4	Second Scene in the Story Editor	113
B.5	Third Scene in the Story Editor	113
B.6	Sixth Scene in the Story Editor	114
B.7	Final State of the Story	114
B.8	Beggining of the Map Creation Process	115
B.9	Placement of the Anchors	115
B.10	Naming the map	116
B.11	Final State of the map	116
B.12	Creating a new Character	117
B.13	List of Experiences in the Story Player	118
C.1	Results for the "How would you rate the overall usability of the application?" question (Story Editor)	119
C.2	Results for the "How difficult was it to navigate your story structure?" question .	120
C.3	Results for the "How difficult was it to use the "Inspector" tab to edit the content of your Scenes?" question	120
C.4	Results for the "How difficult was the character creation and editing process?" question	121
C.5	Results for the "How intuitive was the map calibration process?" question	121
C.6	Results for the "How useful was the "Map" tab to annotate the physical location of your experience?" question	122
C.7	Results for the "How difficult was it to export your story?" question	122
C.8	Results for the "How difficult was it to add your custom content to your story?" question	123
C.9	Results for the "How difficult was it to incorporate AR content in your experi- ence?" question	123
C.10	Results for the "How clear were the explanations on how the different trigger modes for AR work?" question	124
C.11	Results for the "How clear were the export warning/error messages?" question . .	124
C.12	Results for the "How would you rate the application's performance in terms of responsiveness?" question	125
C.13	Results for the "How would you rate the application's performance in terms of speed?" question	125
C.14	Results for the "How would you rate the overall usability of the application?" question (Story Player)	126
C.15	Results for the "How satisfied were you with the content displayed in the Profile tab?" question	126
C.16	Results for the "How satisfied were you with the Experience playing feature?" question	127

List of Tables

2.1	Terms obtained from the literature review	10
2.2	Visual assistance features in each authoring tool	19
2.3	Additional features in each authoring tool	26
5.1	Test Sheet for the Story Editor Tasks	92
5.2	Test Sheet for the Story Player Tasks	94

Abbreviations

AR	Augmented Reality
MR	Mixed Reality
WWW	<i>World Wide Web</i>
POI	Point of Interest
AT	Authoring Tool
CAT	CHESS Authoring Tool
API	Application Programming Interface
JSON	JavaScript Object Notation
PDF	Portable Document Format
PICOC	Population, Intervention, Comparison, Outcome, Context
2D	Two Dimensional
3D	Three Dimensional
IoT	Internet of Things
DSR	Design Science Research
RFID	Radio Frequency Identification
KPI	Key Performance Indicator
LIDAR	Light Detection and Ranging
NPM	Node Package Manager
BSON	Binary JSON
UI	User Interface
AI	Artificial Intelligence

Chapter 1

Introduction

1.1 Context

Narratives were pivotal in society well before the advent of literacy, serving as a foundational element of the human experience. Telling stories is an important part of how we communicate and make sense of our experiences. Creating and sharing stories helps us understand and relate to who we are as individuals and as social beings (Kelliher and Slaney, 2012). Storytelling is a creative process through which people share and reflect on life experiences, solve problems, or teach lessons (Nisi, 2017). It may manifest in diverse forms, spanning digital or physical realms, interactive or static, and can be conveyed through an array of mediums, including written, oral, or visual expressions (Riedl and Bulitko, 2012).

With the advent of modern technology, storytelling has undergone significant evolution in the last decades, with stories becoming increasingly more interactive, open-ended and participatory, with the line between author and audience becoming increasingly blurred (Spanoudakis et al., 2015). The fusion of storytelling and technology, particularly in the realm of Augmented Reality (AR), presents a gateway to reimagining how we perceive and interact with our cultural legacies.

Considering the prevalence of storytelling in human culture, great effort has been invested into creating tools that allow for accessible narrative creation. The digital era has witnessed a surge in user-friendly authoring tools that cater to a broad spectrum of users, from people with programming experience to non-programmers. These tools have seen widespread adoption in education contexts, e.g.: (Quah and Ng, 2022), video games, e.g.: (Kuhn, 2018), and videogame development, among others. As the demand for immersive experiences grows, authoring tools have extended their capabilities to support virtual and augmented reality content creation, ushering in a new era of interactive and engaging storytelling.

The application described in this thesis will be developed as a new iteration of the FronTowns project¹. The ultimate goal of the project is to understand further the role played by small villages in the border regions in the relationship between Portugal and Castile by exploring their connections, such as institutional links in the form of trade deals and partnerships and the flux of people

¹Website: <https://frontowns.fcsh.unl.pt/>

between them. In the first iteration, the aim was to provide a 3D historical recreation of villages on the border of the two political entities, specifically the recreation and study of the evolution of two villages, Castelo de Vide and Cáceres. The previous iteration focused on the development of an editor for narratives that took place in a virtual 3D environment in order to create historical reconstructions and help historians validate and test their hypotheses.

This new iteration cycle aims at prototyping a system that will similarly enable historians/ museum curators/ tour guides to create multimedia experiences aimed at enhancing visits to cultural heritage locations, integrating Augmented Reality. Additionally, a story player application will be developed in order to allow for the experience of the created narratives. The new narrative Editor and Player will then be tested and validated in Museu Soares dos Reis² in the context of co-authoring workshops.

1.2 Motivation, Questions and Research Objectives

Experiencing cultural heritage is a profoundly enriching and essential aspect of human life, contributing to individual identity, collective understanding, and the preservation of our shared history. Moreover, cultural heritage experiences promote cross-cultural appreciation and respect. They facilitate the celebration of diversity and encourage dialogue, breaking down barriers between communities and fostering a more inclusive and interconnected world. Preserving and sharing cultural heritage is, therefore, not merely a matter of nostalgia or tradition but a dynamic process that contributes to the ongoing evolution of societies.

However, the individuals who hold the key to unlocking the rich tapestry of our cultural narratives aren't often empowered to use authoring tools due to a lack of programming expertise. This thesis hopes to bridge that gap by creating an intuitive, user-friendly tool that still retains a satisfying level of complexity of output, which will allow for the creation of engaging interactive cultural experiences. With the goal of achieving this, the research will focus on answering the following question:

How can co-authoring tools enhance the collaborative creation of AR narratives?

In order to answer the question, the following objectives were put forward:

- Determine the optimal degree of freedom that should be provided to users (historians, curators, and tour guides).
- Design the user interface of the AR story editor to ensure usability for individuals without programming backgrounds.

The ultimate goal of this dissertation is to contribute towards the democratization of content creation, empowering users without programming backgrounds to engage in content creation seamlessly. Incentivizing inclusivity ensures that storytelling, multimedia expression, and cultural heritage preservation become endeavors open to a wider audience so that individuals from various

²<https://museusoaresdosreis.gov.pt/>

backgrounds can actively shape and share their narratives, fostering a richer, more representative cultural diversity.

This dissertation outlines the creation process of an interactive story authoring tool, accompanied by a story player designed to execute stories exported from the authoring tool. The goal of the application, alongside enabling non-programmer content creation, is also to enable co-authoring of narratives.

The application has some uses. The first and most important is to provide historians, curators, and tour guides the ability to create interactive cultural narratives. These narratives would be developed in co-creation environments, such as workshops. Another context in which the tool could be used is personal self-expression and collaborative story-building. Ideally, the tool will be generalistic enough to allow for content creation outside of its main domain.

In both cases, for the tool to provide value, the stories must be able to be built quickly, with a minimal learning curve. Therefore, the application must meet the following requirements:

- Enabling the creation and editing of interactive multimedia stories by non-programmers.
- The story editor must work in a web environment.
- The story editor provides co-authoring functionality.
- The story editor has a user-friendly design.

1.3 Methodological framework

This section details the methodology that was employed in the research process and in the development of the project.

Hevner (2007) divides design science research into three closely related cycles, as shown in Figure 1.1.

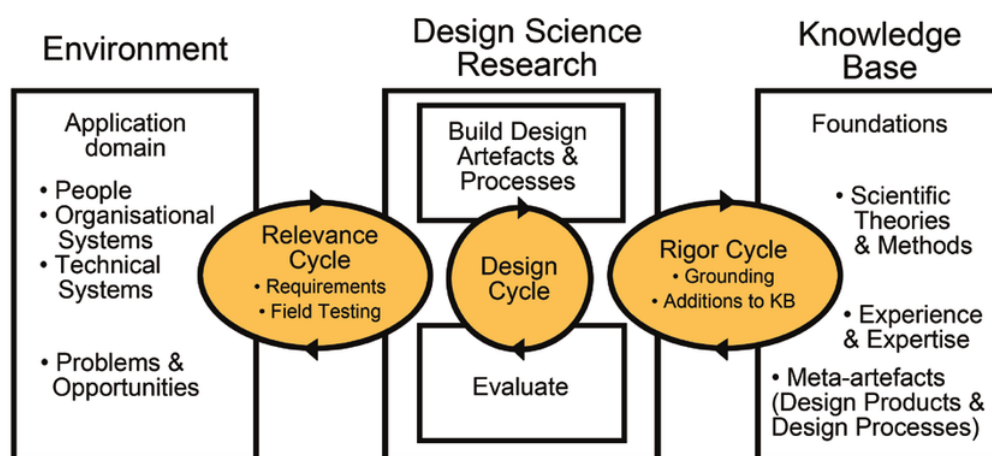


Figure 1.1: Design Science Research Methodology

The Relevance cycle in Design Science Research (DSR) serves as the foundational phase, commencing with the identification and representation of opportunities and challenges within a real-world application environment. Another key question at this stage revolves around assessing how the proposed design artifact contributes to improving the environment and, crucially, determining how this enhancement can be effectively measured and evaluated. In the context of the project, this translated roughly to the presentation of prototypes to the thesis advisors and stakeholders in order to receive feedback and enable further refining of the requirements. Ideally, this should also have encompassed the presentation of the prototypes to potential end-users such as historians and museum curators through co-authoring workshops. However, due to time constraints, this has only been possible in user testing, as part of supporting a designer in a co-creation workshop in a museum.

The Rigor cycle is essential for ensuring innovation and distinguishing the project from routine designs. It also encompasses additions to the knowledge base with the goal of keeping the state of the art as up-to-date as possible. Drawing from a broad knowledge base of scientific theories and engineering methods, researchers incorporate experiences, expertise, and existing artifacts from the application domain. This thorough research enables the development of a compelling and well-founded rationale for crafting the design artifacts (Hevner, 2007). In the context of this thesis, this phase of the DSR materialized in the form of the literature and systematic review conducted in section 2. The conclusions drawn from it provided guiding lines for the development of the application. Although strict application of the DSR methodology would mean that this phase should be iterative, due to time constraints, the review was only conducted once at the beginning of the process.

The Design Cycle is characterized by rapid iterations among artifact construction, evaluation, and feedback for refinement. Simon (1996) defines this cycle as a continuous process of generating design alternatives and assessing them against requirements until a satisfactory design is achieved. These requirements stem from the Relevance cycle, and design and evaluation theories are derived from the Rigor cycle. In the context of this project, this involves devising actionable solutions to meet the specified requirements and developing prototypes. These prototypes will then be thoroughly tested in controlled settings before undergoing subsequent evaluation during field testing during the Relevance cycle.

As Section 1.1 outlines, the work developed in this dissertation comes as a new iteration of an already existing project. However, due to the addition of new technology, Augmented Reality in this case, a new literature review and Design cycle were deemed necessary. For this dissertation, the starting point for the DSR Process Model was identifying the problem. This was achieved through the early meetings with the advisors and through the literature review described in 2. This allowed for the definition of concrete requirements that the application must meet to be deemed acceptable, thus creating the goals of the solution. Then, an application prototype was developed, which would be evaluated and validated in subsequent meetings with the stakeholders. From this evaluation, problems pertaining to the application would be identified, and based on them, new requirements would be defined. After recognizing possible solutions to these, a new prototype was

developed based on viable solutions, which would usually be ones that do not require excessive development costs. Afterward, user testing was conducted, and some refinements were integrated into the application. A new Design cycle could bring new inputs to the project, which will be presented as future work.

1.4 Document structure

In addition to this introductory section (Chapter 1), this dissertation has 5 other chapters. In Chapter 2, an evaluation of the current state of the art of authoring tools for interactive narratives is performed. The methods through which the papers were obtained are described in this section. An analysis of prominent features/design choices of these kinds of tools is performed, and their inclusion in the final tool is discussed. Chapter 3 describes the proposed solution, outlining the procedure that guided the research and development of the project. Additionally, this section defined the plan for the application's development. Chapter 4 describes the implementation of the proposed solutions, outlining the features present in the final products. Chapter 5 covers the testing of the developed applications with end-users, describing the methodology employed along with an analysis of the results obtained. Finally, Chapter 6 presents the conclusions of the overall work, answering the initial research questions and proposing some future work.

Chapter 2

State of the Art

This chapter describes the review process, along with the results from that process. Before the review process began, there were a few questions posed from prior meetings:

- What degree of freedom should users have (historians, curators, tour guides, etc.)?
- How can the user interface of the AR story editor be designed to ensure usability for individuals without programming backgrounds?
- How can we implement co-authoring tools to enhance the collaborative creation of AR narratives, and how do they contribute to the efficiency and creativity of the co-creation process?

The review process consisted of two sequential steps: a literature review followed by a systematic review. The goal of the literature review was firstly to provide additional context and to gain insight on the terminology of the field. Secondly, it aimed to establish the search terms for the systematic review. Thirdly, it sought to refine the inclusion criteria for the systematic review. Finally, it aimed to gather general information about various authoring tools. The systematic review's goal was to gather comprehensive information on a variety of authoring tools and their functionalities to ensure the novelty of this work and consider incorporating some of these functionalities. Figure 2.1 illustrates the process.

2.1 Literature review

Firstly, "axes" relevant to the search were defined. An axis is defined as one or more terms which convey the same idea. Additionally, these terms should be frequently used in papers so the searches result in as many relevant results as possible. For example, "Story" was defined as an axis, and its terms are "story", "drama", "storytelling", and "narrative". For the intent of this research and considering the concept, context, and perspective, I created the following axes:

- Story

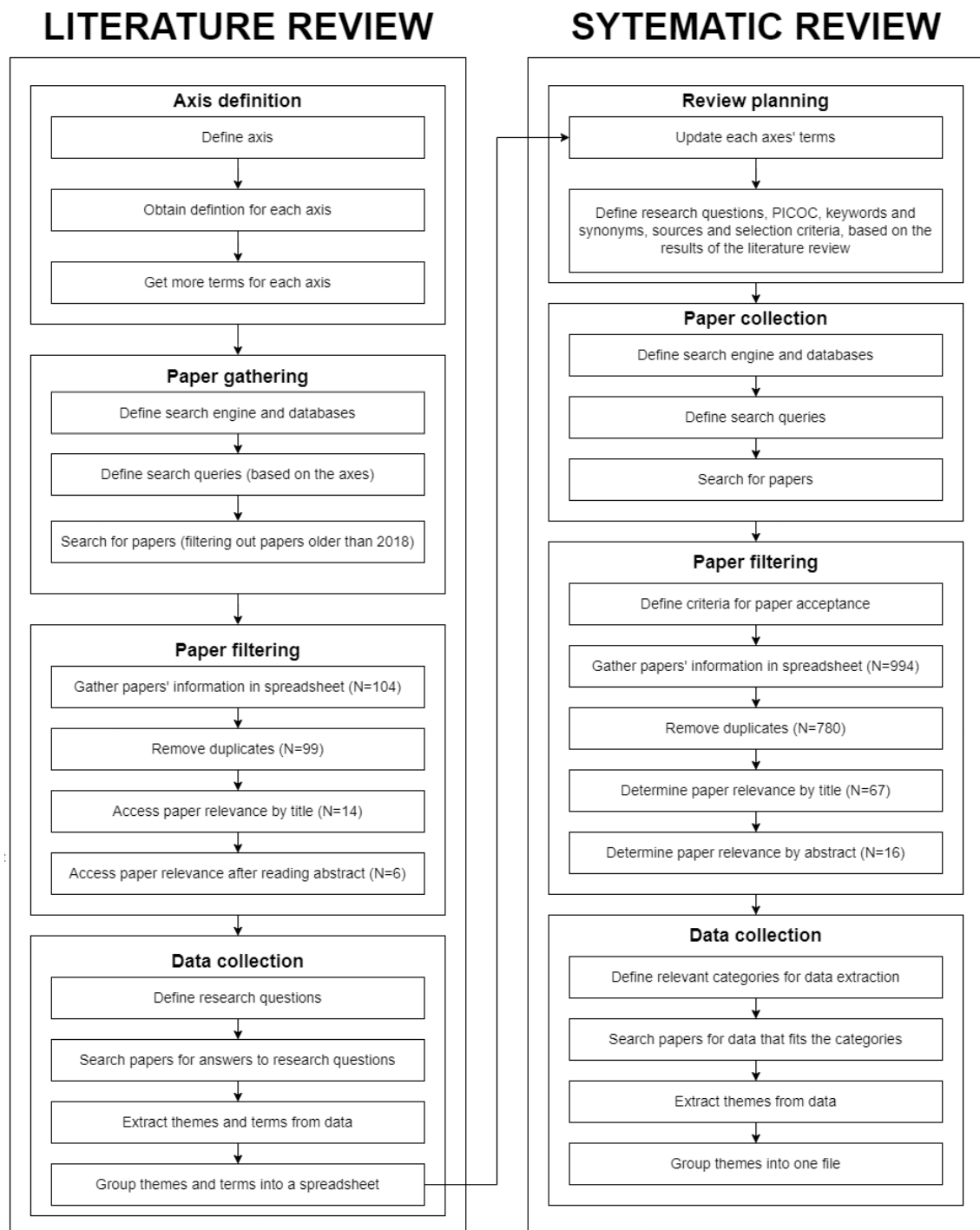


Figure 2.1: Diagram representing the review process

- Editor
- End-user
- Interactive

- Augmented Reality
- 3D

Following the creation of the axes, I searched for papers including definitions for each of the axis, which produced the following definitions:

- **Story** - Attempts to engage the player under the condition that the players' behaviors and actions can directly impact the story's direction and/or outcome (Riedl et al., 2011).
- **Interactive** - Interactivity refers to the degree to which users of a medium can influence the form or content of the mediated environment (Nakevska et al., 2017).
- **Editor** - A computer editor, in the context of software and computing, is a program or application that allows users to create, modify, and manage documents.
- **End user** - An "end user" refers to the individual or entity that ultimately interacts with and utilizes a product, service, or technology, typically for its intended purpose.
- **Augmented Reality** - A real-time direct or indirect view of a physical real-world environment that has been enhanced/augmented by adding virtual computer-generated information to it (Carmigniani and Furht, 2011).
- **3D** - "3D" stands for "three-dimensional." It is a term used to describe objects, spaces, or environments with three dimensions: length, width, and height.

The following is the list of terms defined for each axis. Note that this list does not include pluralized terms; however, the search queries use them, as the search engine used does not automatically include them:

- "Interactive" - "Interactive"
- "Story" - "Story", "Drama", "Storytelling", "Narrative"
- "Editor" - "Editor", "Creator", "Creation", "Modeling", "Authoring Tool"
- "3D" - "3D", "Virtual"
- "End user" - "End user", "audience", "non programmer"
- "Augmented Reality" - "Augmented Reality", "Mixed Reality", "Extended Reality", "Enhanced Reality", "Immersive Augmented Reality", "Interactive Reality"

In order to conduct the literature review, the software "Publish or Perish" ¹ was used. The search engine "Google Scholar" ² was deemed good enough.

¹<https://harzing.com/resources/publish-or-perish>

²<https://scholar.google.com/>

The whole search query had to be present in the paper's title for the literature review to improve their relevance and narrow down the results. Furthermore, the string "(survey OR review)" was included in every query since these types of documents were the target for the literature review.

Including every axis resulted in 0 results therefore the following queries combining the search terms were created:

- (story OR storytelling OR drama OR narrative) (editor OR editors OR creator OR creators OR authoring OR modeling OR meta-model OR tool) (survey OR review)" - 84 results (as of November 20th, 2023)
- (Interactive) (Story OR Drama OR Storytelling OR Narrative) (Editor OR Creator OR Creation OR Modeling OR Authoring Tool) (survey OR review) - 1 result (as of November 20th, 2023)
- (Story OR Drama OR Storytelling OR Narrative) (Mixed Reality OR Augmented Reality OR Enhanced Reality) (survey OR review) - 5 results (as of November 20th, 2023)
- (Interactive) (story OR storytelling OR drama OR narrative) (survey OR review) - 14 results (as of November 20th, 2023)

The results were then merged into a single spreadsheet. Initially, the corpus of documents had $N = 84 + 1 + 5 + 14 = 104$. After removing the 5 duplicates, there were 99 papers left. Next, all the paper's titles were read, and those deemed irrelevant to the context at hand were excluded ($N=14$). Finally, the remaining papers' abstract was read, after which the following 6 papers remained:

- Interactive Digital Narrative Authoring Tools and Hybrid Experiences in Cultural Heritage: An integrated review (Massidda et al., 2022)
- Define "authoring tool": a survey of interactive narrative authoring tools (Green et al., 2018)
- A systematic literature review on digital storytelling authoring tool in education: January 2010 to January 2020 (Quah and Ng, 2022)
- A review of agency architectures in interactive drama systems (Moallem and Raffe, 2020)
- Usability, user experience, and mental workload in a mobile Augmented Reality application for digital storytelling in cultural heritage (De Paolis et al., 2023)
- A systematic review of research undertaken into the use of digital storytelling as a pedagogical tool in the language classroom (Abderrahim and Cigerci, 2018)

From the remaining 6 papers, their relevant sections for this thesis were analyzed. In contrast, irrelevant sections (such as ones describing methodologies or answering questions out of the scope of this thesis) were discarded. A spreadsheet was created to organize information. It was divided among the categories of "AR Stories", "Authoring Tools Design Practices", "Interactive

Augmented Reality	Storytelling	Interactivity	Authoring Tool
hybrid technologies	creative process	immersive	digital storytelling environment
immersive environment	interactive fiction	branching story	digital creation
technology-enhanced	digital narrative	telling stories	design guidelines
		virtual puppetry	writing a script
		interactive comic	composing a digital story
			design framework
			story structure
			story plan
			assisting narration

Table 2.1: Terms obtained from the literature review

Story Creation Methods", "Authoring Tools Features", "Storytelling terms", "Interactivity terms", "Authoring tool terms" and "List of authoring tools". The list of authoring tools contained a name and a brief description of the tool.

The Storytelling, Augmented Reality, Interactivity and Authoring Tool terms were used as a way of expanding the terms for each of the corresponding axes for the systematic review phase. The collected expressions are enumerated in Table 2.1.

The literature review led to the conclusion of a "flowchart" system preference for visualization of the overall story structure in the editor, as it was consistently the most intuitive for the non-developer audience.

The following themes were also extracted by the end of this phase:

Virtuality of output

The virtuality of output, that is, for what type of environment the stories are projected, can be digital, the mix of physical and digital media/interactions, or a fully virtualized environment (Virtual Reality).

Authoring Tools Evaluation Categories

There are many possible categories that one may use in order to evaluate authoring tools, such as extensibility, end-user output (identifies what kind of output can be created using the tool), the degree of customization, user-friendliness (defines if the tool can be used easily by designers and creators without programming skills), accessibility, interoperability, availability (types of license or subscription needed; is the tool open-source or freely available), and compatibility (describes how well the tool integrates with other platforms such as AR/VR and digital archives as 3rd party services) (Massidda et al., 2023).

Authoring Tool Classification Categories

Three major authoring tool classification categories were discovered: Technology, Information Architecture, and UI/UX. The Technology category examines how different technologies are integrated with the experience, including aspects such as virtuality of output, display, interface, Internet of Things (IoT), collaboration, software output, and ownership. Information Architecture focuses on the complexity and abstraction level of the AT data structure exposed to the user, considering elements like multimedia, type of tool, level of abstraction, game mechanisms, and data

structure. Lastly, the UI/UX category describes the proposed content types, interactions involved, and navigation modes, with specific attention to engagement, user roles, and graphics.

Story Structure Display Methods

There are various approaches to Story Structure Display Methods. Image-based POI (Points of Interest) methods, such as those used in CurioPublisher, utilize images to highlight points of interest. Map-based POI methods, exemplified by tools like MapBox and ArcGis-StoryMap, use maps to pinpoint locations of interest. Mixed-widget stories, seen in platforms like HistoryPin and TumultHype, combine various widgets to create dynamic and interactive narratives.

Types of editing interfaces

There are multiple types of editing interfaces. Graph interfaces use visual representations to connect elements. Blocks or forms offer a modular approach, often utilizing block programming to create interactive content. Mark-up language interfaces provide a structured way to define content using tags. Text interfaces rely on traditional text input for creating and editing content.

End-user types

End-user types can mostly be divided into people with programming experience and non-programmers.

Presenting and editing story fragments

Presenting and editing story fragments can be done using various methods, including the most common flowchart approach, timelines (such as those in Movie Maker), slideshows (like PowToon³), and keyword-based systems (such as TextureWriter⁴).

UI/UX Liabilities

UI/UX liabilities include unrecognizable icons, missing basic undo/redo commands, unclear error messages, lack of debugging/simulation utilities, and insufficient guides, templates, and documentation. Additionally, the absence of customizable gestures, speech-based interactions, and input from external sensors further detracts from the user experience.

Interactive drama systems (by authorial control)

Interactive drama systems can have different degrees of authorial control. Possibilities range from allowing naturally occurring emergent features to shape the narrative, to providing agents with some autonomy while monitoring and adjusting their behaviors according to the story's goals, or using a tightly controlled "director" system to oversee the game's narrative course.

Types of interactivity

Interactivity can take the form of procedural story plot generation through natural language processing of user input or selection between multiple author-defined choices, as well as a combination of these two systems.

Authoring Tools Design Guidelines

Authoring tools design guidelines include ensuring age-appropriateness, maintaining consistent iconography, and incorporating a story planning system. The design should abstract computational technology, avoid overwhelming users with too much data at once, and include both

³<https://www.powtoon.com/>

⁴<https://texturewriter.com/>

pre-defined and self-authored multimedia. Promoting collaboration and addressing privacy issues with clear terms and conditions are also essential. Additionally, ensuring accessibility is a crucial aspect of the design.

Multi-step story creation process

The multi-step story creation process can involve either the use of multiple tools for each part of the story development or an all-in-one tool that integrates all aspects of the process.

Types of story fragments (for editing purposes)

Story fragments can include stages or scenes, such as sequences of actions within a game's virtual location, episodes, events, and characters. Additionally, other elements, like individual audio or video clips in a timeline-based video editor (e.g., Vegas Pro 11), can also be considered story fragments.

Authoring Tool features

Authoring tools were found to offer a range of features, such as the ability to export to different file formats, facilitate collaborative narrative creation, and publish finished content online. They often include robust error handling, highlighted syntax with autocomplete, and support for content duplication, shortcuts, and autolayout for automatic content structuring/tidying. Authoring tool functionalities also encompass content browsing, search and filter options, simulation, preview, and debugging capabilities. To support new users, tools usually provide example projects. Some ATs are capable of displaying and animating 3D models in AR, incorporating navigation maps, inventory systems, characters, game mechanisms, and dialog systems.

Interactive Story Creation Process

According to [Massidda et al. \(2023\)](#), some authors have reported being hesitant to condense the story creation and design process into an unfamiliar data-entry process. It was also reported that designing AR/VR story-driven experiences represented a complex challenge due to the unpredictability of the physical surroundings, directly influencing the user's attention span.

[Lambert \(2013\)](#) proposes seven essential components of digital storytelling to provide guidance for storytellers throughout the narrative construction process: point-of-view (identifying the main point of the story and the author's perspective to establish a clear narrative focus), dramatic question (formulating a central question that captivates the viewer's attention and is resolved by the end of the story, creating suspense and engagement), emotional content (addressing serious issues in a vivid and personal manner to evoke powerful emotions, fostering a strong connection between viewers and the narrative), voice (personalize the story to help viewers understand the context), economy (using adequate content to tell the story without overloading the viewer), pacing (managing the rhythm of the story to maintain a dynamic and engaging flow), and soundtrack (music or other sounds that embellish the story).

Power vs usability

According to [Massidda et al. \(2023\)](#), the power of an Authoring Tool refers to a relation between its breadth of potential (the possible number of configurations of its technological components) and depth of specialization. The breadth and depth are usually inversely proportional.

An AT's usability is impacted by its level of abstraction, the presence of helpers and utilities (error handlers and debugging), and the availability of example projects and documentation. The usability, which can be broken down into learnability and productivity, is strictly related to the AT's complexity. Complexity can be defined as the number and composition of abstracted values and structures exposed by the AT to the end-user.

While increased power allows for greater flexibility and capability in crafting intricate and sophisticated content, it often comes at the expense of simplicity and ease of use. As ATs become more intricate, users may encounter steeper learning curves and potentially experience reduced productivity due to the intricacies involved in harnessing the tool's full potential.

Conversely, a focus on maximizing usability by simplifying interfaces, providing ample documentation, and minimizing complexity, more often than not results in a reduction of the tool's overall power. Striking the right balance between power and usability is a central challenge in the design and development of authoring tools, requiring careful consideration of the target user's proficiency, the complexity of tasks to be accomplished, and the overall goals of the tool. In short, power and usability are usually at odds with each other.

2.2 Systematic review

This review serves as an update to the systematic review carried out by [Cardoso \(2023\)](#), although it has been expanded to include AR-related findings and papers from the year 2023. The systematic review was carried out with Parsifal ⁵.

The process started out by delineating specific elements, encompassing the definition of objectives, the application of PICOC (Population, Intervention, Comparison, Outcome, Context), the formulation of questions, the compilation of axes and their respective terms, the creation of search strings, identification of sources, the establishment of selection criteria, and the development of the data extraction form.

The objective of the systematic review was to investigate interactive story creation tools tailored for non-programmers that primarily used flowcharts to represent a narrative. The population was specified as "non-programmer end-users", the intervention as a "Flowchart story creation tool", the outcomes as "Identifying solutions employing flowcharts for story authoring", and the context as "Non-developer/programmer adults."

The following research questions were formulated:

- QSR1 - What are the main design guidelines for creating flowchart-based authoring tools?
- QSR2 - What are the main methods/technologies used in the creation of authoring tools employing flowchart visualization?
- QSR3 - What flowchart-based authoring tools are there?
- QSR4 - What features do these tools usually include?

⁵<https://parsif.al/>

- QSR5 - What limitations arise from the lack of programming experience of the end-user?

The list of axes and their corresponding updated terms obtained through the literature review, alongside the newly defined "flowchart" axis in Section 2.1, is as follows:

- "Interactive" - "Interactive", "Dynamic", "Responsive", "Participatory"
- "Story" - "Story", "Drama", "Storytelling", "Narrative", "Fiction"
- "Editor" - "Editor", "Creator", "Creation", "Modeling", "Authoring Tool"
- "Flowchart" - "Non-linear", "Branching"
- "3D" - "3D", "Virtual"
- "End user" - "End user", "audience", "Non-programmer", "no-code"
- "Augmented Reality" - "Augmented Reality", "Mixed Reality", "Extended Reality", "Enhanced Reality", "Immersive Augmented Reality", "Interactive Reality", "Hybrid Reality", "Immersive Environment"

These will be used to conduct the systematic review. The search was conducted using the "Publish or Perish" tool ⁶ and employing "Google Scholar" ⁷ as the search engine.

Since the goal was to go more in-depth with our research for this phase, instead of only searching for certain words in the title of papers, words that are included in the paper's keywords were also searched for.

The following queries, alongside their respective number of results, were used:

- **Title words:** (story OR storytelling OR stories OR narrative OR drama OR authoring) **Keywords:** (flowchart OR "non-linear" OR "branching") AND ("authoring tool" OR ((interactive OR dynamic) AND (editor OR tool OR creator OR creation OR creating))) AND ("no-code" OR "no code" OR "end-user" OR "end user") - 725 results (as of 10th December 2023)
- **Title words:** (interactive OR dynamic) AND (story OR storytelling OR stories OR narrative OR drama OR fiction) **Keywords:** (flowchart OR "non-linear" OR "branching" OR "dynamic") AND ((editor OR tool OR creator OR creation OR modeling OR "authoring tool") AND ("no-code" OR "end user" OR "end-user")) - 87 results (as of 10th December 2023)
- **Title words:** (interactive OR dynamic) AND (story OR storytelling OR stories OR narrative OR drama OR fiction) **Keywords:** ("Augmented Reality" OR "Mixed Reality" OR "Hybrid Reality") AND ((editor OR tool OR creator OR creation OR modeling OR "authoring tool") AND ("no-code" OR "end user" OR "end-user")) - 48 results (as of 10th December 2023)

⁶<https://harzing.com/resources/publish-or-perish>

⁷<https://scholar.google.com>

- **Title words:** (flowchart OR "non-linear" OR "branching") **Keywords:** ("no-code" OR "no code" OR "end-user" OR "end user") AND (interactive OR dynamic OR changing) AND (story OR storytelling OR drama OR fiction OR narrative OR authoring) AND (editor OR tool OR creation OR creator OR modelling) - 106 results (as of 10th December 2023)
- **Title words:** ("Augmented Reality" OR "AR" OR "Mixed Reality") AND (story OR storytelling OR stories OR narrative OR drama OR fiction) **Keywords:** ("no-code" OR "no code" OR "end-user" OR "end user") AND (editor OR tool OR creation OR creator OR modelling) - 28 results (as of 10th December 2023)

Some criteria for determining if a given paper was to be accepted were established:

- The paper was peer-reviewed.
- The paper was written in the English language.
- If the paper was about a specific authoring tool, it was only accepted if the tool used a flowchart visualization for editing purposes and the target user-base was non-programmer adults.

After merging the results from each search into a single spreadsheet, there were $N = 994$ papers. After removing duplicate papers, $N = 780$ (214 duplicates) remained. Next, the title of each paper was read, which resulted in the elimination of 713 papers. From the remaining $N = 67$ papers, the paper's abstract was read to judge its relevancy ($N = 16$). All the articles were accessible.

In order to effectively select and organize the information from the papers, data was extracted according to the following categories:

- Authoring tool story creation method(s) - 31 results
- Authoring tool usability concerns - 44 results
- Authoring tool names/identifiers - 78 results
- Authoring tool requirements - 22 results
- Authoring tool features - 177 results
- Limitations due to lack of programming experience - 8 results
- Augmented Reality concerns - 2 results

This information was organized in a spreadsheet with one page per category. Each of the collected excerpts was then simplified into a more concise and readable string to streamline the process of writing the state of the art. For example, the excerpt "While testing, current states of all game elements are directly accessible and editable." was shortened to "Edit and continue debugging".

2.3 Results

This section provides an overview of the discoveries from the systematic review.

2.3.1 Authoring tool story creation method

This subsection discusses findings relating to the primary methodologies employed in authoring tools for the creation or representation of stories.

2.3.1.1 Story representation

The reviewed papers revealed some variety in representing narratives inside of authoring tools. The most common way to represent a story is through the use of a graph, a data structure that consists of nodes (vertices) and edges connecting pairs of nodes. In this context, nodes usually represent the content of some part of the story, while the edges represent conditions for the transition from one part to the next, which are typically heavily dependent on user choices. Graph-based interfaces accounted for half of the reviewed papers. [Narawi et al. \(2020\)](#) argues that the story graph is the ideal data structure for representing non-linear interactive narratives: it allows for the possibility of repeating certain sections of the narrative, backtracking (returning to a past story point) and having multiple story paths (branching).

In [Erdem et al. \(2009\)](#), the nodes of the graph represent logic statements, and the links of the graph represent types of actions and flow of the logic state data. In [Vayanou et al. \(2014\)](#), a story is represented by "a directed graph decomposed into script units and script branching points".

[Verkuyl et al. \(2022\)](#) propose a comprehensive classification of virtual simulations compatible with stories, delineating distinct types based on their structures and functionalities. The linear simulation involves a singular navigation path with nodes representing steps or points in the simulation, offering limited interaction as learner choices don't influence the scenario's unfolding. Branched simulations progress from node to node, with outcomes varying based on learner decisions at decision points. Pseudo-branched cases limit possible paths, and some choices may lead learners back to previous nodes or merge paths. Exploratory simulations are open-ended, enabling learners to explore situations and make decisions, receiving feedback on their choices. Responsive simulations, also open-ended, are engine-powered, responding in real-time to learner actions. Communication-focused platforms use natural language processing for realistic interactions, while procedural simulations teach manual procedures in a linear or pseudo-branched scenario. This classification provides a nuanced understanding of the diverse approaches in virtual simulation design for educational purposes.

[Schenk et al. \(2021\)](#) propose a hierarchical story graph composed of two layers in order to provide "a view that is more visually consistent with the story model". This contrasts with the previous version of their proposed tool, which used a "text-based [...] indented tree-like structure".

Vrettakis et al. (2019) propose abstracting away the dynamic graph structure in favor of a more understandable hierarchical organization of the contents in story terms. In the context of this storyboard paradigm, they define some building blocks. Stories serve as comprehensive experiences, encapsulating all other layers. Chapters function to organize substantial and cohesive sections of the storyline, while scenes break down chapters into smaller, self-contained plot segments characterized by coherence in time, space, or characters. Pages, as self-contained components, house all user-experienced content, utilizing predefined screen templates to present the content effectively. This hierarchical layering facilitates a structured and organized approach to storytelling, enhancing the user's engagement and understanding.

Batsaras and Xinogalos (2023) discuss the Taleblazer⁸ authoring tool, which makes use of a drag-and-drop visual block programming interface. They also mention ARIS⁹, which features a built-in scripting language for the creation of complex branching narratives.

Green et al. (2021) discuss three authoring tools: Quest, which "makes extensive use of multifaceted user inputs for configuration and editing of the produced narratives", resulting in a control-based interface; Articy¹⁰, which uses the common dynamic node graph; and Inform which operates functionally as an augmented text editor. Furthermore, it mentions EAT's Visual Scenario Editor, which relies on a "node-based visual scripting system to create narrative experiences".

Transitions between different nodes can be triggered by a variety of events. Player choices account for the majority of transitions, as is the case in Not and Petrelli (2019). Schenk et al. (2021) propose a system where player choices can include simple explicit interactions such as presenting the player with a multiple-choice question, or it can be caused by the player performing an action at some story point. They also propose a system where a node can succeed, fail, or neither. Succeeding enables the immediate successors of that node, failing deactivates the current node, and neither signifies a neutral state where no action occurs, allowing for the possibility of revisiting that node. It is also possible for transitions to happen by random chance (Erdem et al., 2009).

2.3.1.2 Collaboration in story authoring processes

Laurent et al. (2022) mention the importance of adopting agile processes in order to cope with developing a story with a multidisciplinary team. Brett and Hargood (2023) also mention collaboration and co-authoring support as "essential to facilitate seamless work among multiple creators". Tools like Figma¹¹ allow team members to work together seamlessly in real time, enhancing teamwork and streamlining the collaborative design and creative process.

2.3.1.3 Digital design vs in situ creation

Brett and Hargood (2023) discuss the distinction between digital design (or desktop authorship),

⁸<http://taleblazer.org/>

⁹<https://fielddaylab.org/make/aris/>

¹⁰<https://www.articy.com/>

¹¹<https://www.figma.com/>

where a map is used as a reference to define the relevant locations and contents while users are working from a fixed workstation, and in situ creation, where the user is physically located in the place where the relevant locations and contents will be defined.

2.3.2 Story authoring features

This subsection contains findings about features present in authoring tools, ranging from essential core features to more optional ones.

2.3.2.1 Visual story authoring assistance

Torres et al. (2022) uses color-coded nodes to better communicate their functionality and address accessibility issues, as seen in Figure 2.2.

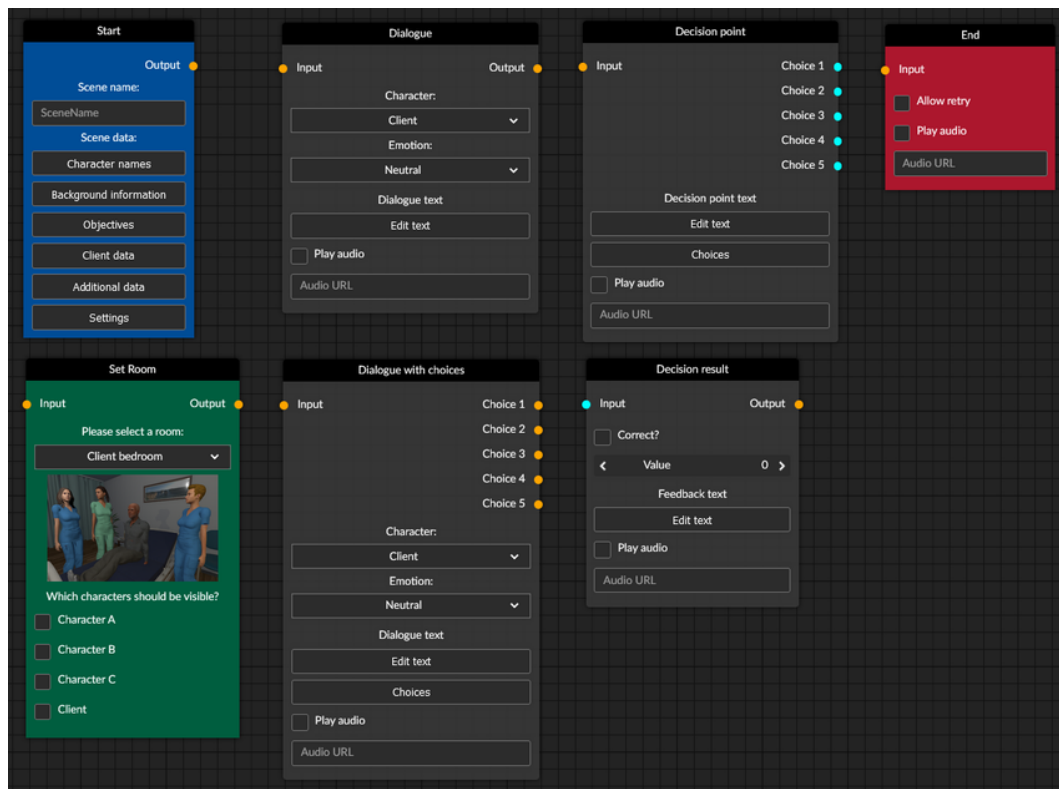


Figure 2.2: Moirai's color-coded nodes

A semantic level-of-detail zoom feature displays story elements with different amounts of information and icons depending on the level of zoom.

Winterbottom and Blake (2004) propose multiple visualization techniques to help users manage the complex set of interactions, including a timeline visualization for "time-based sequences of events and actions", floorplan visualizations to specify space-related events and sequence diagrams suited for narrative sequencing (cause and effect).

Göbel et al. (2008) mention how allowing the author to freely place the story graph's nodes in the editor simplified the author's understanding of their creations. This was further aided by

	Moirai	Storytec	Narrative	Articy
Semantic zoom		X	X	
Color-coding nodes	X			
Freely place graph nodes in the editor	X	X	X	X
Interactive overlay map of the story structure	X			
Allows uploading of background images		X		

Table 2.2: Visual assistance features in each authoring tool

allowing the upload of background images in the editor. Göbel et al. (2008) illustrate this by noting that, for a site plan, the author could organize the story’s scenes based on their geographic positions.

Laurent et al. (2022)’s proposed WEGAS system allows for different levels of abstraction for different users with different programming experience. This way, computer scientists are allowed greater flexibility at the cost of increased complexity while also letting users with less technical skills enjoy a simpler user interface. This has a greatly positive effect on the system’s versatility and usability. The different interfaces for users with distinct roles can be seen in Figure 2.3.

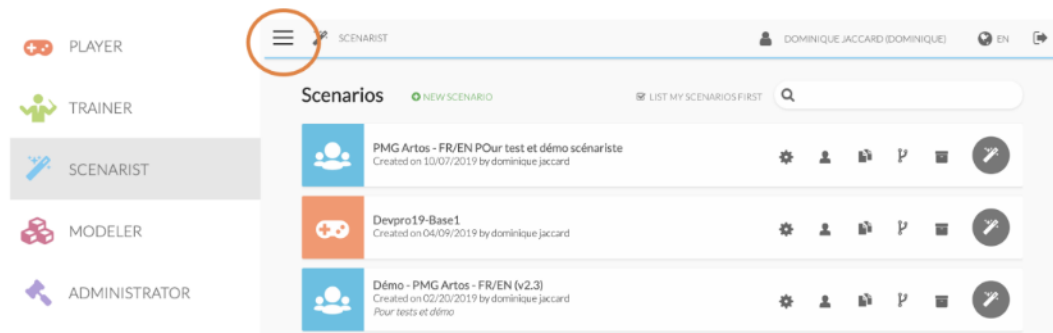


Figure 2.3: WEGAS’s role-based access. The left side shows how users can switch between the 4 roles

A compilation of various visual assistance features organized by authoring tool can be seen in Table 2.2.

2.3.2.2 Story debugging

The ability to debug or simulate the story is a very common feature of authoring tools, being present in nearly every single paper reviewed. However, this feature can take very different shapes across different tools.

Kauhanen et al. (2007) states the necessity of allowing story authors the ability to play through their stories from any given point, avoiding the time-consuming task of playing the story from the beginning. Green et al. (2021) similarly stresses the importance of allowing the user to return to a given state of the story as quickly as possible, enabling more rapid and focused testing sessions.

In Articy, testing is instigated by selecting a node and either using a toolbar button or the context menu of that node to enter into testing mode. This will fast-track the testing session to begin

at this particular point. It also features a Journey mode, which lets authors record pathways chosen so that not only can the state be restored but also the particular pathway repeated on subsequent debug sessions (Green et al., 2021).

In Inform, authors can use the 'Skein' feature, which shows a hierarchy of all actions taken during previous test runs with visual branches indicating where paths differ (Green et al., 2021). The Quest authoring tool features clickable links for interactivity during the testing mode, which helps authors get back to a given state quicker and makes exploring easier, especially when commands are unknown.

Laurent et al. (2022) propose a debugging system with the ability to observe the current game state via an interactive preview, with current states of all game elements being directly accessible and editable. Additionally, it also has the capacity to identify errors and connections between story objects using a search engine.

The authoring tool mentioned by Erdem et al. (2009) in their paper has a separate testing and debugging tool that allows for the validation of the design of the non-linear narratives. It also offers the capability to export scenarios to the testing and debugging tool, irrespective of their completion status. Furthermore, the testing tool allows authors to play a random branch from the story.

Winterbottom and Blake (2004) assert that the tool outlined in their paper is designed to provide feedback on the structure of generated stories. This includes the identification of gaps or deficiencies in the sequence of interactions within a narrative.

2.3.2.3 Core authoring tool functionalities

In authoring tools that employ nodes as a means of story visualization, most of them mention the use of specific types of nodes, most commonly start and end nodes. Most tools, as seen in Narawi et al. (2020) allow for defining a singular start node, multiple end nodes, and scene-changing nodes. They also define repetition nodes which can loop over certain parts of the story. Torres et al. (2022) also mentions special nodes that present choices to the player and additional nodes that handle the player's choice.

Crucial authoring tool features include the ability to create a new story (Torres et al., 2022), the ability to browse/search through a list of all stories (Erdem et al., 2009), the ability to load an existing story, the ability to save/export a story, and a basic undo/redo feature (Göbel et al., 2008).

Differing versions of save/export functionalities were present across the reviewed papers. Laurent et al. (2022)'s WEGAS platform offers the possibility to export content in standardized file format such as Portable Document Format (PDF) or open data interchange format such as JavaScript Object Notation(JSON). Erdem et al. (2009) mention the use of a relational database to store both intermediate and output data. Schenk et al. (2021) discuss utilizing files for storing the story scenes. It further specifies that these files are tailored to a game-dependent tool.

Batsaras and Xinogalos (2023) refers the following functionalities as fundamental for location-based games authoring tools:

- Non-linear authoring: opportunities for event handling and setting corresponding triggers, ensuring the possibility of different experiences every time the game is played
- Visual authoring: allow for people with little to no programming knowledge to create experiences
- *In situ* authoring: designers can be physically present on-site with the use of a mobile device
- Re-use and re-editing
- Content adding and managing
- GUI customization: ability to alter the program's interface to suit the author's preferences
- Simulation/Debugging mode
- Game analytics: allow access to game performance statistics and player behavior
- Map authoring: represent in-game events directly on the map

2.3.2.4 Additional functionalities

[Laurent et al. \(2022\)](#) details the importance of enabling incremental game development with versioning possibilities. They also propose that authoring tools should enable the creation of generic and parameterizable high-level components and, therefore, encourage reusability.

[Brett and Hargood \(2023\)](#) also mention the importance of a versioning system, a "history" feature that allows other authors to track project changes over time. They also stress the importance of tagging parts of the story to allow easy access at any time. [Kauhanen et al. \(2007\)](#) additionally introduces the relevance of keeping track of versions of resources and enforcing the distinction between a template and an instance of the template (an object). Furthermore, they suggest incorporating a mechanism for monitoring the components within an exported resource. They claim that it "supports reuse, portability, and the ability to share components of stories".

[Vrettakis et al. \(2019\)](#) details an authoring tool with both content display and story branching templates. It also allows the user to download experiences crafted by other users. [Not and Petrelli \(2019\)](#) mention providing simple story structure templates (also referred to as 'recipes') with tutorials in order to invite experimentation and build confidence with the AT. These recipes would be complemented by traditional search filters. Additionally, they suggest a recommendation algorithm for recipes, informed by the profile of the author. This profile can contain parameters such as indoor vs outdoor, size, permanent collection vs temporary exhibition, type of heritage institution, etc. Similarly, [Abawi et al. \(2004\)](#) mentions the inclusion of story structure templates, which follow logically from the ability to replace individual components in a story with others without making changes to the overall story structure. They state this functionality allows for the creation of story patterns by storytellers, which are tailored to provide a particular structure, course of action, or suspense.

Several papers mention the importance of allowing the import of custom assets, in the form of 3D models, images, audio, and video, among others. Two different import methods were found: [Torres et al. \(2022\)](#) allowed for audio imports via URL, while [Göbel et al. \(2008\)](#) used file names. [Laurent et al. \(2022\)](#) allows for the definition of high-level composite components made out of basic components, which can be exported and imported to enable reuse and sharing. [Brett and Hargood \(2023\)](#) also defend that an authoring tool should provide stress-free importing of images, illustrations, and videos. Furthermore, they state that it should also allow the importing of libraries of user-generated content. Another principle they defend is that created artifacts should be easily exported and accessed from within the authoring tool in order to facilitate reusability.

[Schenk et al. \(2021\)](#) describes the ScriptEaseII authoring tool, which features a translation feature, allowing for compatibility with any game engine or API, as long as a custom translator is built for it. This implied that the stories crafted in the authoring tool are not restricted to a specific platform. The basic story-creation process in this tool involves users establishing connections between diverse actions and conditions to formulate sequences. The users are then able to use an annotative flow chart to select desired behaviors that occur when an event is fired on an object. Furthermore, they state their intention to implement story groups for further organization of the story graph. [Schenk et al. \(2021\)](#)'s ScriptEaseII also features the ability to delay the presentation of story contents for a specified duration. Additionally, this work introduces the possibility of using so-called implicit elements, which allows the author to define behaviors for dynamic objects that will be defined at playtime and will differ depending on which choices the player makes during the course of the experience.

[Vayanou et al. \(2014\)](#) mention supporting the creation of narration for the interactive narratives. [Torres et al. \(2022\)](#) mention allowing the setting of a time limit for the user to choose an answer as future work.

The ability to drag objects onto an area map and edit properties of story elements is a common feature among the reviewed papers, being mentioned explicitly by both [Göbel et al. \(2008\)](#) and [Kauhanen et al. \(2007\)](#). Property-editing of components is also present in [Abawi et al. \(2004\)](#)'s toolkit, which allows for on-the-fly adaptation of components without exposing the author to all the complexities of the component's inner workings.

[Torres et al. \(2022\)](#) encompassed the feature to optionally utilize a text-to-speech program to play a segment of dialogue. Some authoring tools, such as TaleBlazer ¹² or [Schenk et al. \(2021\)](#)'s ScriptEaseII, feature the ability to define entity/game behavior through a visual block programming interface, as seen in figure 2.4. [Erdem et al. \(2009\)](#) similarly includes a "graphical source editor" that allows the creation, modification, and viewing of programs through mostly editing of graphical language components in a canvas instead of typing code into text files. These belong to the "low-code" subset of authoring tools.

[Laurent et al. \(2022\)](#)'s WEGAS platform provides an automated backup system to recover intermediate versions of the developed games. Some systems propose retrieving user information from the players in order to create "user profiles" which would then have some sort of impact on the

¹²<http://taleblazer.org/>

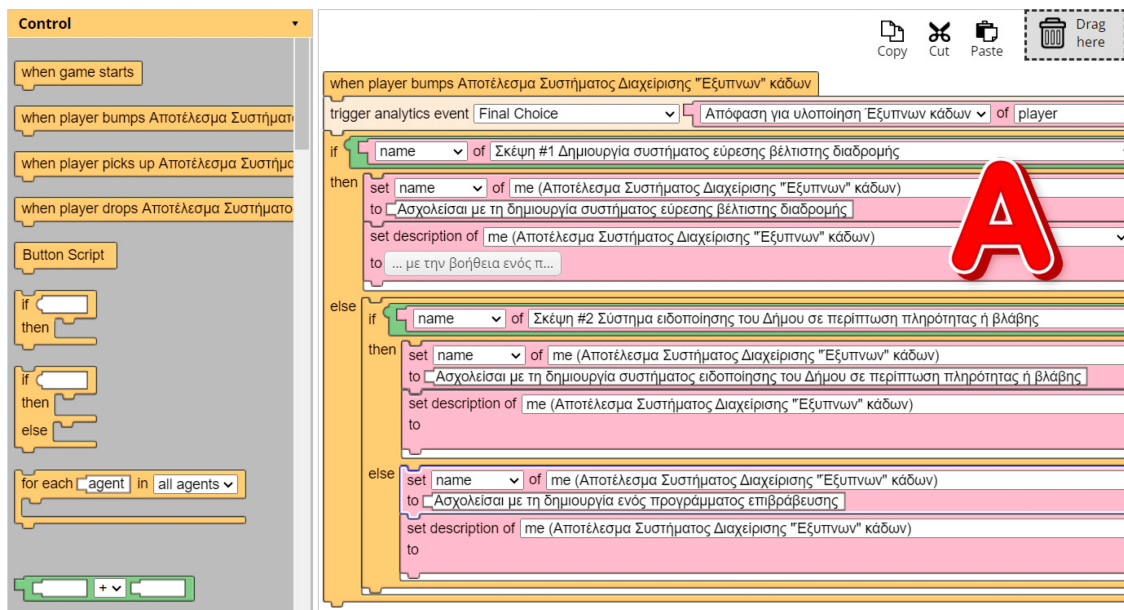


Figure 2.4: TaleBlazer's block programming interface

course of the interactive experience. Economou et al. (2008) introduces the idea of personalizing the provided information to the visitor's interests and experience. It further promotes the retrieval of visitor's feedback in order to enhance the experience. The CHESS authoring tool, described in Vayanou et al. (2014), further expands on this idea, implementing generic visitor tracking and dynamic profile update techniques and refining the visitor profile as an experience progresses. Additionally, it injects specific menus into the experience to ask users questions with the goal of further enhancing profiling accuracy. The profile is then subsequently used to make an implicit decision when a player arrives at a story branch. Authors can leverage this system by annotating the story graph with a set of weighted features.

In Ganapathi and Gokhale (1995), the utilization of a glossary was mentioned, connecting variable names to corresponding entity names displayed onscreen. Economou et al. (2008) recommend that the authoring tools be able to support pre and post-interactions with visitors. Vayanou et al. (2014)'s CHESS authoring system implements this suggestion, allowing users to browse information about a museum or play digital games related to its content before visiting and making an overview of the visitor's experience in the museum available afterward. Additionally, the CHESS system allows authors to define soft or hard constraints over the branches, depending on factors such as visitor choices, previous actions, visitor features, and location.

Vrettakis et al. (2019)'s Narralive defined menus of type 'Choice', which represent a unique turn of events in the plot, and menus of type 'More', which provide the user with extra optional content. Narralive also features live component preview. It also proposes player interaction-enhancing features, such as the possibility for players to add their own photos and their comments at specified points in the visit. Furthermore, they cite their intention of allowing authors to create timed events that will take place during the experience at pre-designed points in time or after

another specific event has taken place. Additionally, it includes a story design tool combining a text-editor look and feel with a graph-based visualization.

Economou et al. (2008) recommends that authoring tools allow authors to guide the player by specifying a particular order to experience the exhibition. The izi.TRAVEL CMS ¹³ gives the author the flexibility to define story points either in a linear sequence with a predetermined order or as an experience where the user can freely visit each one of them.

Vayanou et al. (2014)'s CHESS authoring framework and many other location-based games authoring tools, such as Locatify ¹⁴, allow the author to load and edit a customized map of a particular site.

Verzini et al. (2015) state the importance of allowing for different methods of defining real-world locations, such as GPS or QR-code scanning, allowing for the authoring of both indoor and outdoor location-aware experiences. Their proposed authoring tool also highlights the importance of decoupling the content layer from the location layer, which enables their reusability independent from each other. The authoring tool lets users build stories in terms of Points Of Interest (POI), which are defined when content is associated with a location.

Batsaras and Xinogalos (2023) similarly recommends that a wide variety of technologies should be available to track players, such as GPS, QR codes, Bluetooth beacons, augmented reality (AR) markers, and RFID. They also state that players' actions and creations within the game should be stored and organized in order to facilitate later group discussions, feedback, critical thinking, and conclusions about the experience.

Granting players points and badges in order to reward goal achievement and task completion is also mentioned in their work, referring to a feature of the ARIS ¹⁵ platform. They further state that Locatify ¹⁶ offers multilingual support, expanding its possible users.

They also mention the need to allow multiplayer experiences where players are able to communicate among themselves and collaborate to progress. Additionally, they mention that experiences must work across various operating systems in order to broaden the audience. They further recommend that offline experiences should be offered to players, allowing connectionless experiences. In the same vein, **Economou et al. (2008)** states that experiences should be able to adapt to a lack of network connectivity. They also recommended that authoring tools support multiple languages for accessibility purposes.

A compilation of various additional features organized by authoring tool can be seen in Table 2.3.

2.3.2.5 One tool vs many tools

Laurent et al. (2022) defends that a tool should ideally encompass the entire story authoring process, from design and development to the distribution and usage, while **Göbel et al. (2008)** de-

¹³<https://www.izi.travel/>

¹⁴<https://locatify.com/>

¹⁵<https://fielddaylab.org/make/aris/>

¹⁶<https://locatify.com/>

scribes a pluggable framework composed of five different tools: The Story Editor, the Stage Editor, the Action Set Editor, the Property Editor, and the Asset Manager. The 'pluggable' aspect of the framework makes it possible so that different editors can be used for different applications, as they may have different needs.

2.3.2.6 Context-specific features

[Erdem et al. \(2009\)](#) mentions a face animation tool that provides for the automatic generation of lip and emotion animation, using recorded speech as input.

[Ganapathi and Gokhale \(1995\)](#) emphasize the incorporation of simple text-editing features, including the ability to delete words or lines and automatic word wrapping. [Torres et al. \(2022\)](#) features rich text formatting.

[Abawi et al. \(2004\)](#)'s proposed toolkit lets the author include a placeholder object, such as an image loader, a geometry loader, or a text component in a scene, in order to display multimedia content. They affirm its utility lies in permitting a quick exchange of a component's visual representation without removing the component itself from the scene. Additionally, the authoring kit provides the possibility of defining story branching conditions with the user's distance to a certain object. Furthermore, in order to better support mixed-reality authoring, the ability to align virtual objects such as text, video, annotations, or 3D geometries, with reference points in real space, generally referred to as "calibration", is also provided by their toolkit.

[Erdem et al. \(2009\)](#) discussed incorporating a real-time rendering engine into an authoring tool. This enabled authors to design 3D environments with a modeling tool and utilize the same tool to render them in real-time during playback.

In [Not and Petrelli \(2019\)](#), the incorporation of IoT (Internet of Things) devices was explored as a means to seamlessly integrate physical interactions into a virtual narrative. The authors proposed a user-friendly approach to IoT utilization, advocating for the abstraction of the setup of interconnected devices. This involved configuring parameters for sensor triggers, thereby abstracting the sensor layer and simplifying the debugging process during testing. Additionally, they outlined the capacity to associate multimedia elements with physical objects through either menu selection or drag-and-drop action.

[Torres et al. \(2022\)](#)'s Moirai provides file size reduction methods in order to prevent users from having to download excessively large files over the Internet. Firstly, it reduces texture file sizes, capping their maximum resolution at 512 x 512 pixels. Secondly, they make use of low-polygon count models and thirdly they constantly monitor build reports to identify unused components and areas that need to be optimized.

[Economou et al. \(2008\)](#) mentions how limitations of memory and processing power of mobile devices might be able to be overcome through allowing users to disable certain features such as higher resolution images and dynamic frame movement.

	Moirai	Storytec	Narrative	ScriptEaseII	Locatify	meSch	CAT	TaleBlazer	ARIS	ActionBound
Import custom assets	X	X	X		X	X	X	X	X	X
Rich media (AR, games, 3D)			X				X			
Search filter (story components)				X			X			
Export story components								X		
Text-to-Speech	X				X					
Rich text formatting	X									
Publishing			X			X	X			
Block programming interface				X				X		
Semantic annotations (tagging)						X	X			
Templates (content display)			X				X			
Templates (story structure)			X	X		X			X	
Content duplication (copy/paste)			X							
User Data collection						X	X		X	
UI customization						X		X		
Automated versioning								X		X
Help/Tutorials					X	X		X		X

Table 2.3: Additional features in each authoring tool

2.3.3 Accomodations due to lack of programming experience

The systematic review focused exclusively on authoring tools developed for people without programming experience. Authoring tools are quite complex applications, therefore, measures must be taken in order to abstract away the more complex inner workings of ATs in order to cater to non-developers. This subsection will cover those methods and describe how they minimize issues.

2.3.3.1 Managing assets

Vayanou et al. (2014)'s CHESS tool recognizes the importance of ensuring easy asset management by authors, enabling them to "browse, search, and display their project digital assets as well as to visualize the relations between them through a graphical, ontology-like diagram (the generation of which is based on the provided tags)". Additionally, the tool features an integrated Asset Manager Tool that enables authors to upload their own digital content with associated metadata, like copyrights and tags, contributing to the protection and reusability of online assets.

Brett and Hargood (2023) mention how the presence of comprehensive user-generated libraries and the capacity to integrate user-generated content additionally nurtured creative autonomy. Kauhanen et al. (2007) additionally stresses that this common library of resources preferably allows the addition of custom/edited content. Laurent et al. (2022) introduces that the components belonging to this library should be inferred based on usage. This feature should assist the author in keeping track of the various assets in their project and their different versions if they exist (Kauhanen et al., 2007).

2.3.3.2 Authoring tool terminology

Kauhanen et al. (2007) suggests incorporating language and concepts from the storytelling domain into the design of story authoring tools, emphasizing that the terminology chosen significantly in-

fluences the end user's comprehension of the application. They specifically recommend employing straightforward terminology for commands and error messages, proposing the adoption of terms associated with story creation. For instance, they suggest using the term "Go" instead of the more technical "compile-run". **Not and Petrelli (2019)** also recommend avoiding technical jargon.

2.3.3.3 Error messages

It is not only crucial for error messages to be easily comprehensible for end-users, but they should also be non-intrusive, recognized as an integral aspect of the development process, as emphasized by **Winterbottom and Blake (2004)**. For instance, rather than employing a pop-up error approach, errors functioned as status indicators, akin to compiler warnings (**Winterbottom and Blake, 2004**).

2.3.3.4 Educational material

The availability of easy-to-understand educational material in a wide variety of formats can provide a giant boost in the usability of authoring tools. **Batsaras and Xinogalos (2023)** mention how the Locatify platform ¹⁷ provides extensive educational material in the form of documentation, manuals, and video tutorials. **Vrettakis et al. (2019)**'s Narralive provides textual help and video tutorials to assist users in using templates.

User-friendly onboarding processes featuring thorough tutorials and sample projects were identified as essential elements in enabling designers to swiftly comprehend the software and promptly prototype their ideas by **Brett and Hargood (2023)**.

2.3.3.5 Community support

An active development community is often the best way to tackle very specific user difficulties. When discussing the various authoring tools in their study, **Batsaras and Xinogalos (2023)** mention the existence of a digital gathering/discussion space (forum) and high community member counts as crucial to support inexperienced users.

2.3.4 Usability concerns

Some usability considerations may be extracted from **Not and Petrelli (2019)**'s usability survey, in which users complained about a lack of system status visibility, with users highlighting problems like insufficient feedback, particularly after clicking "Save" in the "Content" editing panel, leading to a lack of clarity about system status. Additionally, users expressed concerns about inconsistencies in user interfaces across different tabs, emphasizing the importance of adhering to standards. The non-persistence of relevant information was also called out by users, citing that relevant menu information would disappear with scrolling, impacting the user experience. **Not and Petrelli (2019)**'s feedback also expressed user satisfaction with the platform's immediacy -

¹⁷<https://locatify.com/>

changes were reflected instantly. In the same vein, Economou et al. (2008) and Brett and Hargood (2023) also recommend providing visual feedback in response to user input.

Brett and Hargood (2023) emphasize small compilation times are a must so that tools don't get in the way of the story authoring process.

Laurent et al. (2022) stresses the importance of platform independence, advocating for the use of web technologies in the process of building authoring tools.

2.4 Discussion

This section will discuss the findings present in section 2.3. The likelihood of each feature being present in the final product is discussed based on the current understanding of end-user requirements obtained during the initial meetings and considering the potential benefits against the cost of implementation. The applicability of each feature in the project's context is also considered as a factor.

2.4.1 Authoring tool story creation method

2.4.1.1 Story representation

Among all the possible representations, the story graph emerges as the one most suited to this project. It works quite well with the non-programmer audience as the concepts of nodes within the graph denoting distinct states of the same narrative and edges representing choices available to the player are quite intuitive ways to represent a branching story structure. The story graph data structure also enables easy backtracking to a previous point in the graph, which could be important for certain types of stories.

The story graph is considered to be a solid visual story metaphor to represent story structure and connectedness, which according to Green et al. (2021), will result in less testing of non-complex stories, speeding up the authoring process. They also state that the familiarity of the metaphor should encourage more experimentation with the story structure and the creation of more branches, fostering the creation of truly non-linear experiences.

2.4.1.2 Collaboration in story authoring processes

Adopting agile methods could prove quite useful for the authoring process, however, this is dependent on the final end-users which haven't been communicated with yet, rendering the importance of this unknown.

The adoption of a synchronous collaboration method, akin to tools like Figma¹⁸, seems interesting and would be greatly beneficial in the context of the authoring environment that the application will be built for - co-authoring workshops. The ability to have multiple users editing in one session would provide quite a boost to creativity and productivity and, therefore, should be highly considered, although coming at a pretty high development cost.

¹⁸<https://www.figma.com/>

2.4.1.3 Digital design vs in situ creation

The question of digital design vs in situ creation is surpassed by the fact that the application will be developed using web technologies, meaning that it will be usable on any device with access to a browser, therefore including mobile devices. This means that it will allow both digital design from a remote workstation and *in situ* creation through the use of a mobile device.

2.4.2 Story authoring features

2.4.2.1 Visual story authoring assistance

Color-coded nodes are a very useful feature and cost very little in terms of implementation time, so they are highly likely to be implemented in the final application. Accessibility issues regarding people with color blindness will have to be addressed by ensuring that easily distinguishable colors are used.

The capacity to zoom in and out within the story graph view was crucial for aiding the author in visualizing the entire narrative quickly, with the advantage of being easy to implement. Implementing a semantic zoom feature, which would display story nodes with lower detail when more zoomed out, posed the challenge of requiring a longer development time with potentially lower benefits. Consequently, a non-semantic zoom feature was deemed more likely for implementation.

The ability to freely rearrange nodes within the story editor is very likely to be present in the final application as it provides a very significant boost to the author's understanding of their stories and comfort with using the authoring tool.

The background image uploading feature posed the challenge of requiring saving images alongside their position/rotation/scale, which would increase load/save times and drive file sizes up. However, given that the stories created will probably be played in the real world, in museums/-cultural heritage places, the advantage of having a background image to better plan stories around real world locations was judged to outweigh the implementation challenges and will therefore likely be implemented.

The idea of allowing for different levels of abstraction in the authoring tool for different kinds of users is a very positive one, providing users with more expertise with more power. However, this feature comes at a high development cost, and the end-users for this particular project are not considered to be diverse enough to justify the implementation of such a feature.

The presence of an overlay minimap would allow authors to more easily comprehend their overall story structure and provide a massive boost to the capability of navigating within the editor, speeding up the authoring process. It is a potentially useful feature to implement, but it will depend on the end-user requirements, more specifically, the complexity of stories intended to be built with the tool.

2.4.2.2 Story debugging

The tool will definitely host a story testing/simulation feature designed to be easy to use for a non-developer audience.

The ability to play the story starting at any given point would enable faster prototyping, avoiding having to start from the beginning every time. Therefore, it would be ideal to implement as it shouldn't come at a significant enough implementation cost.

Articy's Journey mode, with its recording of chosen pathways, allowing for repeated testing of the particular story branch, was considered too expensive to implement, given the time available. Inform's 'Skein' feature, which shows users a visualization of all actions taken during previous test runs, including differing branches of the story, is quite useful. However, its upsides weren't considered worth the development cost. Quest's clickable links for interactivity in its debugging mode are very advantageous for letting users get back to a given story state quickly. However, its usefulness is lessened if we consider that the ability to start testing at any point in the story is implemented. Therefore, its implementation is not needed.

The ability to monitor the state of the story while debugging has a very valuable upside, so its implementation is quite likely. The capacity to edit said states while in the testing mode comes with a substantial added development cost, and for this reason, it will probably not be featured in the final version of the application.

The feature of providing feedback on the structure of stories is very advantageous. The editor should warn users of component combinations that would generate invalid story states/structures, for example, gaps within the narrative. The warning messages should be made non-intrusive and their language must be end-user friendly. This feature's usefulness greatly outweighs its development cost, so it should be featured in the tool.

The ability to perform random scenario testing could potentially be beneficial, but this depends on how hard dead-end scenarios would be to spot in the final design of the authoring environment and on how complex the stories created with the tool are.

2.4.2.3 Core authoring tool functionalities

The authoring tool will allow you to define one start node and multiple end nodes. Scene-changing nodes that trigger a change in the setting mid-story were deemed inappropriate for the context, given that the stories are meant to be experienced in the physical world, so they will not be implemented. Repetition and choice nodes seem appropriate for the context at hand, and their implementation is necessary to fulfill the requirement of being able to author non-linear stories using the tool, so they are quite likely to be featured in the final version of the tool.

The abilities to create a new story, load an existing one, and save/export will definitely be present in the tool, as they are absolutely necessary for the basic functioning of any authoring tool. The stories are likely to be exported in JSON, seeing as it is a widely supported data interchange format, on top of being a human-readable format. An undo/redo feature is also very likely to be implemented, as it has come to be expected by users from any modern editor, and it plays a crucial

role in accelerating narrative creation. The ability to browse through a list of every existing story would be quite easy to implement, but it implies that stories would be uploaded to the internet and hosted, which requires the development of additional database architecture, so its implementation depends on the importance placed on the online publishing of the narratives, which is currently unknown and will necessitate further refining of the end-user requirements.

2.4.2.4 Additional functionalities

The inclusion of a search filter for story components is a possibility; however, its practicality is contingent on how the authoring tool is implemented and the breadth of stories it aims to support. Consequently, determining the usefulness of this feature is challenging and depends on various factors, making it uncertain whether it is worthwhile to implement.

The provision of story structure templates is of great importance as it allows the author to focus solely and exclusively on their core domain and competencies without needing to create an underlying story structure from scratch, as [Abawi et al. \(2004\)](#) state, and therefore they are likely to be implemented. Traditional search filters should accompany these recipes as they speed up the selection process and are relatively easy to incorporate. A more complex recommendation algorithm for recipes is outside the scope of this dissertation, as the variety of available recipes to choose from will not be significant enough to justify its inclusion.

The inclusion of content display templates should also be heavily considered, as it does not require significant development effort and comes with the upside of allowing authors to not start from scratch when developing content, providing inspiration and streamlining the content creation process.

The ability to create generic and parameterizable high-level components is likely more useful for users with some programming experience, who are more familiar with the concept of modularity, offering less payoff for non-developers, who are less likely to engage in the creation of these generic components, and therefore it is not likely to be implemented. The ability to tag certain parts of the story is quite easy to implement and provides a significant upside, so it is likely to be present in the final product.

The capacity to import custom assets in the form of 3D models, images, audio, video, and others will be present in the tool, as it has already been established as part of the requirements. Supporting both URL and file-name imports should be feasible in terms of development cost. The ability to import components from a library of user-generated content will not be present as there won't exist a community of dedicated active users around the tool, so it is considered outside the scope of the dissertation.

A translation feature allowing for compatibility with any game engine or API is outside the scope of the present dissertation. The ability to group certain components into story groups is easy to implement, and it has the major upside of allowing authors to organize their stories to their liking, improving their comfort with the tool, so it is likely to be incorporated.

The ability to create narration will likely be present in the final tool, either in explicit or implicit form. The capacity to delay the presentation of story contents for a duration is trivial to implement

and is, therefore, likely to be made available to users. The ability to set a time limit for the player to make a decision is similarly quite trivial to create, so it should also be featured.

The ability to define implicit elements dynamically assigned at play-time is interesting, but it is quite context-dependent, so its priority of implementation is very low. Likewise, the ability to use text-to-speech is dependent on end-user requirements, which require further refining, so its priority is currently unknown.

Visual block programming interfaces were deemed unfit for this project. Such "low-code" approaches would likely have a negative effect on the curators and historians, who would find such an interface unfamiliar and daunting, worsening the tool's learning curve. Instead, a drag-and-drop interface featuring many different components to account for a variety of behaviors will be implemented.

An automated backup system that recovers previous versions of the stories would take more resources than the ones available to develop this project. Similarly, the creation of user profiles through data gathering and dynamic profile update techniques is far beyond the scope of the project as it would require a significant infrastructure to implement.

The creation of a versioning system requires a lot of development time which is likely outside the realm of possibility for this dissertation. The ability to support pre and post-interactions with visitors could potentially be achieved. Allowing users to browse information about a museum before the visit would be easy to achieve. Allowing for an overview of the visitor's experience in the museum comes at a significant development cost and lies outside the scope of this project. These interactions are dependent on how the final experience is meant to be consumed by museum visitors which is still to be defined as it requires further requirements refining, so their feasibility is unknown.

The distinction between story branching points and extra optional content points can also easily be explicitly made and is, therefore, likely to be a feature. The ability to define constraints on the branches depending on visitor choices, previous actions, and physical location should ideally be present in the final tool.

Letting players add their own photos and comments could be made a possibility, however, this would necessitate the implementation of filters for inappropriate content, which greatly adds to the development cost, making this a low-priority feature. To address users' anxieties about condensing the story creation process into a foreign data-entry process, the inclusion of storyboarding functionality with a text-editor feel in the developed tool should be greatly considered.

The ability to specify a particular order for experiencing an exhibition or defining an experience where the user can visit in any sequence should be made available to authors, as it is quite easy to implement. The ability to load and edit a map of a particular site is likely to be implemented as it works well for most location-based authoring tools, where a particular piece of story content is always tied to a physical location.

As the authoring of both indoor and outdoor location-aware experiences will be a requirement, allowing real-world locations to be defined through GPS and QR-code scanning is likely an important feature, and its inclusion is highly likely. Recording and storing players' actions within

stories is very improbable to be included in the tool as it requires considerable infrastructure and an amount of development time outside the possibility of the present project.

Allowing multiplayer experiences is outside the context of the present dissertation, as it works better for games where there is a puzzle or mystery to solve. Although this approach can work in the context of some museum/cultural heritage sites, its use cases are too niche to seriously consider its implementation.

Supporting multiple languages is not a necessity for the scope of the project, so it will not be featured in the final version of the tool. Granting rewards to players for completing certain tasks and goals could be a way to boost engagement with the app (namely on the story player's side), but its implementation will likely be a very low priority.

2.4.2.5 One tool vs many tools

The authoring tool will encompass the entire story authoring process seeing as the audience at hand, non-developers, will probably prefer it in comparison to having to undergo the process of learning many different platforms.

2.4.2.6 Context-specific features

Rich text formatting is a feature with a small upside but with an equally small development cost, making its implementation a low priority.

In order to fulfill the augmented reality requirement, the inclusion of a "calibration" feature might need to be considered to align mixed reality components with reference points in real space. The ability to include placeholder objects could be potentially really useful to test mixed-reality scenes, and therefore, its implementation should be highly considered.

A real-time rendering engine is not particularly useful in the context of location-aware experiences, so its implementation will not be considered. IoT integration with the narrative would be appropriate for the realm of augmented reality experiences, but its inclusion lies outside the scope of this project.

Implementing file size reduction methods and using low-polygon count 3D models are both pretty crucial methods to speed up the delivery of the experiences to the players, so they should be considered. For the same reason, the option to disable high-resolution images should be provided.

2.4.3 Accomodations due to lack of programming experience

2.4.3.1 Managing assets

The ability to search and filter assets is very useful for authors to be able to easily find the resources they are looking for, and it comes at a relatively low development cost, so it is likely to be implemented. A common library of resources that allows the addition of user-generated content is likely too time-consuming to consider, so it will not be incorporated.

2.4.3.2 Authoring tool terminology

The authoring tool should strive to incorporate non-technical language whenever possible. Additionally, it should try to borrow terminology from the storytelling domain to increase usability and accessibility.

2.4.3.3 Error messages

The error messages should strive to use easy-to-understand language and not get in the way of the story authoring process.

2.4.3.4 Educational material

The inclusion of supporting material in the form of tooltips for first-time users of the authoring tool is very likely to be included in the final version of the application. The inclusion of at least one sample project would also be ideal. The creation of documentation, manuals, and video tutorials is likely not possible, given the time available.

2.4.3.5 Community support

The creation of an online forum dedicated to the authoring tool does not make sense in this initial phase of the project, due to the absence of a significant enough user base, although it would be interesting to consider as future work.

2.5 Non-academic apps analysis

After the completion of the systematic review, it was noted that some notable AR platforms were not present in the academic literature. It was therefore deemed that a non-academic review, an exploratory search of commercially available authoring tools that allow for the integration of AR experiences, was necessary to get a more complete view of the state of the art. Furthermore, while the academic review provided very satisfactory results in terms of authoring tool features, it was rather devoid of AR-related content. This absence will be made up for by this section. The upcoming sections will provide insightful details about these tools, shedding light on valuable information that may have been overlooked in the academic domain.

2.5.1 ZapWorks Designer

ZapWorks Designer¹⁹ is a versatile AR authoring platform that empowers users to create immersive and interactive augmented reality experiences. It allows for the definition of various scenes and provides several customized transitions. It does not provide the ability to create branching narratives, as the scenes can only transition between each other linearly. In terms of asset management, users have access to a library of components of 3D models, with the possibility of uploading

¹⁹<https://zap.works/designer/>

their own 3D models, animations, videos, images, and audio. Alpha channel videos, which are videos with an additional channel that defines areas of transparency, can additionally be added to any Designer project.

In terms of AR functionality, the platform allows for image tracking (the attachment of digital content to a flat or curved image), world tracking, which tracks AR objects to a real-world location, and finally, face tracking, which detects and tracks AR content to the user's face, with 12 possible relative attachment points.

A notable feature that was absent from the academic review is the so-called "Screen UI", a no-code tool that allows the user to define UI components in a screen in 2D space, thereby allowing the user to customize the UX aspect of interacting with the AR experience. This feature is visualized in Figure 2.5. ZapWorks Designer lets the user overlap multiple devices on the screen so that the user can better preview how the UI components will look in multiple screen dimensions, which can also be observed in Figure 2.5.

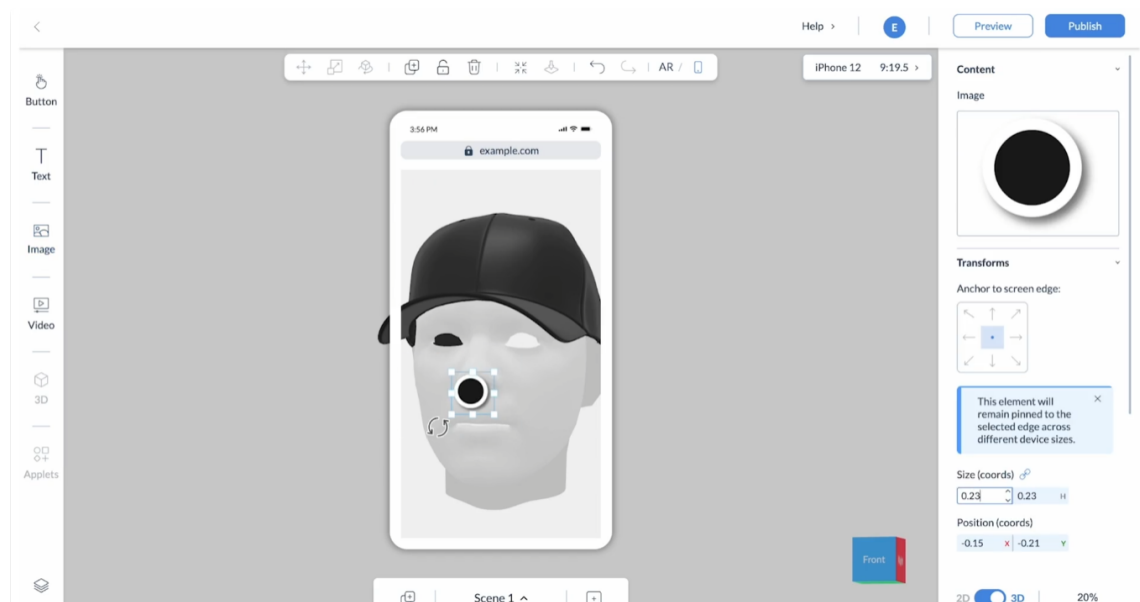


Figure 2.5: ZapWorks Designer - User defining a new button for the AR scene. Note the possibility of anchoring the component to any edge of the screen and configuring the size and position.

The platform employs WebAR, allowing users to view experiences using their browser, ensuring end users can access content without having to install anything. The platform also offers self-hosting of the WebAR experiences. ZapWorks Designer offers the possibility of exporting the created experiences in AR Web Embed form, meaning these can be easily integrated into any existing website.

ZapWorks Designer offers templates, extensive documentation, and tutorial videos to users getting started with the platform. Finally, the platform has integrations with Google Analytics and Microsoft Clarity to gather valuable data points and KPIs, such as acquisition, impressions, engagements, and dwell time.

2.5.2 Minsar Studio

Minsar Studio ²⁰ attempts to be an intuitive workspace to design interactive AR environments and experiences. The platform offers the opportunity to develop AR experiences in a virtual reality setting, with the possibility of connecting the device to a PC to harness the computer's rendering power for a more detailed and high-fidelity view of the creation. This additionally has the upside of allowing the end-user to physically walk around their creation, which boosts previewing capabilities to new heights.

Minsar Studio offers extensive cloud integration, allowing for the importing of assets from services such as Google Drive and DropBox²¹, as well as local content. Furthermore, most file formats for 3D models, audio, image, and video are supported so the tool boasts considerable third-party compatibility. The platform also offers a considerable amount of presets, namely 3D models. A cloud auto-saving feature is also provided, which streamlines the development process seeing as the user never has to stop to perform a save.

For defining interactive behaviors with AR content, Minsar Studio introduces actions and triggers. Triggers include gazing at an object and tapping, while actions can include rotating, levitating, or pulsing, allowing for the creation of simple animations. Any object in the experience may be associated with a URL, which opens upon tapping the object.

The platform utilizes LIDAR (Light Detection and Ranging) technology to provide a stable AR/VR experience. LIDAR is a remote sensing technology that uses laser light to measure distances and create precise, three-dimensional maps of the surroundings. LIDAR sensors emit laser pulses and then measure the time it takes for the light to bounce back after hitting an object. By calculating the distance to various points in the environment, LIDAR generates a detailed and accurate representation of the surfaces and structures in its line of sight.

Minsar Studio offers various forms of AR image tracking, which the platform calls anchors. It provides an image anchor, superimposing AR experiences to any custom picture, a spatial anchor, which scans the surrounding environment to attach the experience to a specific location, an inner 360 anchor, which allows for placement of AR objects inside of 360° videos, and a horizontal and vertical anchors, which scan the surroundings for horizontal or vertical surfaces respectively, on which the experience will be placed.

The final experiences are exported with WebAR, and they may be accessed either through a direct link, a Minsar-generated QR code or simply embedded on an existing website.

2.5.3 ARWay

ARWay is a no-code platform that aims primarily to allow users to create indoor navigations of real-world spaces, leveraging the power of AR to do so. The development experience with the app always begins by uploading a picture of the venue's floorplan and generating a primary QR code which serves as the way for final users to access the AR experience. ARWay then requests for the

²⁰<https://www.opuscope.com/minsar/overview>

²¹<https://www.dropbox.com/>

user to place pegs on various points within a 10-meter radius of the physical location of the QR code to align the map. The platform allows for other ways to access the experience, namely the scanning of images, which serves the additional purpose of correcting any possible drift that might have occurred during the visit.

Maps can then be enriched with AR content such as 3D models, whose animations, if they exist, can be previewed in the web editor. Audio can also be added to the map, which can be set to autoplay upon the user encountering it or playing only after user interaction. ARWay additionally lets users define Guided Tours, by defining start, pivot, and end nodes on the map. Hotspots function as AR hyperlinks, taking the person to any given URL upon interacting with the object. The platform allows the addition of images and videos as AR billboards, which can be set as 'Dynamic Viewing', meaning that the content will always face the direction of the map viewer. 'Dynamic Viewing' can also be applied to AR Text in the experience. Figure 2.6 shows the process of adding an image to the map.

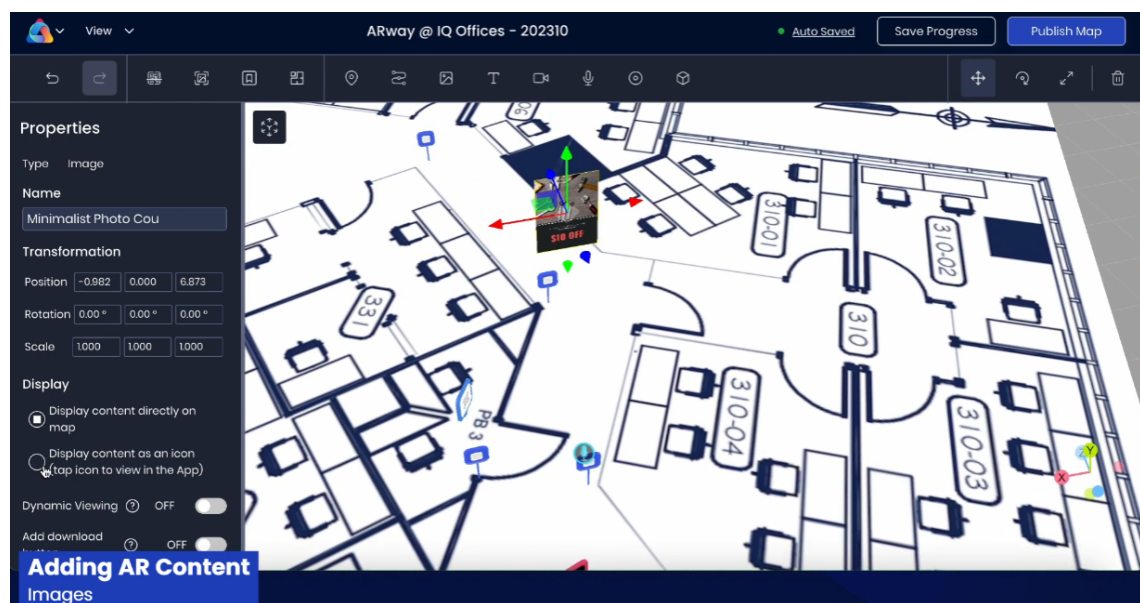


Figure 2.6: ARWay - Adding an AR image to the floorplan.

ARWay enables the definition of location pins for common places such as restrooms, cafés, elevators, and security booths, among others. Users can select between custom navigation styles, which are different ways of representing the path visually during the AR experience. In order to guide users to locations, ARWay requires users to annotate their map with available paths, defining these as either all-access or walkable only. Path widths are customizable.

For venues where multiple maps might be needed (e.g. buildings with multiple floors), ARWay offers so-called 'Venue maps' which let users define connector pins that define points where one map transitions into another.

Furthermore, the platform offers some analytics information, allowing users to monitor map visits occurring over time and offering statistics on the locations within the map with the most engagement. For an individual map, creators can additionally see the total accumulated visit time,

average visit duration, total visits, and map size calculation. The most searched for places are also displayed alongside a user traffic heatmap, which can be seen in Figure 2.7.

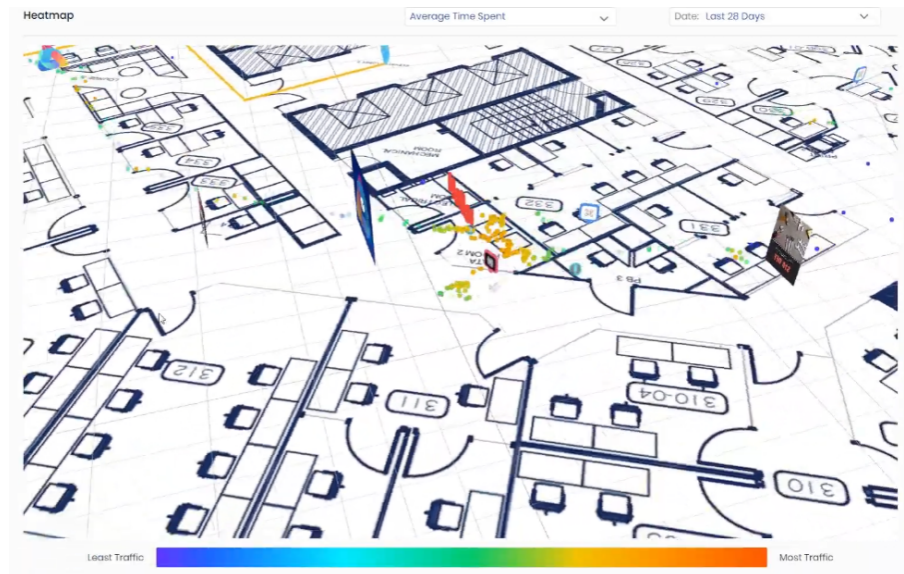


Figure 2.7: ARWay - Heatmap displays the amount of traffic across different map zones.

2.5.4 Metaverse Studio

Metaverse Studio²² is one of the most relevant interactive augmented reality platforms. When a user begins creating a new experience, they are taken to the storyboard view, a graph-based visualization of the branching narrative. The Storyboard offers the ability to undo actions and zoom in/out on the story graph. Nodes in this graph are called 'Scenes' and can have many types, with the most basic one being a simple character card with a place to choose a character, a space to add dialogue, and a space to add some buttons to interact with the scene or transition to another one. Characters can be 2D images, GIFs, or 3D models. Figure 2.8 details Storyboard usage.

²²<https://studio.gometa.io/>

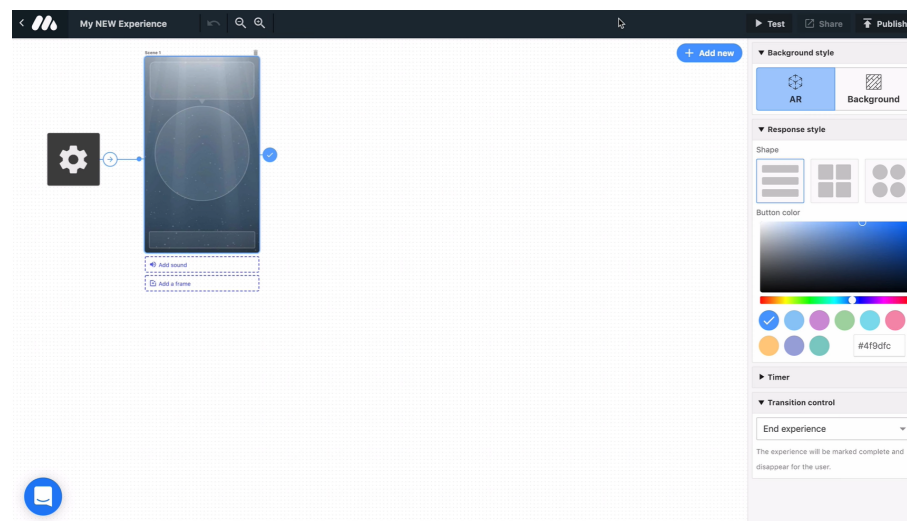


Figure 2.8: Metaverse Studio - Storyboard view with one Character Scene.

Metaverse Studio experiences are developed using building blocks belonging to two different categories: Scenes, the visual elements of the experience, and Blocks, which provide the logic for the behavior of the experience. The previously mentioned Character Scene is joined by other types such as:

- Text Input Scenes - lets the player write text
- Item Scenes - allows the gifting of digital items to a user that are kept in their inventory (only viewable in the Metaverse app)
- Webview Scenes - allows for the embedding of a webpage within an experience
- Camera Scenes - asks users to take photos and videos
- 360° Video & Photo - Embeds 360° content in an experience
- Media Wall - display photos, videos, or text captured in Metaverse
- Poll - display the results of a vote

On the logic side of things, Metaverse Studio provides so-called Blocks, which can be categorized as:

- Property Blocks - allows for saving and manipulating values
- Google Vision Blocks - leverages image recognition to detect simple objects, faces, and text
- Inventory - checks a user's inventory to see if they have a specific item
- Leaderboard - adds or decreases a user's score in any given Metaverse Leaderboard

- Record Poll - records a response from a button on a Character Scene or text from a Text Input Scene
- Probability - use probability to determine where a user transitions in the experience
- Text - allow the creator to verify and manipulate text from a Text Input Scene
- Timer - deals with setting timers and conditions based on whether said timers are active
- Media - saves a user picture or text to a Media Wall

Metaverse Studio allows users to author location-locked experiences, by adding a GPS proximity requirement in order for players to access the experience. The Metaverse application features a map that displays the locations of experiences to users. The experiences will change color on the map if a user is close enough to interact with it.

After titling and optionally adding some description tags to the experience, users can publish an experience with Metaverse Studio. The Metaverse app provides a listing of all available experiences and allows for filtering based on the aforementioned creator-defined tags.

Chapter 3

Development

This chapter offers some background for the creation of these applications, along with a broad overview of the planned development process for the applications and prototypes discussed in this thesis.

3.1 Process employed

The development process involved a series of iterations. Each iteration included gathering requirements through meetings, using those requirements to develop solutions, creating a prototype, implementing the solutions, and then evaluating the prototype by presenting it to stakeholders and thesis advisors to gather feedback and additional requirements. The development began with the Story Editor. However, since its development was partially intertwined with the Story Player, they ended up being worked on simultaneously.

The process began with meetings with the thesis advisors. These meetings had the goal of explaining the context of the project and its current state to the author, as well as the project's goals, which can be read in Section 1.1. Following this, the State of The Art, whose methodology, results, and discussion of this phase can be consulted in Section 2, was elaborated.

After this, development began on the Story Editor. Some requirements for it were already known due to an overlap in requirements with the previous iteration's editor. The development was based on this previous iteration.

A brief Prototype Design phase contemplated the creation of a preliminary version of the Story Editor in order to validate and test design concepts, functionalities, and user interactions. Taking as input the clearer understanding of the project requirements and objectives provided by previous phases, this phase's main tasks consisted of sketching and wireframing, with the development of sketches that illustrate the layout, structure, and basic elements of the user interface. This helped visualize the overall design and layout of the prototype. This prototype was shown to the thesis advisors to obtain feedback about its flaws.

Following this, the Implementation phase began with the creation of a functional prototype for the Story Editor using the React framework. Both prototypes were then constantly iterated on

according to feedback from both the stakeholder and thesis advisors, given during meetings held regularly throughout the entire process. The stakeholder for this project was Mariana Sarmento, a student of the Masters in Multimedia developing her dissertation on co-creation methodologies with the ultimate goal of creating an AR application for Museu Soares dos Reis¹.

The Story Player followed a similar development process, being worked on alongside the Editor and receiving new functionality as they were implemented in the Editor.

3.2 Previous iteration analysis

The designs for the current iteration's story editor and player were based on a previous story editor and player made by João Cardoso (Cardoso, 2023). The previous iteration focused on the development of an editor and player for narratives that took place in a virtual 3D environment.

3.2.1 Story Editor

There were two iterations of the previous Story Editor: a Unity prototype and a React one. The Unity prototype influenced the final solution by its flowchart-like approach to story structure, which was better suited for visualizing complex narrative structures, while the React prototype had a better user interface. Integration of both these aspects into the final product was sought after.

3.2.1.1 Unity Editor

Unity was chosen for the first iteration of the Story Editor as the previous iteration from which Cardoso (2023)'s work was based on was already employing the technology. Additionally, Unity is a platform well-suited for 3D visualization, so it meshed well with the project's goal of visualization of stories in 3D environments.

The main screen of the application featured a header containing buttons to erase, save, and load a story, along with a button to add an empty action to the story and duplicate the selected action in the story. This screen, which can be seen in Figure 3.1, contained a scrollable panel displaying the current story, as well as buttons in the bottom right corner to zoom in and out, as well as a button that, when clicked, opened a menu that shows the characters, places, and objects available. The menu that displays story contents can be seen in 3.2.

¹<https://museusoaresdosreis.gov.pt/>

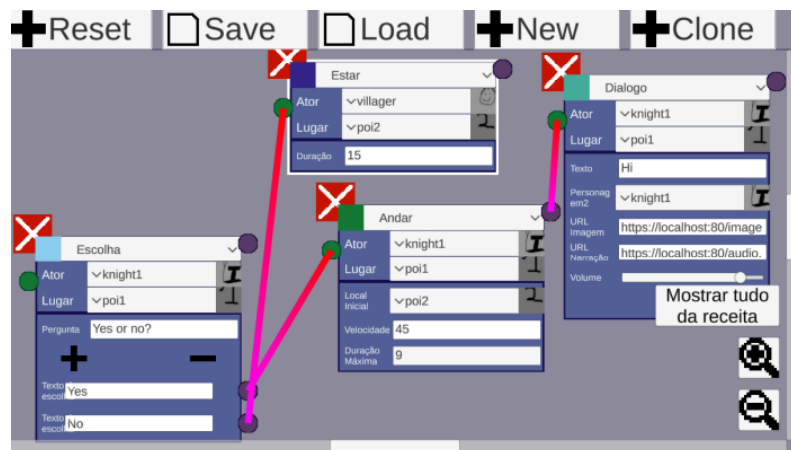


Figure 3.1: Previous Iteration's Unity Prototype Main Screen - © João Cardoso

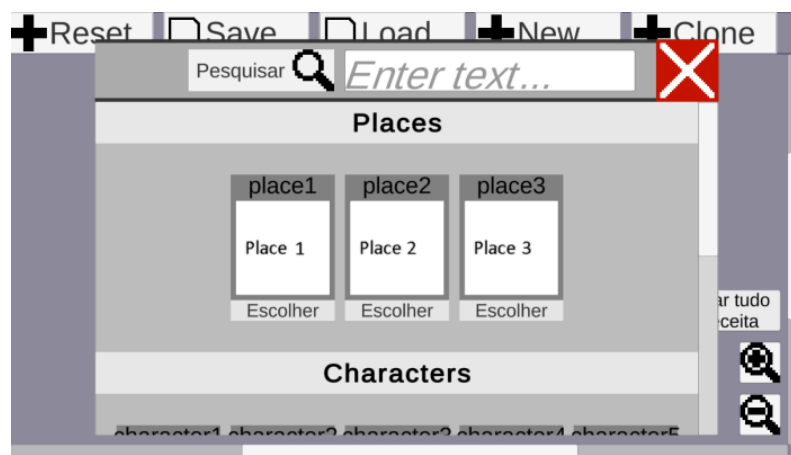


Figure 3.2: Previous Iteration's Unity Prototype Display Story Contents - © João Cardoso

Users created stories through Actions which could be of type Stay ("Estar"), Move ("Andar"), Dialogue ("Dialogo"), Choice ("Escolha"), Hide ("Esconder") and Show ("Mostrar"). Each action could be customized with a certain group of fields.

The user could drag an action by clicking the left mouse button over that Action and then moving the mouse while holding the left mouse button. The Action that was last clicked with the left mouse button by the user was considered "selected" and had a white border surrounding it.

This version of the editor did not have a method to "erase" a story. Additionally, this editor did not include the ability to create a new story. The only ways users could create a new story is by loading an existing story, modifying it, and then saving it under a different name or by closing and reopening the application.

The Load menu contained a dropdown that listed all the available stories. Once the user selected a story they wanted to load, they could click the load button to open the story. This menu can be seen in Figure 3.3.

The Save menu contained a text field for the story's name, a button to save the story, and text that displayed whether the story has been saved successfully, failed to save, or is currently saving. This menu can be seen in Figure 3.4.



Figure 3.3: Previous Iteration's Unity Prototype Load Menu - © João Cardoso

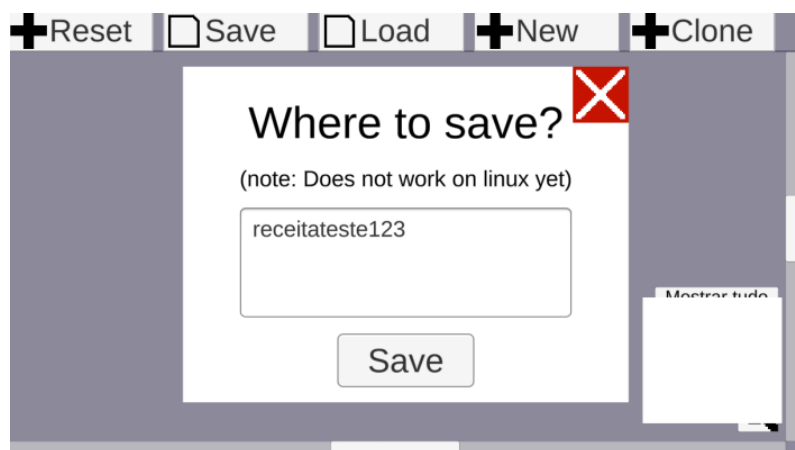


Figure 3.4: Previous Iteration's Unity Prototype Save Menu - © João Cardoso

3.2.1.2 React Editor

After the realization of a workshop, it was deemed that people without computer skills needed an interface that more closely resembled the final result to understand what was happening.

Therefore, a new editor was created with the React framework. It replaced the previous tree-like branching format with a more linear approach, with stories now being divided into sequential scenes, which were further subdivided into slides.

In this version of the editor, Recipes have been introduced. Every story is associated with a specific Recipe, chosen at the beginning or when a new story is created. Each recipe includes a list of characters and locations that can be used within the story.

The application starts at the Create Story menu, which allowed the user to select a Recipe. This menu features a search bar that enables users to filter recipes by matching the typed string with the recipe names. It can be seen in Figure 3.5.

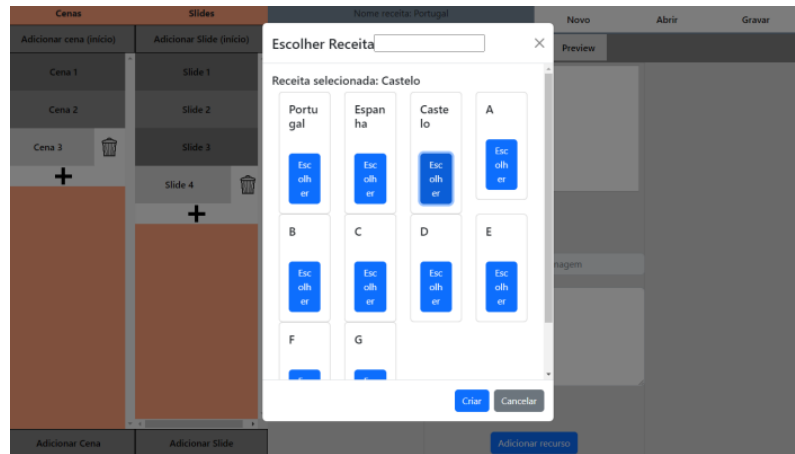


Figure 3.5: Previous Iteration's React Prototype Create Story Menu - © João Cardoso

The main menu of this version of the editor can be seen in Figure 3.6.

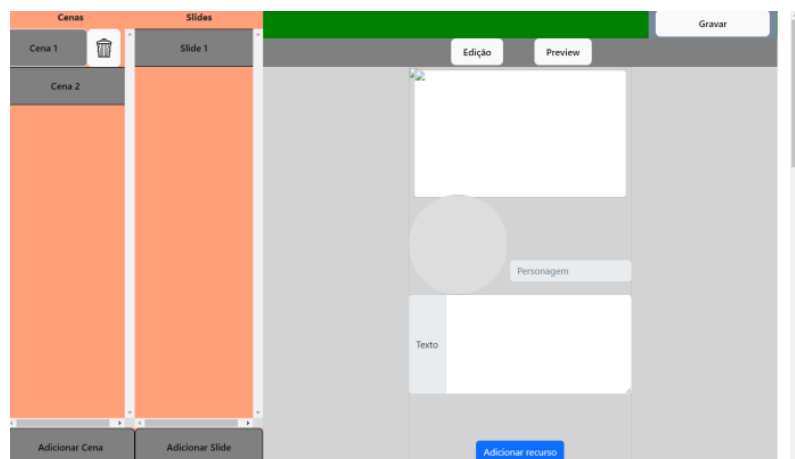


Figure 3.6: Previous Iteration's React Prototype Main Menu - © João Cardoso

The user could append a new scene or a new slide to any scene, but they could neither delete nor reorder slides or add scenes/slides to anywhere but the end of the list. The user could select a scene/slide by clicking on them. The list of slides shown was for the scene that was selected. Each slide contained the following fields:

- A background image
- A place, including an image and a name
- A character, including an image and its associated name
- A text input

- An additional filed button

Each action could include one extra field, except for image fields, which might also have accompanying text. Extra fields might include additional text, an image paired with text, a pair of "before" and "after" images, a 3D model embedded from Sketchfab, a YouTube video embed, or a quiz that presents the user with choices leading to different actions based on their selection.

The editor featured a preview for individual actions, simulating their appearance in a 2D application. However, clicking buttons that would typically navigate to other scenes or actions had no effect. Within this preview, the main fields, such as background image, character image and name, place image, and text, were displayed just as they would be in the editing menu. At the bottom of the preview, a button allowed users to view the content of any additional field set for that action. If no additional field was set, or if it was a destination field, this button would not appear. For destination actions, text provided directions on where the user should go and which object to look at.

Clicking on the character image opened a menu where users could choose the character for that slide, as shown in Figure 3.7. It featured a top bar containing a search bar, a button to add a new character, and a button to close the menu. Below the top bar, characters whose names matched the search string were displayed. Each character entry included an image at the top, the name below the image, and a button to select the character beneath the name. At the bottom of the menu, there was a bar with two buttons, both of which simply closed the menu.

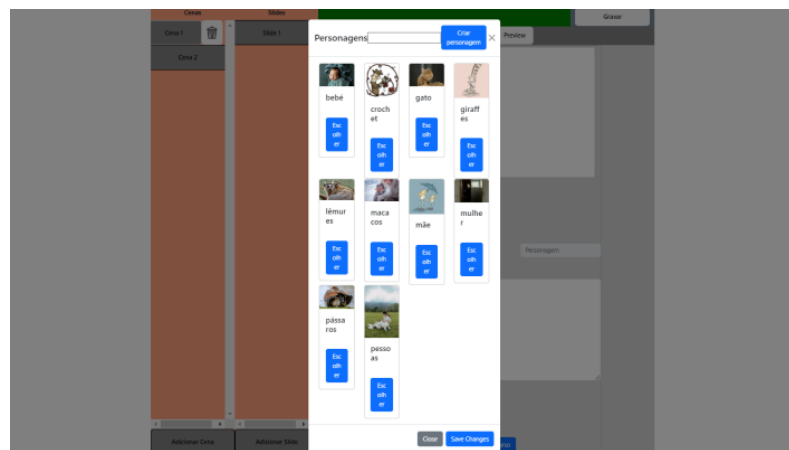


Figure 3.7: Previous Iteration's React Prototype Select Character Menu - © João Cardoso

The Create Character menu featured a top header displaying the menu name and a close button. The central section included a text field for the character's name and an image field for the character's image. At the bottom, there were two buttons: one for creating the character and another for closing the menu without creating the character. In this prototype, it was possible to create a character without providing a name or an image. This menu is illustrated in Figure 3.8.

When the user clicked the save button, a menu appeared. By entering the desired name for the story and clicking the save button, the story was saved. After the saving process is complete, the user can close the save story menu. This menu is illustrated in Figure 3.9.

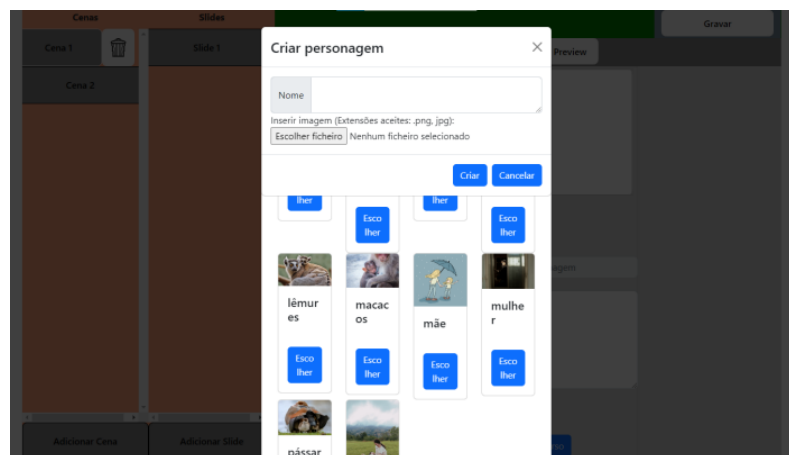


Figure 3.8: Previous Iteration's React Prototype Create Character Menu - © João Cardoso

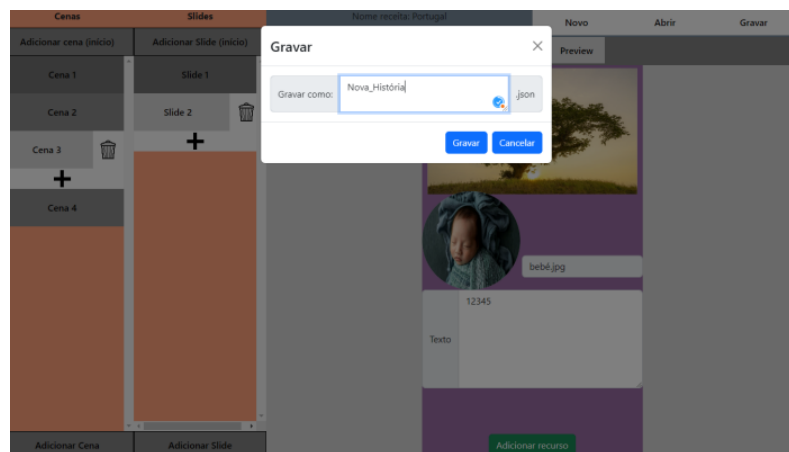


Figure 3.9: Previous Iteration's React Prototype Save Story Menu - © João Cardoso

To load a story, the user would click the load story button, which opened the menu for selecting the desired recipe, as shown in Figure 3.5. After choosing a recipe, another menu would appear, allowing the user to select the story they want to load. The list of stories was displayed alphabetically in ascending order (A to Z), with each list item showing the story name. A search bar at the top allowed the user to filter stories by name. After selecting a story, the user could click the open button to load it. This menu is depicted in Figure 3.10.

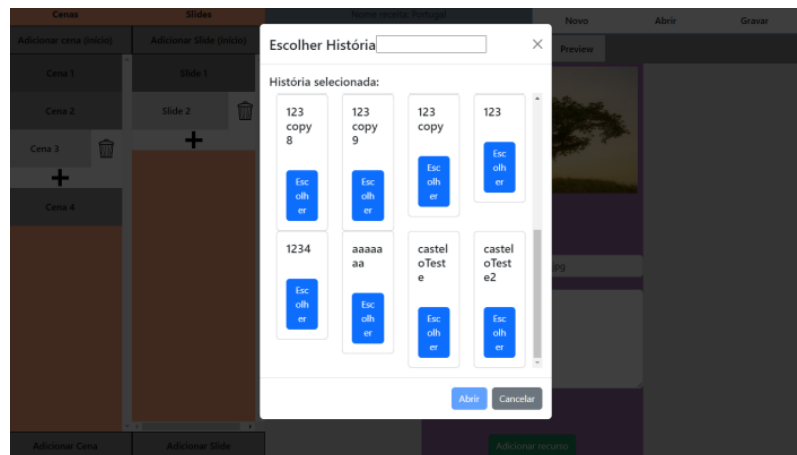


Figure 3.10: Previous Iteration's React Prototype Load Story Menu - © João Cardoso

3.2.2 Story Player

The previous iteration's Story Player was a web application using the Unity engine. The player comprised two distinct screens: the main menu, enabling users to select a story, and the story scene, which either played the chosen story or enabled the free exploration of the environment if the user opted for it. The player's main menu can be seen in Figure 3.11



Figure 3.11: Previous Iteration's Player Main Menu - © João Cardoso

Following the story selection menu, the application operated in three primary states:

- Story loading: the application retrieves the file from the server and converts it into a story.
- Story playback: story actions are executed sequentially until the story concludes.
- Exploration mode: users can control a specific character within the scene.

After the story finished playing, the user would then switch to Exploration Mode.

3.3 Requirements Elicitation and Objectives

The fundamental functionalities and requirements were derived from a combination of literature review findings, insights from meetings with thesis advisors and stakeholders, and the analysis of the previous iteration.

3.3.1 Story Editor

The Story Editor application should enable historians and cultural heritage professionals to create engaging, location-based augmented reality (AR) narratives. To meet the diverse needs of its users and ensure a seamless experience, the application must incorporate the following key functionalities:

Web Environment

The application must operate in a web environment, ensuring accessibility from any device with an internet connection. This approach leverages the ubiquity of web browsers and eliminates the need for users to install additional software, providing a frictionless entry point for all users.

Cross-Platform Compatibility and Responsive Design

Given the diverse range of devices that users may employ, the application must be cross-platform compatible. This includes desktops, tablets, and smartphones. A responsive design is crucial, ensuring that the user interface adjusts seamlessly to different screen sizes and orientations. This adaptability will enable users to create and edit stories on-the-go, regardless of their device.

Focus on Real-World Locations

While the previous iteration of the application focused on stories played in 3D environments, this iteration shifts to real-world locations. This transition necessitates robust geolocation capabilities and AR features that allow stories to be tied to specific physical sites. Users should be able to place virtual elements in real-world settings, creating immersive experiences that blend digital content with the physical environment.

AR Integration

Seamless AR integration is a cornerstone of the application. Users must be able to easily configure AR content, such as 3D models, images, text, and audio, to appear at designated locations. This process should be straightforward, allowing users to focus on storytelling rather than technical details. The editor should include AR preview features so that users can test out their AR content before deploying it.

Degree of Freedom for Story Creators

A significant consideration is the degree of freedom granted to story creators. Greater freedom would allow users to design more complex and personalized narratives without needing modifications to the application. However, this can increase both development and story creation time and potentially overwhelm users with too many options. Therefore, the application should balance flexibility with simplicity, providing essential tools and features while avoiding unnecessary complexity.

User-Friendly Interface

A user-friendly interface is paramount. The application should feature intuitive navigation, ideally with drag-and-drop functionality. This approach minimizes the need for programming or design assistance, empowering users to modify and enhance their stories independently. The interface should guide users through the creation process with clear instructions and helpful prompts.

Flowchart Visualization

To help users visualize the structure of their narratives, the application should include a flowchart visualization of the story graph. This feature allows users to see the connections and sequence of events in their story, making it easier to manage complex narratives and ensure logical progression.

Rich Media Integration

Rich media integration is fundamental. The editor should support the inclusion of images, audio, video, and 3D models in the narratives. Additionally, the editor should aim to support a wide variety of multimedia file formats to ensure a smooth and seamless user experience.

Co-Authoring Features

Co-authoring features should be available to support real-time collaboration. These features enable multiple users to work on a story simultaneously, facilitating teamwork and the sharing of ideas. Co-authoring capabilities are particularly valuable for collaborative projects involving multiple historians or cultural heritage experts.

Export Functionality

Finally, the application should allow users to export their stories to the Player app. This functionality ensures that once a story is created, it can be easily shared and experienced by others. The export process should be seamless, maintaining the integrity and structure of the original narrative.

3.3.2 Story Player

The Story Player application is designed to deliver the immersive and interactive experiences created by the AR Story Editor to end-users. It should adhere to the following requirements:

Web Environment and User-friendly interface

Just like the Story Editor, the Story Player must work in a web environment in order to minimize entry barriers and allow quick playing of stories from a simple web browser. Additionally, it must also feature a user-friendly interface that makes it easy for users to navigate through different stories.

Playback of Editor-Created Stories

The primary function of the Story Player is to accurately and effectively play the stories developed by the AR Story Editor. This includes rendering all multimedia elements, such as 3D models, images, audio, and video, as well as ensuring that interactive elements and geolocation features function correctly in the real-world settings specified by the story creators.

User Progress and History Tracking

The Story Player should allow users to view their play history, including which stories they have played and their progress within those stories. This feature should help users track their engagement and revisit stories of interest.

3.4 Prototype design

Following the elicitation of the requirements and functionalities, the next phase in the development process involved creating a preliminary design using Figma². This step was essential to establish a starting point and gather ideas for the final artifact. Additionally, it facilitated interactions with stakeholders to determine whether the design proposal met their needs. Finally, this quick design phase aimed to bridge the gap between theoretical insights gained from analyzing the previous iteration's work, setting a solid foundation for the subsequent implementation.

3.4.1 Editor's prototype

The first prototype for the editor application included the main page, here named 'Flowchart,' which is the tab designed for the user to edit the overall story structure, the map tab where the user could upload and customize any given plant, and a dialog editing tab for the user to create rich and complex interactions between characters. It also features some pop-ups that are displayed when certain buttons are pressed, allowing the user to choose from menus or input certain necessary data. Furthermore, it displays the Inspector tab, which is meant to be the main way the user edits the content of each scene in their story.

Figure 3.12 shows the Flowchart tab alongside an open Inspector tab for editing the content of an AR Animation component. Clicking on the plus sign in the top bar displays the menu for adding a new scene to the flowchart, which can be seen in Figure 3.13.

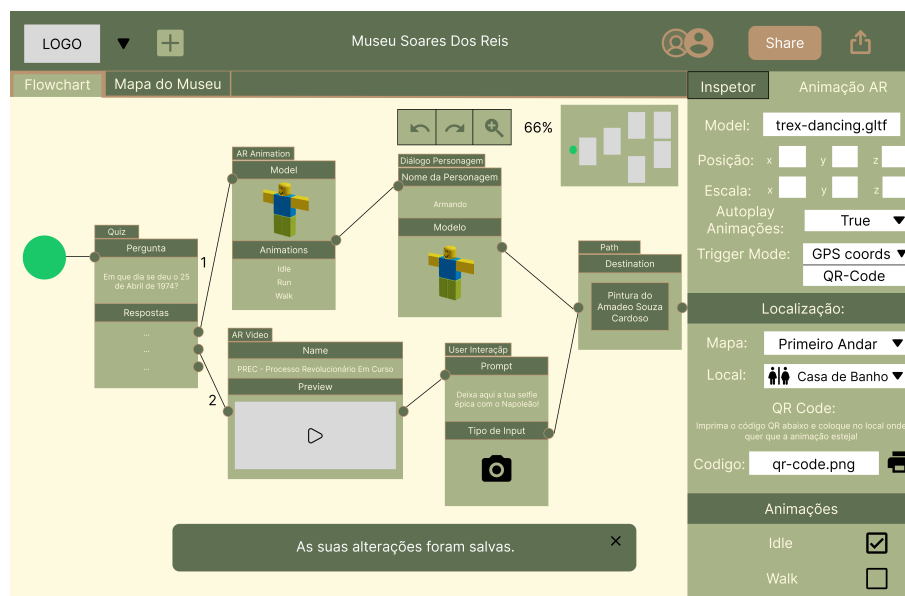


Figure 3.12: Editor Design Prototype's Flowchart tab with the Inspector displayed

²Website: <https://www.figma.com/>



Figure 3.13: Editor Design Prototype's Add Scene Menu

Figure 3.14 shows the Map tab in its initial state when the user has not added any maps. Pressing the plus sign in the middle transitions to the screen with the two anchors, displayed in Figure 3.15. Following this, pressing the 'Continue' button opens the popup, which prompts the user to name their new map, which can be seen in Figure 3.16. After naming the map, the map creation process is finished, and the tab displays the newly created map, along with a dropdown showing all the maps the user has created. The plus sign next to the dropdown initiates the map creation process for a new map. This final state of the map can be seen in Figure 3.17



Figure 3.14: Editor Design Prototype's Map Menu Initial State

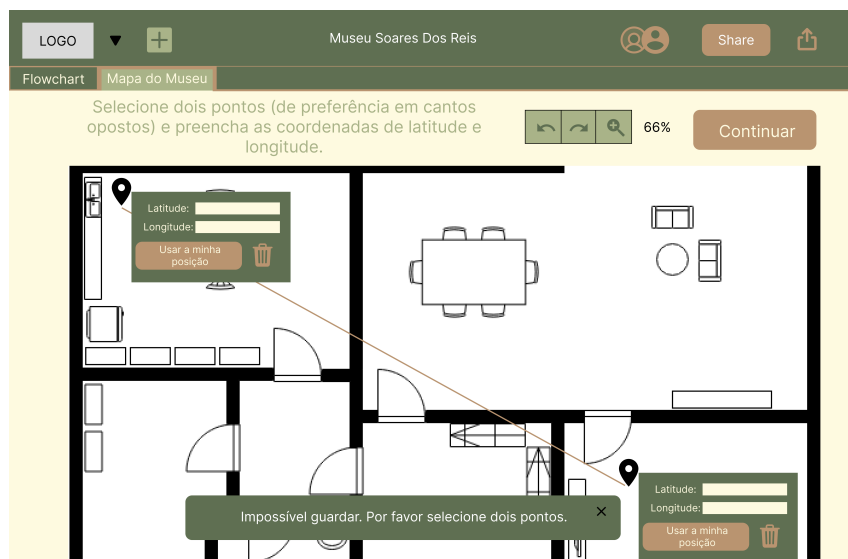


Figure 3.15: Editor Design Prototype's Map Tab with two anchors



Figure 3.16: Editor Design Prototype's Popup for naming a new map

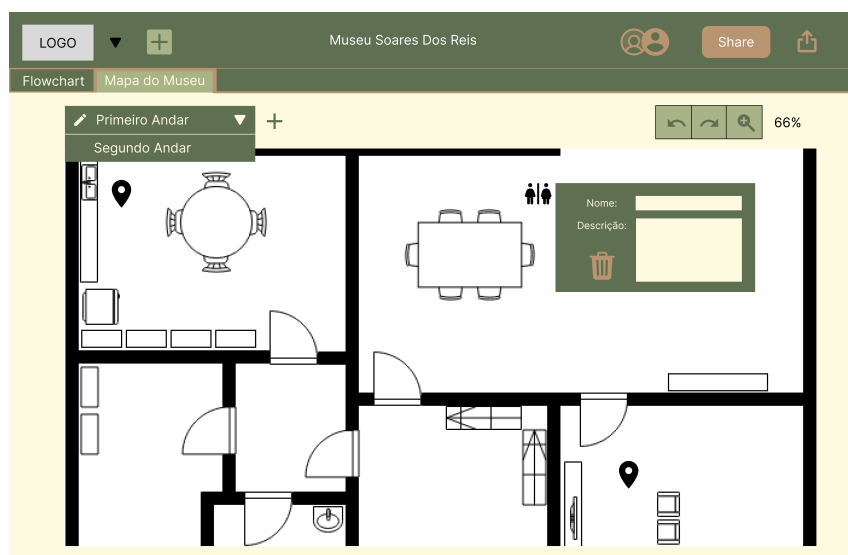


Figure 3.17: Editor Design Prototype's Map Tab

Figure 3.18 displays the Dialogue tab, the interface for creating interactions between the different characters. Pressing the Plus button on the top bar opens the menu that allows choosing a new dialogue component, as seen in Figure 3.19.

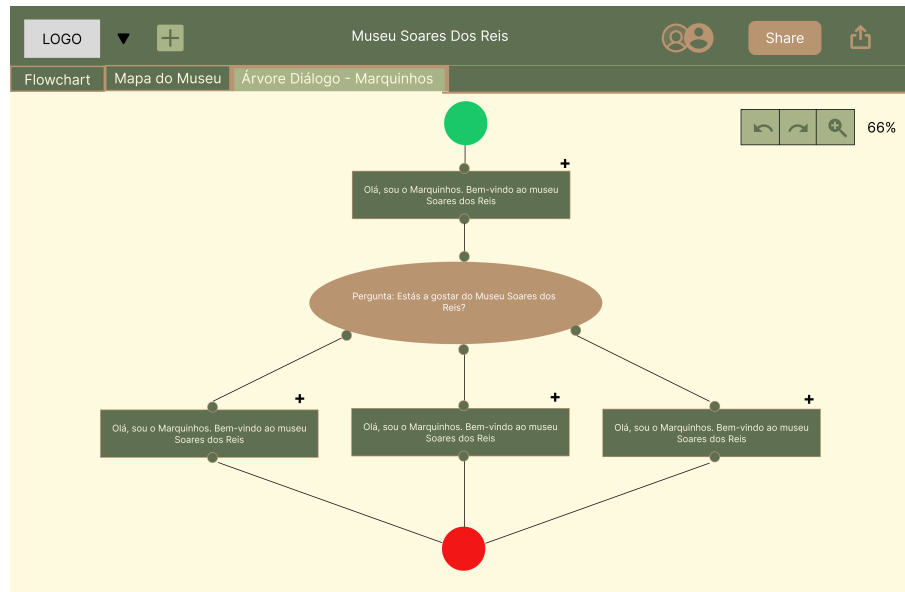


Figure 3.18: Editor Design Prototype's Dialogue Editing Tab



Figure 3.19: Editor Design Prototype's Dialogue Add Dialogue Component Menu

On top of every design lies the Top Bar of the application, which can be seen in Figure 3.20, displaying at its center the current project name and some essential buttons such as the New, Save, and Load actions, as well as the Export and Share Actions. The icons to the left of the Share

button are meant to represent users editing the same project. Finally, the Export Menu can be seen in Figure 3.21.

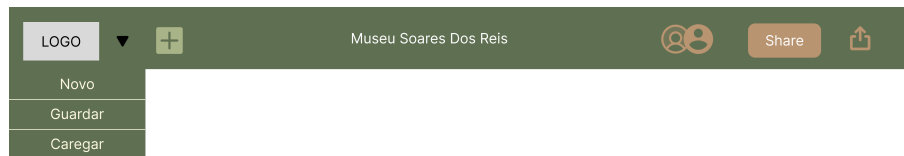


Figure 3.20: Editor Design Prototype's Top App Bar



Figure 3.21: Editor Design Prototype's Export Story Menu

3.4.2 Player prototype

A small Player prototype was also developed at this stage. Given the AR nature of some scenes, it was designed with a mobile context in mind. It contained a main page, the Experiences tab, containing a listing of the exported and ready-to-consume experiences made by users, alongside a search bar to filter the list. This can be seen in Figure 3.22. Pressing the Play Button on any list item will begin playing the experience, as shown in Figure 3.23. The bottom bar also contains an option for a Profile tab, which displays a simple History of the experiences that the user has played and how many endings they have seen from each one. Clicking the Play Button here also starts the experience. The Profile tab can be seen in Figure 3.24.



Figure 3.22: Player Design Prototype's Experiences tab



Figure 3.23: Player Design Prototype Playing Screen

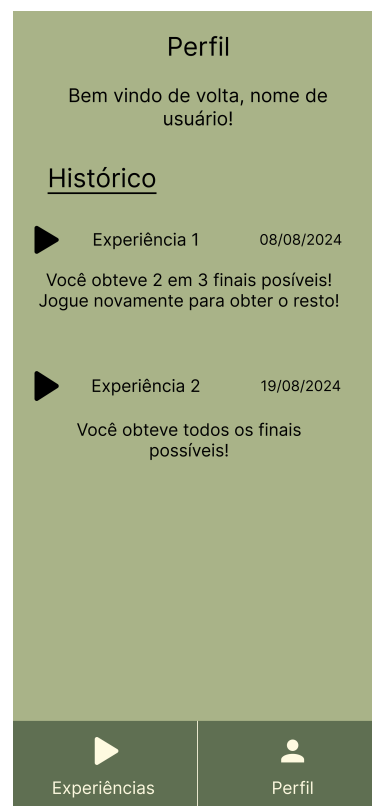


Figure 3.24: Player Design Prototype Profile Tab

Chapter 4

Implementation

This chapter details the relevant portions of the implementation of the applications.

4.1 Architecture

The architecture diagram represented in Figure 4.1 displays the general architecture of the project. The backend, implemented using NodeJS, serves as the central hub for data management. It saves user data, experiences, and custom assets, and it also retrieves experiences and custom assets when requested. On the frontend there are two key components: the Editor Frontend and the Player Frontend, both built with React¹. The Editor Frontend allows users to create and edit experiences, which are then saved to the backend. The Player Frontend is designed for users to play and interact with the experiences retrieved from the backend. Web Browsers facilitate the interaction between users and the frontends, ensuring a seamless experience across different devices and platforms.

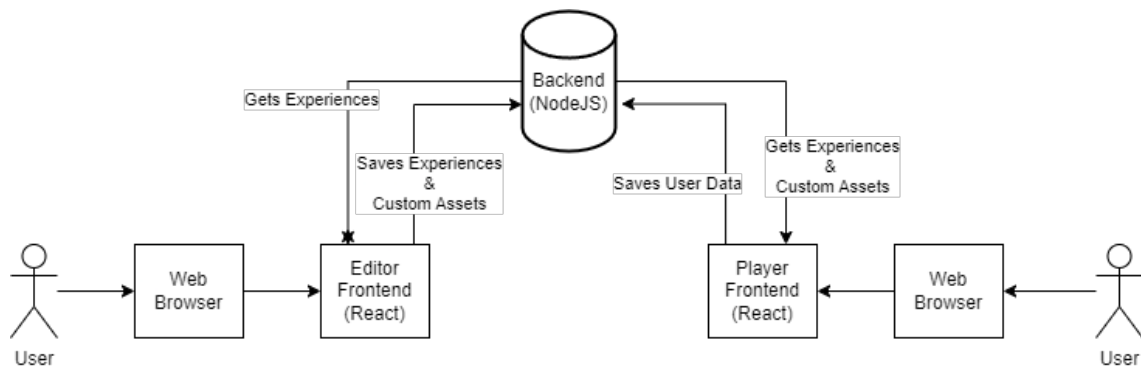


Figure 4.1: Project Overall Architecture

¹<https://react.dev/>

4.1.1 Editor Architecture

The architecture diagram shown in Figure 4.2 illustrates the components and their interactions within the system. At the core, the web application runs in a Web Browser, leveraging React as the main framework for building the user interface. Two additional libraries, ReactFlow² and LeafletJS³, are utilized within the React environment. ReactFlow manages and visualizes complex workflows and interactions on the Dialogue and Story tabs, while LeafletJS provides interactive map functionality. Additionally, on the Story tab, A-Frame⁴ is employed for previewing models on 3D scenes. This setup ensures a modular, responsive, and interactive user experience, with React providing the foundation and ReactFlow and LeafletJS adding specialized capabilities.

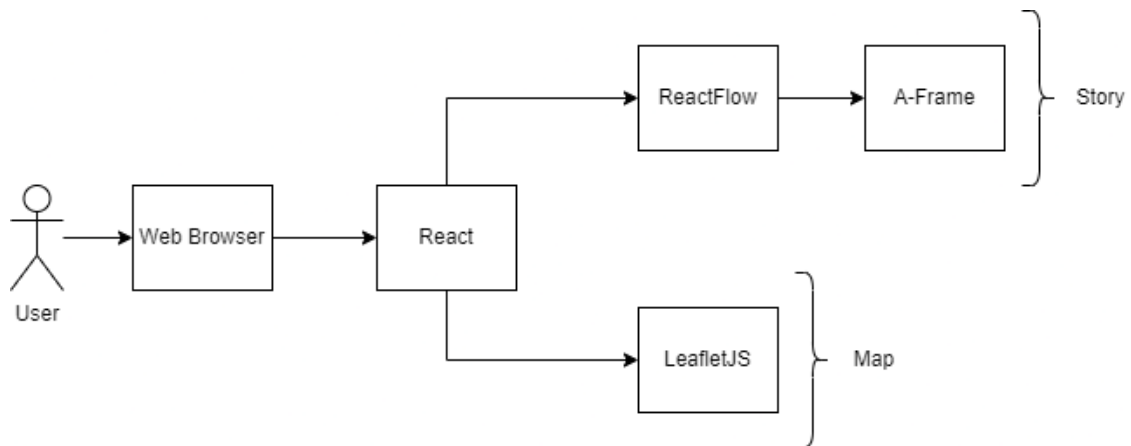


Figure 4.2: Editor Component Architecture

4.1.2 Player Architecture

The architecture diagram illustrated in Figure 4.3 the components and their interactions within the system. The web application runs in a Web Browser, leveraging React as the main framework for building the user interface. Within the React environment, the player application uses AR.js⁵ in conjunction with A-Frame to deliver augmented reality (AR) experiences. The Google Auth API is also integrated to manage user authentication and logins.

²ReactFlow website: <https://reactflow.dev/>

³LeafletJS website: <https://leafletjs.com/>

⁴A-Frame website: <https://aframe.io/>

⁵ARjs documentation: <https://ar-js-org.github.io/AR.js-Docs/>

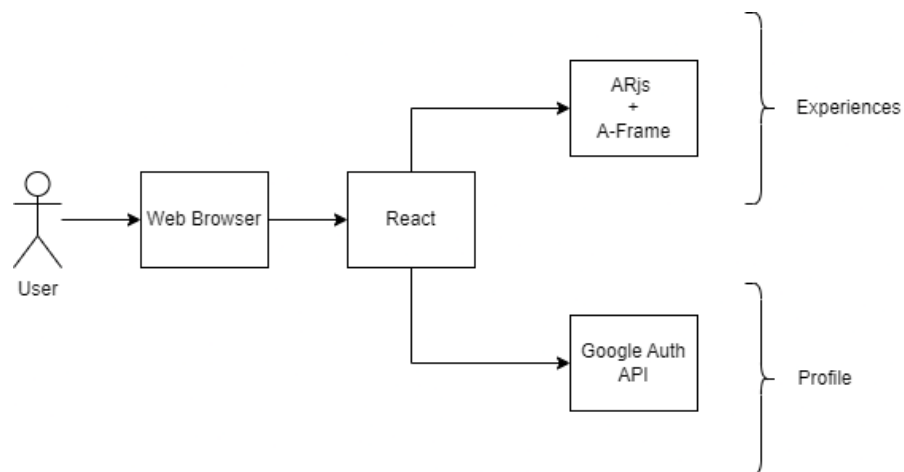


Figure 4.3: Player Component Architecture

4.2 Story Editor

This section details the development of the Story Editor. The Story Editor can be divided into three main components, corresponding to its three possible types of tabs: Story, Map, and Dialogue. We will dive into their functioning in the following sections.

4.2.1 Editor layout

The Story Editor's overall layout can be seen in Figure 4.4. This is what the user sees when opening the application for the first time. The Plus button on the top bar will blink to signal the user to the first step in using the editor and adding content to their story. The button's behavior is further expanded on in Section 4.2.7.

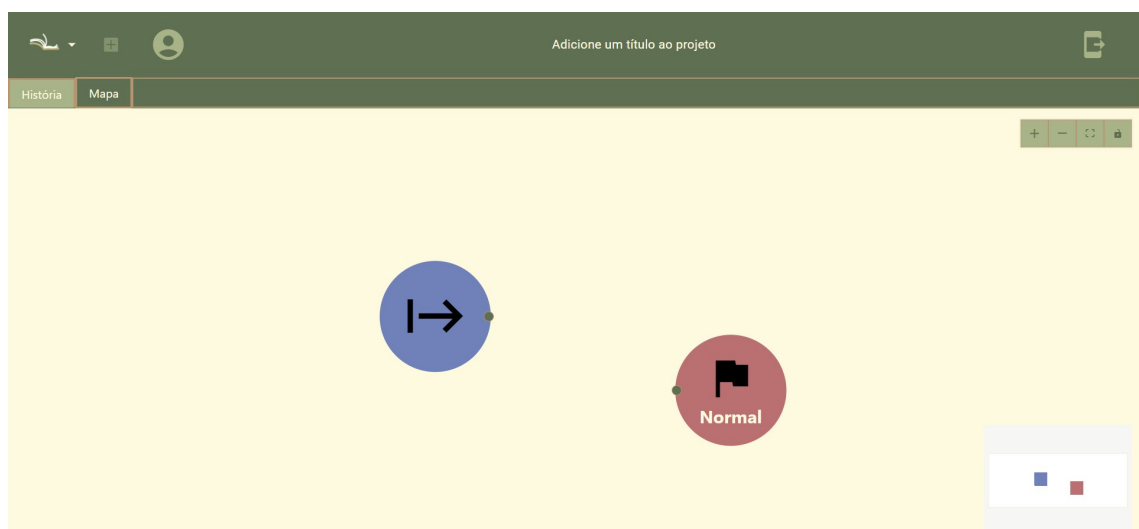


Figure 4.4: Story Editor's layout

4.2.2 Story Tab

The Story Tab is where users define their overall story structure. It is the tab users will be greeted by when using the application for the first time. Story structures are divided into individual scenes, which can be of the following types:

- Begin
- Quiz
- End
- Video (AR)
- Audio
- Image (AR)
- Text (AR)
- 3D (AR)
- Path
- Dialogue

A Begin Scene simply marks the beginning of the story. Each story always starts with one Begin and End scene by default. The Begin Scene cannot be deleted. In the Quiz Scene, users can present a set of questions to the audience. Depending on the responses, different story paths can be triggered. An End Scene signifies the conclusion of the story. It wraps up the narrative and provides closure to the audience. The Video Scene displays a video clip. The Audio Scene plays an audio clip. The Image Scene will showcase an image. A Text Scene will showcase textual content. A 3D Scene features a 3D model that users can interact with. It adds a layer of interactivity and can be used to display objects or environments in a more immersive way. A Path Scene is meant to guide players through the physical exhibit or space to the next location where the story unfolds. It helps direct the audience to different parts of the physical environment as part of the narrative journey. Finally, a Dialogue Scene presents conversations between characters. It can be used to develop the story through character interactions and spoken exchanges.

Scenes share common attributes. Every scene, with the exception of Dialogue and End Scenes, can be associated with a character and some text spoken by said character. Leaving the text blank will remove the character from the scene. All Scenes, except for End Scenes, have customizable backgrounds, which may be either a color or an image. In the case of an image (or any other custom asset that is inputted by the user), it can be entered as either a URL or the user may upload a file from their machine. From here on, when a field for inputting user custom assets is mentioned, it means that the choice between a URL or file upload is offered.

Scenes also have unique attributes of their own. Quiz Scenes have the "Answers" field where users can add, remove, or edit the possible answers for the question. End Scenes contain a text field for editing an identifier. End scenes may share the same identifier, but they will be treated as being the same ending for the story. This can be useful in large story structures where connecting all scenes to a single End Scene would be impractical. Audio, Video, Image, and 3D Scenes all contain a unique field for inputting custom assets of the corresponding type. Audio Scenes contain an additional field to edit the color of the audio player. Path Scenes have a field for the user to input the final destination that the player must reach for the transition to the next scene to happen. A Dialogue Scene contains a button that opens a new tab to edit the interactions between characters. The functioning of this tab is further elaborated in Section 4.2.5. 3D Scenes contain a field that controls whether or not to enable the auto-playing of animations, should the model file contain any.

4.2.2.1 Adding a Scene

Adding a new Scene to the Story is done by clicking the Plus button on the top bar and then selecting a Scene type from the displayed popup, which can be seen in Figure 4.5.

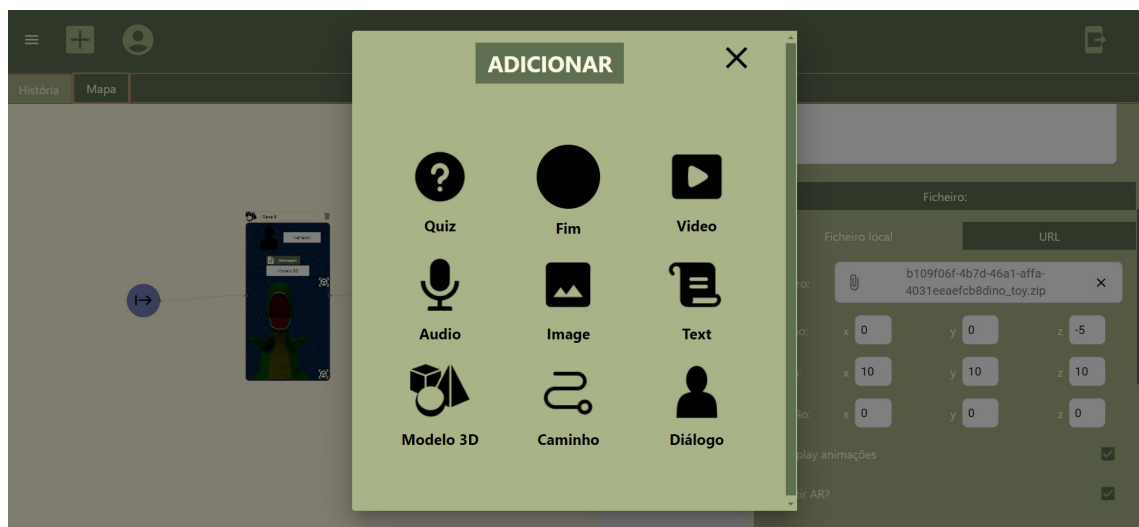


Figure 4.5: Popup allowing users to choose a new Scene to add to the Story

Scenes can be named for easy identification and organization. By default, each scene is assigned a name in the format "Scene " followed by the current total number of scenes, ensuring a unique and sequential identifier. Every Scene has a header containing an icon (varying according to the Scene's media type), the Scene's name, and a Delete button, which will remove the scene from the story graph when pressed. An example Scene can be viewed in Figure 4.6. The initial design wireframes for individual scenes seen in Figure 3.12 were abandoned in favor of a closer depiction of what the final scene would look like when experienced in the Story Player (assuming it is accessed on a mobile device). After meetings with the thesis advisors and stakeholders, it was decided that individual scene nodes should follow a more WYSIWYG (What You See Is What

You Get) design approach, which refers to software that allows content to be edited in a form that resembles its appearance when printed or displayed as a finished product.



Figure 4.6: Audio Scene displayed on the Story graph

4.2.2.2 AR Scenes

The scenes on the list above marked with AR may also be optionally turned into Augmented Reality enabled scenes, with their background changing to reflect this, as seen in Figure 4.7.



Figure 4.7: 3D Scene without AR enabled vs. same scene with AR enabled. Users use the Inspector tab to toggle between the two modes

When a scene is marked as AR, it additionally has position, scale, and rotation attributes. Furthermore, the type of AR activation must be chosen. The Editor provides three options: GPS Coordinates, QR code, and Image Tracking. Choosing GPS coordinates means that the AR content will be displayed at a real-world location defined by the user. A list of the available maps and their places is provided, allowing the user to choose any location from there. The user can additionally input a tolerance (in meters), which is the distance the player must be to the actual location for him to be considered there. The input fields for the GPS Coordinates option can be seen in Figure 4.8. The process by which the application associates a GPS coordinate with a place on the map is further detailed in Section 4.2.4. If the user chooses the QR Code option, the app will prompt them to choose an identifier, which will be used to generate a QR-code with the help of the `react-qr-code`⁶ NPM package. Pressing the printer icon will download the generated QR code. The QR-code input fields can be seen in Figure 4.9. Finally, if the Image Tracking option is selected, the user is prompted to select an image as a URL or as a file from their local machine. The Image Tracking input fields are shown in Figure 4.10. For both Image Tracking and QR-code triggered content, the app will communicate to the user whether the server is still generating the required files or whether the experience is viewable through a warning message.

The AR content will be displayed when the QR code or selected image is detected on the camera feed in the Story Player. For a more detailed explanation of how the camera feed can detect these images, refer to Section 4.4.



The screenshot shows a mobile application interface for setting GPS coordinates. At the top, there is a header bar with the text "Modo de acionamento:" and a dropdown menu currently set to "GPS Coords". Below this, a dark green bar also displays "Modo de acionamento:". The main content area has a light green background and is titled "GPS Coords". It contains a message: "O conteúdo AR será exibido quando o utilizador estiver a uma distância de 5 metro(s) do local selecionado!". Below the message, there are three input fields: "Mapa:" with a dropdown menu showing "stars", "Place:" with a dropdown menu showing "Sculpture", and "Tolerância de distância (metros):" with a text input field containing the number "5".

Figure 4.8: GPS Coordinates input fields. Users can choose the map, the place, and the tolerance distance.

⁶<https://www.npmjs.com/package/react-qr-code>

Modo de acionamento: QR-Code ▼

Modo de acionamento:

QR-Code:

Imprima o código QR abaixo e coloque no local onde quer que o conteúdo apareça!

Código: 



 Marcadores prontos! O conteúdo AR será exibido quando o QR Code for detetado pela câmara!

Figure 4.9: QR-Code input fields

Modo de acionamento: Image Tracking ▼

Modo de acionamento:

Image Tracking:

Quando a imagem seleccionada for detetada pela câmara, o conteúdo AR será exibido!

Imagem:

Ficheiro local URL

Ficheiro:  

 Marcadores prontos! O conteúdo AR será exibido quando a Imagem for detetada pela câmara!

Figure 4.10: Image Tracking input fields. Users must input the image that they wish to be tracked by the camera.

4.2.2.3 Drag and drop interface

The functioning of the story tab is designed to provide an intuitive and interactive way for users to create and manage their stories. Users can drag and drop scenes across the screen, allowing for easy arrangement and organization of their narrative. Each scene represents a specific part of the story, and users can visually position these scenes to reflect the flow of their narrative.

To connect scenes, users simply draw connections between them, indicating that one scene transitions into the next. These connections represent the pathways through which the story progresses, enabling a smooth transition from one scene to another. To delete a connection between nodes the user has to first select an edge and then press the 'Backspace' key. This visual and interactive method of story creation makes it easier for users to see the overall structure of their story, adjust the sequence of events, and ensure that the transitions between scenes make sense. Furthermore, the interface allows zooming in and out of the story's contents and provides a button to center the screen on the whole story so the user can easily get an overview of the whole structure. A button that toggles interactivity is also present and, when toggled, will disable the dragging of Scenes, allowing the user to smoothly navigate their story graph without accidentally moving things. Additionally, a minimap of the whole story graph is present in the bottom right corner, allowing for seamless navigation and ensuring users can maintain an overview of complex story structures. It can be seen in the corner of Figure 4.11.

It is hoped that the story tab's drag-and-connect functionality, powered by ReactFlow⁷ will enhance the story-building process by providing a clear visual representation of the narrative flow, making it more accessible and manageable for users to craft their stories.

4.2.3 Inspector

The Inspector tab, located on the right side of the interface, serves as the primary tool for editing the contents of individual scenes within the story. It is present on both Story and Dialogue tabs. Its header always contains the name of the scene being edited and a Close button to minimize the Inspector tab. In addition to displaying the fields pertaining to every scene, the tab also features a Delete button on the very end, which, when clicked, will remove the scene from the story graph. The Inspector tab is shown in Figure 4.11. Clicking on another Scene while the Inspector for another Scene is open will trigger an update in the Inspector's fields. Clicking on the flowchart pane (the empty space between scenes) will cause the Inspector tab to close. Once a value is edited in the tab, it is automatically saved.

⁷ReactFlow website: <https://reactflow.dev/>

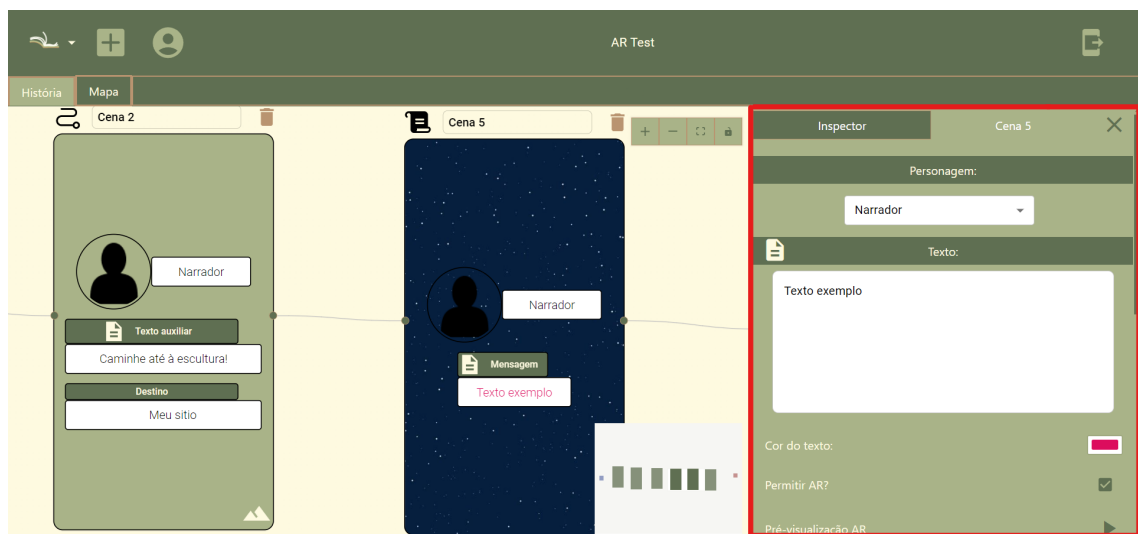


Figure 4.11: Inspector tab on the Story tab

4.2.3.1 Editing AR-enabled Scenes

AR Scenes feature a special component in their inspector tab: the preview AR button, which can be seen in Figure 4.12. Clicking this button will open a popup where users may test their AR content using their device's webcam. This feature allows users to adjust the position, rotation, and scale parameters to match the final desired viewing experience better. In the case of QR-Code or Image Tracking, this feature also helps users to check which images/QR-codes best work in terms of camera detection, tracking stability, etc. This popup can be seen in Figure 4.13.



Figure 4.12: Preview AR button on the Inspector tab



Figure 4.13: Using the AR Preview feature to test out an Image Tracking scene

4.2.4 Map

The Map tab of the application is designed to let users annotate a map of the physical location where they plan for the experience to be played. The process begins by prompting users to upload an image that represents the location. Figure 4.14 displays the Map tab's initial state before the user adds any map.

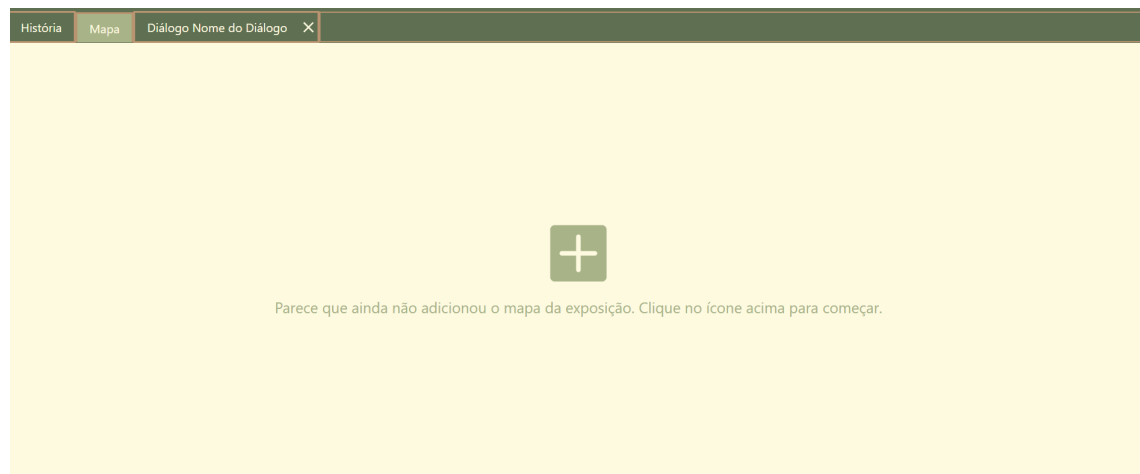


Figure 4.14: Map tab initial state

Following the image upload, users are instructed to place two anchors on the map and input their real-life latitude and longitude coordinates for calibration purposes. Clicking anywhere on the map will create an anchor at that location. Each anchor opens a popup for the user to input the coordinates of the place. Furthermore, the popup also contains a "Use my location" button that will fill the text fields with the user's current GPS position if clicked. It also contains a 'Link' field that accepts an OpenStreetMap⁸ link from which it will retrieve latitude and longitude coordinates. It also contains a Delete Icon that will remove the anchor if clicked. Figure 4.15 showcases this part of the map creation process. This calibration ensures the map is accurately aligned with the physical world.

⁸OpenStreetMap website: <https://www.openstreetmap.org/>

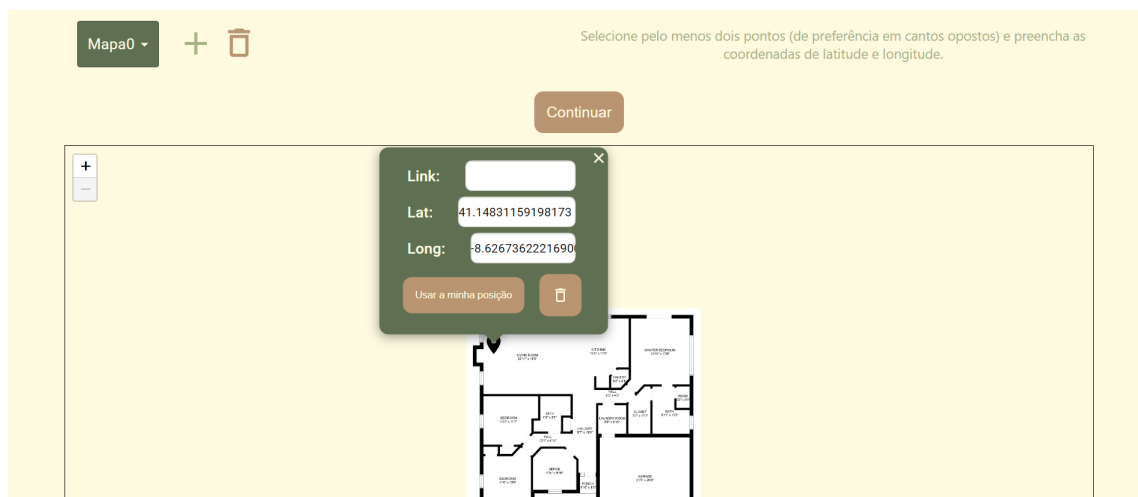


Figure 4.15: Placing anchors on a newly created map

Once the anchors are set, users are asked to provide a name for their new map. Figure 4.16 displays the map naming popup. With the anchors in place and the map calibrated, the application can accurately calculate the real-life coordinates of all subsequent icons placed on the map. This functionality allows users to effectively map out their story or experience, ensuring that each element is precisely located in the real-world context. Finally, after the map-creation process is completed, users may add locations to their map using the Plus icon on the top bar.

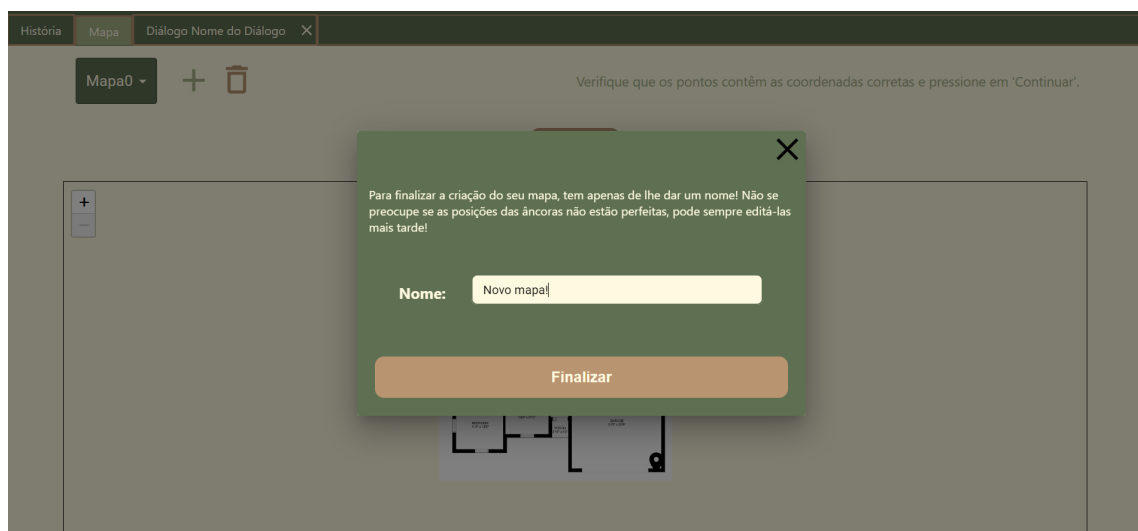


Figure 4.16: Naming the new map

Figure 4.17 displays the tab in this final state. Afterward, clicking on the Plus button on the top bar will display a popup presenting the user with multiple choices for the type of location they'd like to add to their map. It can be seen in Figure 4.18. In this final state, every icon on the map becomes draggable. If the user drags an anchor icon, the map calibration algorithm will be

run again, with the map scale being updated and the real-life position of every other icon already placed. Should the user drag any other icon, only its real-life position will be updated.



Figure 4.17: Final state of the map creation process

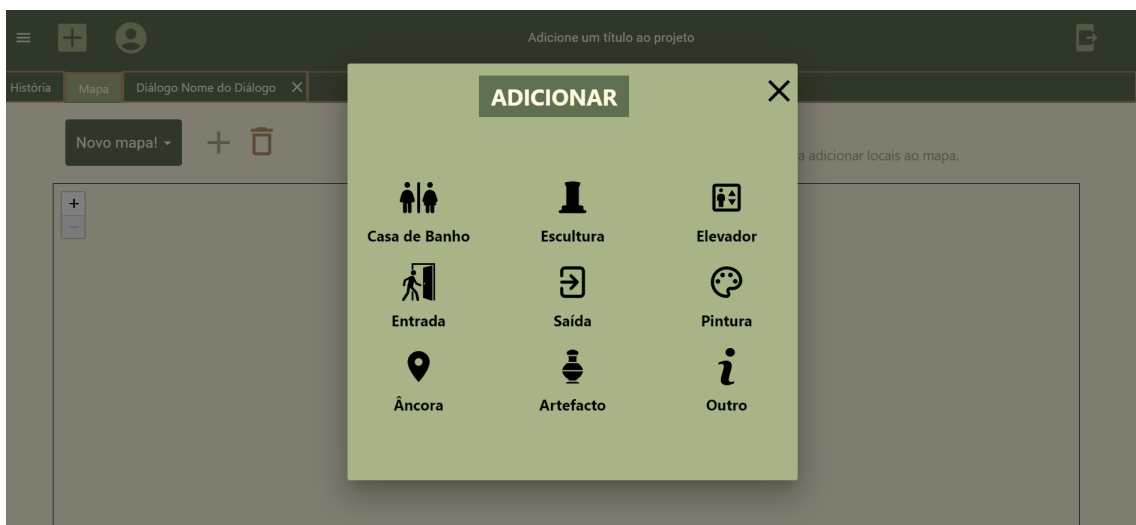


Figure 4.18: Popup prompting the user to select which type of location to add to the map

4.2.4.1 Map calibration algorithm

The following code runs after the two anchors are placed on the map and the 'Continue' button is pressed. It selects the two anchors and proceeds to calculate the real distance between them in meters. Then, it gets the distance between the two points in pixels. From these calculations, we can finally obtain the scale of the map, in meters per pixels.

```
1 const anchor1 = selectedMap.anchors[0];
2 const anchor2 = selectedMap.anchors[1];
3 const realDistance = distanceInKmBetweenEarthCoordinates (
```



```

4   anchor1.coords.lat,
5   anchor1.coords.lng,
6   anchor2.coords.lat,
7   anchor2.coords.lng
8 ) * 1000;
9
10  const imgDistance = Math.sqrt(
11  Math.pow(anchor1.imgCoords.x - anchor2.imgCoords.x, 2) +
12    Math.pow(anchor1.imgCoords.y - anchor2.imgCoords.y, 2)
13  );
14
15  selectedMap.scale = realDistance / imgDistance; // metros / pixel
16  const newMaps = mapsState.filter((map) => map.id !== selectedMap.id);

```

In order to calculate the real-world position of all subsequent markers on the map, we first need to understand some mathematical formulas. We use a known equation for converting from meters to longitude. The number of kilometers per degree of longitude for a given latitude can be approximated by the formula:

$$\pi/180 * R * \cos(\theta * \pi/180)$$

R - radius of Earth in kilometers; θ - latitude in degrees

The number of kilometers per degree of latitude is approximately the same at all locations, calculated by the equation:

$$(\pi/180) * R$$

R - radius of Earth in kilometers

This comes out to roughly 111 km per degree of latitude.

The following code runs after a non-anchor marker is moved on the map. It starts by calculating the distances in meters on the x and y axis between the moved marker and a primary anchor. Now that we understand how to convert between latitude and distances, based on the previous equations, we simply need to take the calculated distances, convert them into latitude and longitude degrees, and then add them to the primary anchor's coordinates to obtain the correct GPS coordinates for the updated marker.

```

1   ...
2   const newAnchors = mapInfo.anchors.map((marker) => {
3     ...
4     const dx =
5       (marker.imgCoords.x - anchors[0].imgCoords.x) * mapInfo.scale;
6     // x distance from anchor in meters
7     const dy =
8       (marker.imgCoords.y - anchors[0].imgCoords.y) * mapInfo.scale;
9     // y distance from anchor in meters
10

```

```

11   const r_earth = 6371e3;
12   marker.coords = {
13     lat: anchors[0].coords.lat + (dy / r_earth) * (180 / Math.PI),
14     lng:
15       anchors[0].coords.lng +
16       ((dx / r_earth) * (180 / Math.PI)) /
17       Math.cos((anchors[0].coords.lat * Math.PI) / 180),
18   };
19   ...
20   return marker;
21 };

```

4.2.5 Dialogue

The Dialogue tab is a dedicated space where users can craft interactions between their characters. It operates similarly to the Story tab, featuring a user-friendly drag-and-drop interface that allows for intuitive scene arrangement. As depicted in Figure 4.19, users can easily connect nodes to define the flow of conversation.

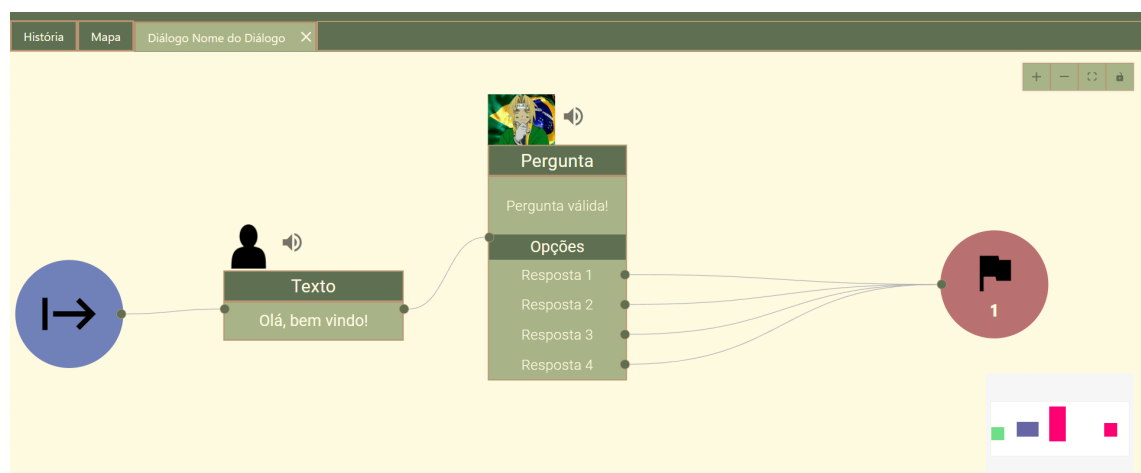


Figure 4.19: Dialogue tab with a Dialogue and Question Scene

Similarly powered by ReactFlow⁹, the Dialogue tab contains the same features and functionalities described in Section 4.2.2.3.

This tab is accessed through the Inspector tab of any given Dialogue Scene by clicking the "Open Dialogue tab" button.

There are several types of dialogue nodes:

- Begin
- End
- Dialogue

⁹<https://reactflow.dev/>

- Choice

When a new Dialogue Scene is created, it comes with a Begin and End node by default. The Begin node cannot be deleted. A Begin node marks the start of the conversation, while the End node marks its conclusion. A dialogue can have multiple endings, allowing users to create branching story paths based on how the player interacts with their characters. A Dialogue node is a simple character speech, while a Choice node presents the player with a question and multiple possible answers. They share some attributes such as a character, the text they speak, and audio, which may be used to provide voice-over or simply background music. In addition to all these, the Choice node contains a field for the user to edit, add, or remove possible answers. Clicking the Plus button on the top bar will open a popup, allowing the user to select the type of node they want to add. It can be seen in Figure 4.20. Similarly to the Story tab, clicking on any node will open up the Inspector tab with their respective attributes to allow for the user to edit them.

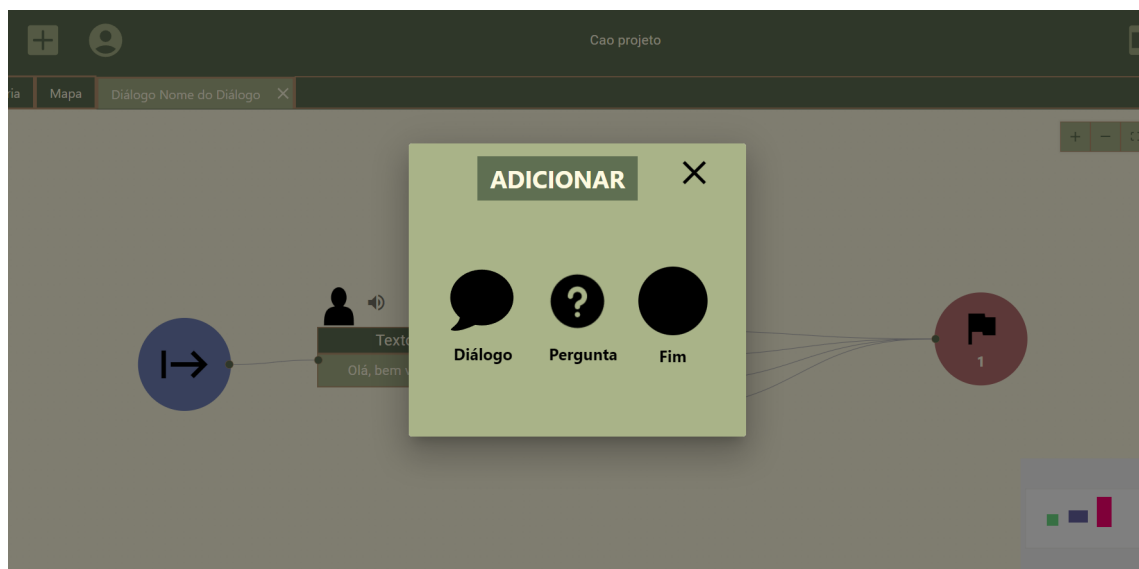


Figure 4.20: Popup prompting the user to select which type of node to add to the dialogue

Dialogue and Choice nodes feature a header containing the sprite of the character performing the action alongside an icon that is displayed in case the action has audio associated with it. Clicking the icon will play/pause the audio. An example of a Choice node can be seen in Figure 4.21.

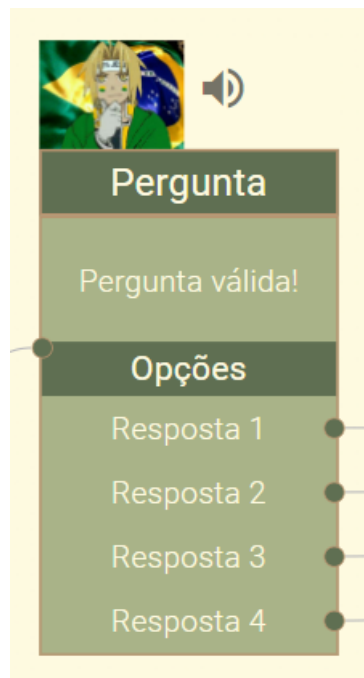


Figure 4.21: Dialogue tab's Choice node

4.2.6 Story Structure code

Every story is represented by a JSON structure like the following:

```

1  {
2    id: "93636d71-35f6-482e-aa1d-1cde5aa9cbfa",
3    characters : [{"id": 0,
4      "name": "Andre",
5      "description": "O narrador da historia",
6      "image": {
7        "inputType": "url",
8        "filename": "../assets/character_dialogue_node.png",
9        "blob": null
10     }
11   }],
12   "description": "O andre precisa de testes e vai pedir ao Miguel.
13   Vamos ver o que lhe espera...",
14   "edges": [
15     {
16       "source": "0",
17       "sourceHandle": null,
18       "target": "f8e27186-cd69-4ff6-815a-41dd8d541177",
19       "targetHandle": null,
20       "id": "reactflow__edge-0-f8e27186-cd69-4ff6-815a-41dd8d541177"
21     },
22     ...
23   ],
24   "experienceName": "Andre precisa de testes",

```

```

25     "lastModified": "2024-06-14T17:15:36.139Z",
26     "maps": [
27       {
28         "id": 0,
29         "name": "FEUP",
30         "progressionState": "name-given",
31         "image": "248e66a7-1a2b-4aaa-a3dc-618b74aeeb6bfejp.png",
32         "mapSize": {
33           "width": 267,
34           "height": 189
35         },
36         "description": "This is your map's description",
37         "scale" : 3.3558146187449744,
38         "anchors": [
39           {
40             "anchorId": 1,
41             "anchorType": "anchor",
42             "coords": {
43               "lat": "41.17841",
44               "long": "-8.59497"
45             },
46             "imgCoords": {
47               "x": "121.2130091222467",
48               "y": "199.875"
49             }
50           },
51           ...
52         ]
53       }
54     ],
55     ,
56     "nodes" : [{
57       "id": "0",
58       "position": {
59         "x": "-2455.804168669649",
60         "y": "-285.9861141108786"
61       },
62       "type": "beginNode",
63       "selected": false,
64       "positionAbsolute": {
65         ...
66       },
67       "dragging": false
68     },
69     ...
70   ],
71   "storyEndings": ["Correu mal", "Correu bem"],
72   "tags": [
73     { "name": "drama", "color": "#FF0000" },

```

```
74   { "name": "luta", "color": "#FFA500" }  
75 ],  
76 "title": "Miguel e o teste do Andre"
```

The structure above is composed of a unique ID, a title, an experience name (the name given to the story after it is exported), some tags, and a description. The date of the last modification is also included.

Additionally, every story contains a list of characters, each containing a unique ID, a name, description, and image information. Furthermore, the edges and nodes, lifted from ReactFlow and modified to contain the relevant data relative to each specific node type, are also stored. Furthermore, the map information is saved, with each map containing a unique id, a name, its progression state (which might be "created", "anchors-placed" or "name-given"), an image name, the map dimensions, a brief description, its scale in meters per pixel, and finally the map's anchors, which have a unique id, an anchor type, real-life coordinates, and image coordinates.

Finally, the story structure also includes a list of all unique story endings to assist with the Story Player's Story History feature, which is detailed in Section 4.3.1.

This structure is saved to the browser's local storage (each field is saved under a different key) after every change the user makes, ensuring they don't lose any progress should they forget to save the story to the server.

4.2.7 Save, Edit and Export

The top bar contains many essential features of an editor application. A general view of the top bar can be seen in Figure 4.22. Clicking on the application logo will trigger a dropdown containing the buttons for the "New", "Load" and "Save" actions. Clicking on the "New" option will reset the story structure to its default state, which entails:



Figure 4.22: Story Editor's Top Bar

- Story graph resets to just a "Begin" and "End" Node
- Characters list is reset to only the default narrator
- Title of project becomes "Add a new title to the project"
- Description of project becomes "Add a description to the project"
- Experience Name becomes "Add a name to your experience"
- All tags (if they exist) are erased
- All maps (if they exist) are erased

The "Load" option will open a popup listing all the stories saved on the backend. This popup can be seen in Figure 4.23. Clicking the "Delete" icon will erase that story from the server. Finally, the "Save" action just updates the current story's values on the backend to reflect the latest edits made by the user.

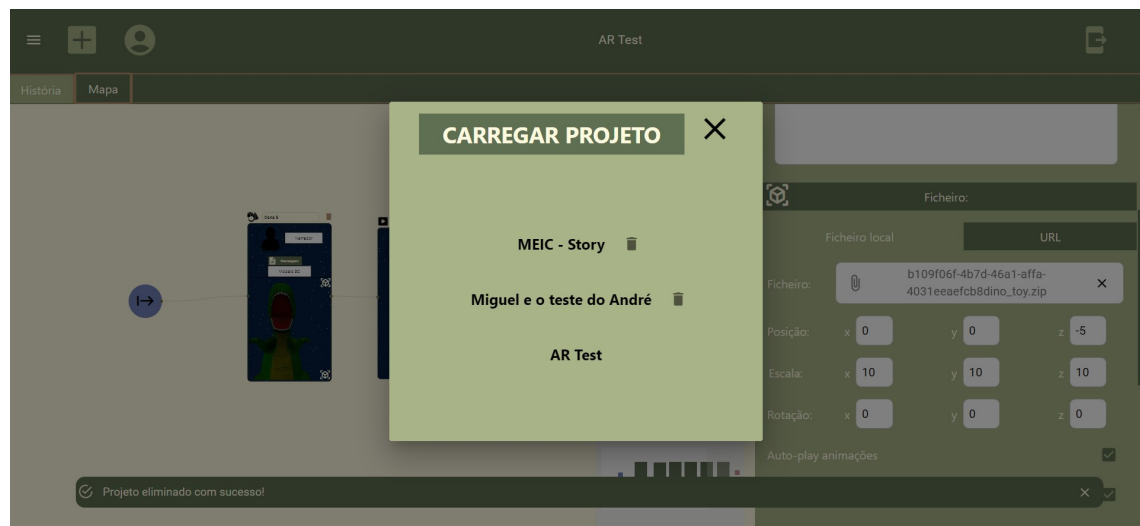


Figure 4.23: Popup triggered upon selecting the "Load" option

The Plus button to the right of the logo will trigger a different popup depending on which tab is currently open. If the user is on the Story tab, it will open the "Add Scene" popup, allowing the user to choose which type of scene they wish to add to their story. It can be seen in Figure 4.5. If the user is on the Map tab, it instead opens the "Add Location" popup, allowing users to add a marker to their current selected map. This is displayed in Figure 4.18. Finally, if the user is in the Dialogue tab, the popup instead allows users to select from the types of Dialogue nodes, which can be seen in Figure 4.20.

In the center of the top bar, the current project title is displayed in a text field. Editing this text field will update the project's title. Finally, on the right-most corner, the Export button will check the current story structure for any errors. By story structure errors, it is meant "anything that would not allow for the experience to be played from the beginning to the end", such as, for example, a missing connection between Scenes, which would trigger an Error alert like the one displayed in Figure 4.29. Certain fields for certain types of Scenes, if left blank, will also trigger error messages, such as in the case of:

- An AR Scene with Image Tracking with no selected image to track
- An AR Scene triggered via QR-Code with an empty QR-code field
- An AR Scene with GPS coordinates without a selected map and/or place.

If no errors are detected, the Export popup, as seen in Figure 4.24, is opened, which allows users to set a name for their exported experience, a description, and some tags meant to give potential players a general idea on what type of story they will experience. When the user presses the

'Confirm' button, if no server errors occur, an alert will indicate that their export was successful, and the experience will be viewable on the Story Player.

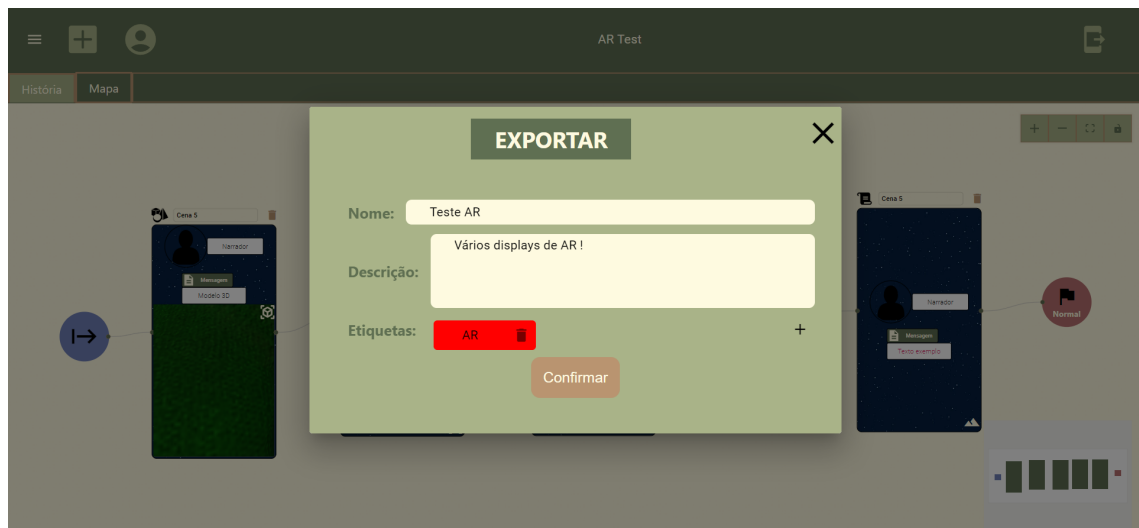


Figure 4.24: Export Story Popup

4.2.8 Character Creation

To the right of the Plus button lies the Character Menu. Clicking on it will open a popup listing the current characters in the project. This may be visualized in Figure 4.25.



Figure 4.25: Popup listing the characters in the project

Clicking on the character portraits will open an "Edit Character" popup, allowing users to alter the character's name, description, and image. It is displayed in Figure 4.26. Clicking the

Delete icon removes the character from the project, while clicking the "Save" button updates the character's information. The button at the end of the listing will open the "Create Character" popup, which can be seen in Figure 4.27. Here, the user can input a name, description, and image for their new character.

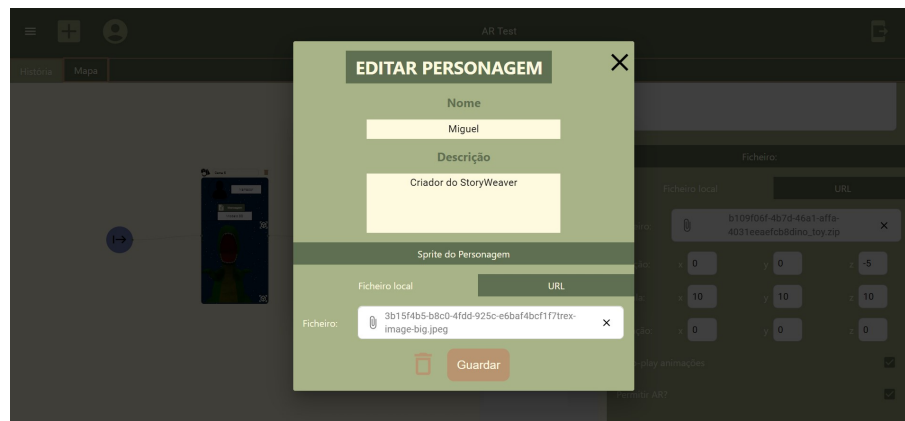


Figure 4.26: Edit existing character popup

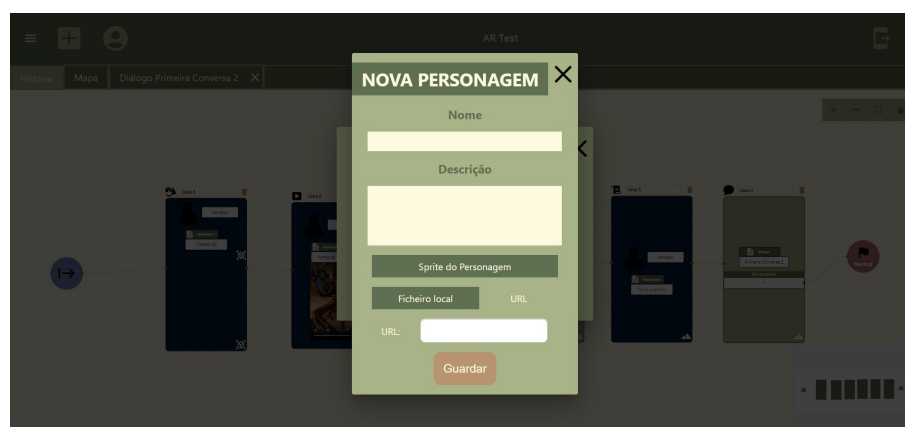


Figure 4.27: Create new character popup

4.2.9 Responsive feedback

Responsive feedback was taken into consideration when developing the Story Editor. The intention was for users to get immediate validation and to keep them informed about the status and results of their actions, contributing to a better user experience. The application provides this feedback by generating alerts on the bottom of the screen when actions such as creating a new project, loading an existing project, saving, creating a new character, or exporting the project, for example, succeed or fail. Error messages, in the case of export-related issues, were made as clear, concise, and helpful as possible so that users could quickly understand what went wrong and how they could fix it. Figure 4.28 showcases an example of a success alert, while Figure 4.29 exemplifies a failure message.



Figure 4.28: Success message indicating that a project has been loaded

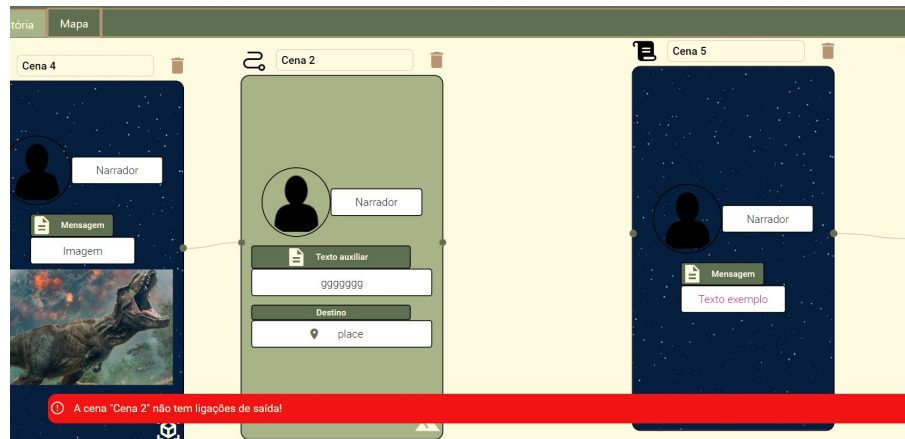


Figure 4.29: Error message displayed due to "Scene 2" lacking an outgoing edge

Another way the application provides feedback is by having the Scene nodes in the story graph immediately update upon field changes on the Inspector tab, reassuring users that their inputs did indeed have an effect. Additionally, file uploads feature a "Loading" message, which is especially useful in the case of larger files that can take up to several minutes to upload (this is the case of 3D model files).

Responsive feedback is very important for these kinds of applications as it:

- Helps build user confidence
- Helps users understand what they need to do, reducing the likelihood of mistakes
- Increases engagement, as users are more likely to want to interact with a website that responds quickly and accurately to their inputs
- Builds trust in the system, proving that it is working correctly and efficiently

4.2.10 AR Markers

Users are encouraged to pick good images for markers, as detailed in the ARjs documentation section on this topic ¹⁰. It states three conditions for a good and stable marker: an image with considerable visual complexity and a large resolution. Beyond these factors, the physical placement of the marker itself will impact the quality of the player experience. There is a limit to how far the software will detect the marker, so if the user prints a small marker, the player will need to get closer in order to see the AR content, and inversely, a huge marker will force the player to get away from the image to see the content. The quality of the print will also affect the final experience.

¹⁰ARjs documentation: <https://github.com/Carnaux/NFT-Marker-Creator/wiki/Creating-good-markers>

Inaccurate colors or thin paper that is not completely opaque can cause inaccurate tracking of an otherwise good marker. Finally, for markers displayed on screens, great care must be placed on their luminosity, as an excess of it may overexpose the camera, causing autofocus and/or white balancing, which will throw off the colors and definition of the marker.

4.3 Story Player

As stated before, a Story Player was developed alongside the editor. The main goal of this application was to allow users to play the stories developed on the editor. The player application can be divided in two main tabs: the Experiences tab and the Profile tab.

4.3.1 Profile

The Profile tab is designed to provide users with personalized information and a summary of their interactions within the app. When users first interact with it, they will see a Google Login button and text prompting them to go through authentication, which can be seen in Figure 4.30. After users complete authentication, a new account is created for them if they don't have one already, and the screen transitions to the one displayed in Figure 4.31. When logged in, the Profile tab displays the user's history in the app, listing the experiences they have interacted with, displaying their names, the date that the user last played the experience, and some information about the number of endings that the user has experienced. The "Play" icon will switch to the "Experiences" tab and start the experience if clicked. The button on the top right will log the user out of the application.

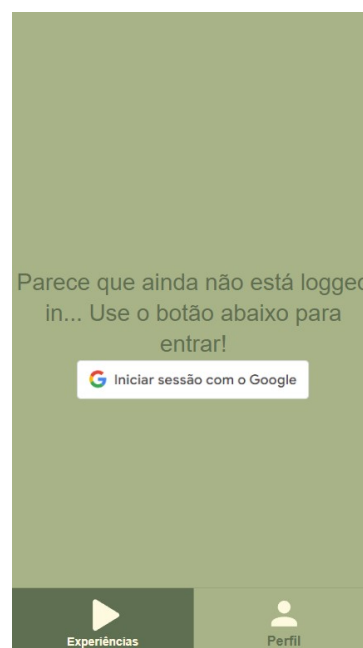


Figure 4.30: Profile tab view when user is logged out



Figure 4.31: Profile tab view when user is logged in

The History feature was created with the intent of incentivizing multiple replays of experiences by showing the users how many possible story paths they have yet to experience.

4.3.2 Experiences

The Experiences tab is the part of the application where users interact with the stories exported from the Story Editor.

4.3.2.1 Selecting an Experience

When switching to the Experiences tab, the user is greeted by a listing of the available experiences to be played. This is showcased in Figure 4.32. This screen features a search bar allowing users to look up stories by their names. The Play button featured on every item of the experience list will start the experience if clicked. Every item also contains a down/up arrow button that will expand or collapse extra information (description and tags) about the respective experience when clicked. In Figure 4.32, the first list item displays the expanded state while the second showcases the collapsed state.

4.3.2.2 Playing an Experience

Experiences begin, without exception, on a Start Screen like the one shown in Figure 4.33. Pressing the "Start" button displays the next Scene defined by the story structure. The screens will vary depending on the type of Scene being displayed, but most of them have a Skip button on the bottom right corner of the tab, which will transition to the next Scene. The only exceptions to this are



Figure 4.32: Experience Selection screen

Scenes where the story branches into multiple paths, and the player must make a choice, such as Quiz Scenes and Dialog Choice Nodes. An example of a Quiz Scene can be seen in Figure 4.34.



Figure 4.33: Example Start Screen for experience on the Story Player

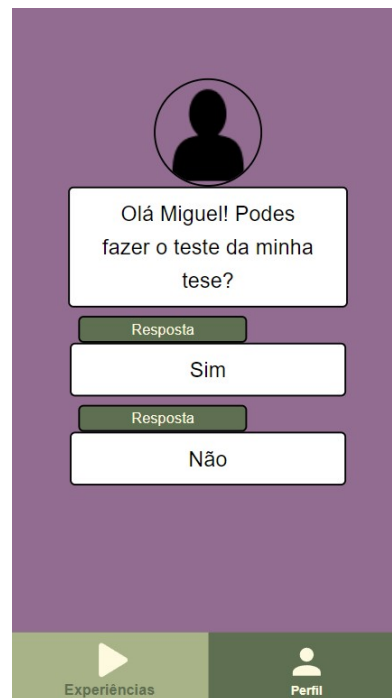


Figure 4.34: Quiz Scene on the Story Player

All Scenes display a character along with their speech text on top of the screen. In the case of AR-enabled scenes, instead of displaying an image/color background, the device's webcam feed is utilized. An example of an AR-enabled 3D model scene is displayed in Figure 4.35.

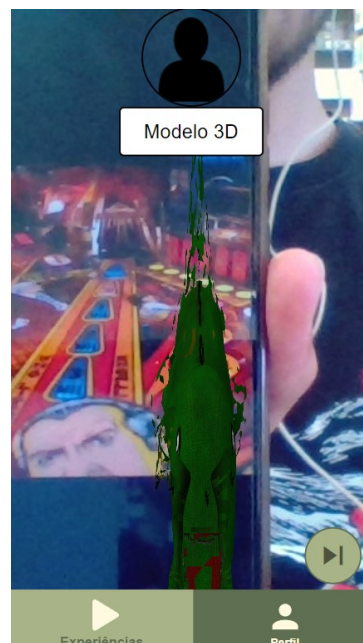


Figure 4.35: AR-enabled 3D Scene with Image Tracking

Path Scenes will display the distance the player is to the destination if this distance is superior

to the tolerance set by the story maker, as seen in Figure 4.36. When the distance to the destination is lower than the tolerance, a success message is displayed, indicating that the player is ready to proceed with the experience, as seen in Figure 4.37.

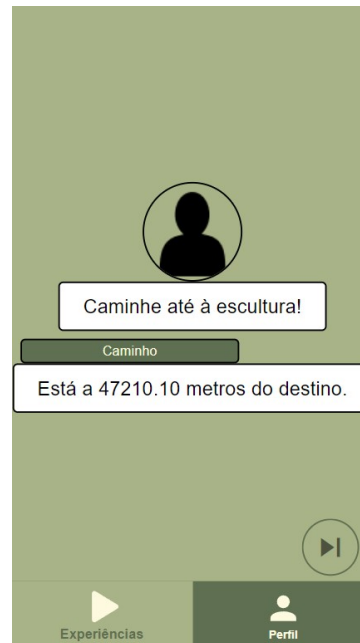


Figure 4.36: Path Scene when the player is far from the destination

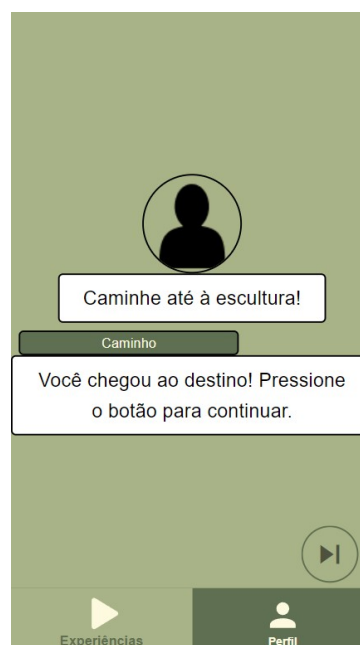


Figure 4.37: Path Scene when the player's distance to the destination is smaller than the tolerance

Dialogue Scenes, when they include audio, will display an additional button on the bottom left that pauses/plays the audio. The audio starts playing automatically when the Scene is loaded. An

example of a Dialogue Scene is displayed in Figure 4.38.

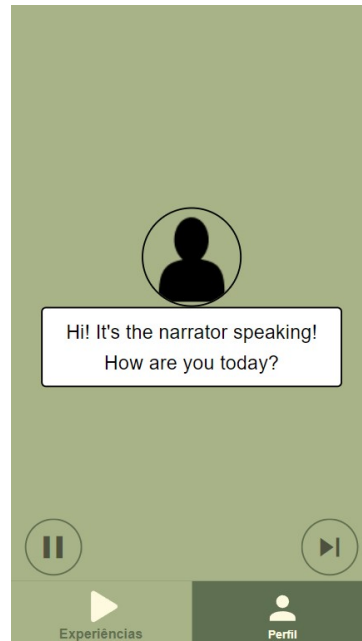


Figure 4.38: Dialogue Scene on the Story Player, with the icon on the bottom left indicating that audio is playing

When the player reaches a story-ending, the screen shown in Figure 4.39 is displayed, thanking the player for experiencing the story and communicating which ending they obtained. Pressing the "Finish" button will return the user to the Experiences selection screen, seen in Figure 4.32.

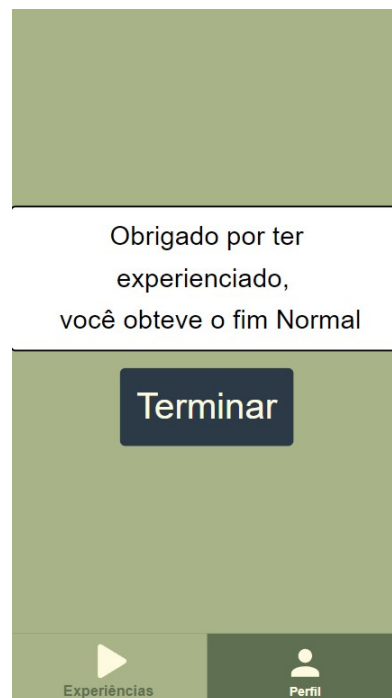


Figure 4.39: End Scene on the Story Player

4.4 Backend

The backend was implemented using NodeJS¹¹, using the Express package¹² to define routes. For the database, MongoDB¹³ was chosen for two main reasons. The first is that MongoDB stores data in BSON format, a binary representation of JSON-like documents. This integrated well with the Story Editor because of the ReactFlow nodes and edges format that already used JSON, therefore reducing the impedance mismatch between the data and the application logic. This enabled efficient storage and retrieval of story content and user profiles, among others. The Google Auth package¹⁴ handles the outsourcing of the authentication process to Google. For handling the upload of files from the Editor, the backend employs the multer package¹⁵.

4.4.1 File cleanup

The backend schedules a file check for every day at midnight. If a file's last accessed time surpasses six months, it is removed from the filesystem, relieving the system of unused content.

4.4.2 Marker generation

When the backend receives a request for marker generation, it will create a new Worker thread to handle it, as this is quite a CPU-intensive task that would block the main thread from handling other incoming requests.

The code that makes the marker generation possible is lifted from the NFT-Marker-Creator repository available at <https://github.com/Carnaux/NFT-Marker-Creator>. The Marker Creator provides a confidence value whose purpose is to inform on the quality of the marker, and it is calculated based on the parameters that make a good marker, already described in Section 4.2.10. When the processing is finished, three files are generated as Image Descriptors, with the .fset, .fset3, and .iset extensions. Image descriptors are quantitative representations of the visual characteristics of images. They are used to extract and describe the features of images in a way that can be analyzed and compared by computers. It is by looking for these features that ARjs can then detect the marker image and display the AR content.

¹¹NodeJS website: <https://nodejs.org/>

¹²ExpressJS website: <https://expressjs.com/>

¹³MongoDB website: <https://www.mongodb.com/>

¹⁴Google Auth NPM Package: <https://www.npmjs.com/package/google-auth-library>

¹⁵Multer NPM Package: <https://www.npmjs.com/package/multer>

Chapter 5

User testing

This chapter details the tests conducted on the applications to evaluate their usability, identify potential bugs in the systems, and propose possible solutions. The Story Editor can be consulted at storyweaver.netlify.app and the Story Player is hosted at storyplayer.netlify.app. You may consult the Appendix B for an example use of the tool as a low-fidelity prototyping tool.

5.1 Procedure adopted

Two types of tests were conducted. One type employed a Think Aloud protocol, encouraging users to vocalize their thoughts, feelings, and actions while using the application. The second type involved having users fill out a user experience survey, where they could evaluate their experience while using the applications.

Prior to participation, all participants were required to sign a consent document outlining the study's purpose, procedures, and confidentiality agreements. This document can be consulted in Appendix A.

5.1.1 Think Aloud Protocol

The first part of the test was made using the Think Aloud protocol which provides insights into the users' cognitive processes and helps identify usability issues and areas for improvement. Additionally, these tests allowed to observe whether the application functioned correctly when used differently by various users. Since different people try different things in different sequences, these tests helped uncover bugs that had gone undetected during internal testing. The goal for this phase of testing was to evaluate whether or not participants could complete certain pre-defined tasks and assess their level of success for each one.

Every test began by explaining the purpose of the test to the participants and making it clear that it was the application being tested, not their skills. Whatever way they chose to interact with the app was valid. Then, participants would be instructed to verbalize everything they are thinking, feeling, and doing as they navigate through the application. This included their expectations, confusion, frustrations, and any positive reactions. Afterward, they were given a simple instruction:

to use the Story Editor to create a story with more than one ending (ensuring at least one branching story path) where users could interact with Augmented Reality content. At least one of these AR media had to be triggered by a real-life location (to force participants to interact with the Map tab). After creating the story, testers then had to access the Story Player, login, and play through their story to the end.

5.1.2 User Experience Survey

After completing the test, participants were asked to fill out a form to gather some information about their background (age, level of education, previous experience with editing software) and to provide some general feedback on the usability of both apps.

Participants could answer the questions by choosing from a scale of 1 to 5, with 1 representing the most negative feedback and 5 representing the most positive. If users had not interacted with a specific feature during testing, they were asked to leave the answer blank. The questions were:

Story Editor

- How would you rate the overall usability of the application?
- How difficult was it to navigate your story structure?
- How difficult was it to use the "Inspector" tab to edit the content of your Scenes?
- How difficult was the character creation and editing process?
- How intuitive was the map calibration process?
- How useful was the "Map" tab to annotate the physical location of your experience?
- How difficult was it to export your story?
- How difficult was it to add your custom content to your story?
- How difficult was it to incorporate AR content in your experience?
- How clear were the explanations on how the different trigger modes for AR work?
- How clear were the export warning/error messages?
- How would you rate the application's performance in terms of responsiveness?
- How would you rate the application's performance in terms of speed?

Story Player

- How would you rate the overall usability of the application?
- How satisfied were you with the content displayed in the Profile tab?

- How satisfied were you with the Experience playing feature?

In addition to these questions, both sections of the form asked:

- Do you have any suggestions for new features?
- Were you ever lost at any moment during the test? If so, please explain why and where.
- Did you encounter any bugs during testing? If so, please describe them and how they were triggered.

5.2 Results

This section presents the results obtained from the user testing sessions. The testing sessions were conducted between the 10th of June and the 21st of June. The testing sessions were conducted in a mixed manner, with some users participating online by streaming their screens to the test supervisor, while others were conducted in person. In total, ten people volunteered to test the applications, with one of them being the project stakeholder.

5.2.1 Think Aloud Protocol

This section presents the results obtained from the Think Aloud portion of the user tests.

5.2.1.1 Story Editor

While participants used the Story Editor, they were expected to complete the following tasks:

- Task 1 - The user is able to start a project from scratch.
- Task 2 - The user is able to add a new character to the story.
- Task 3 - The user is able to connect story nodes to each other.
- Task 4 - The user is able to use the minimap to navigate the flowchart.
- Task 5 - The user is able to change the project title.
- Task 6 - The user is able to add new nodes to the flowchart.
- Task 7 - The user is able to access the dialogue flowchart window.
- Task 8 - The user is able to delete a scene from the flowchart.
- Task 9 - The user is able to zoom in and zoom out of the flowchart.
- Task 10 - The user can upload custom content.
- Task 11 - The user is able to delete edges from the flowchart.

- Task 12 - The user is able to save and load projects.
- Task 13 - The user is able to create a new map.
- Task 14 - The user is able to add places to the map.
- Task 15 - The user is able to add nodes on the dialog window.
- Task 16 - The user can export the story.
- Task 17 - The user is able to edit and delete a character from the story.
- Task 18 - The user can calibrate the map correctly.
- Task 19 - The user is able to add GPS-triggered AR content to their story and preview it successfully.
- Task 20 - The user is able to add Image Tracking - triggered AR content to their story and preview it successfully.
- Task 21 - The user is able to add QR-code-triggered AR content to their story and preview it successfully.
- Task 22 - User can customize their scenes using the Inspector tab.

The gathered data for the Story Editor tasks is presented in Table 5.1.

Users Tasks	U1	U2	U3	U4	U5	U6	U7	U8	U9	U10
T1	S	S	S	S	S	S	S	S	S	S
T2	S	S	S	S	S	S	S	S	S	S
T3	S	S	S	S	S	S	S	S	S	S
T4	S	S	S	S	S	S	S	S	S	S
T5	S	S	S	S	S	S	S	S	S	S
T6	S	S	S	S	S	S	S	S	S	S
T7	S	S	S	S	S	S	S	S	S	S
T8	S	S	S	S	S	S	S	S	S	S
T9	S	S	S	S	S	S	S	S	S	S
T10	S	S	S	S	S	S	S	S	S	S
T11	PS	S	PS	PS	S	S	S	S	S	S
T12	S	S	S	S	S	S	S	S	S	S
T13	S	S	S	S	S	S	S	S	S	S
T14	S	S	S	S	S	S	S	S	S	S
T15	S	PS	S	PS	S	S	PS	S	S	S
T16	S	S	S	S	S	S	S	S	S	S
T17	S	S	S	S	S	S	S	S	S	S
T18	S	PS	S	PS	S	PS	PS	S	PS	PS
T19	S	S	S	S	S	S	S	S	S	S
T20	S	S	S	S	S	S	S	S	S	S
T21	NP	NP	NP	S	S	S	S	S	S	S
T22	S	S	S	S	S	S	S	S	S	S

Table 5.1: Test Sheet for the Story Editor Tasks

The success was noted for each task as:

- S - Success -> The task was completed successfully without guidance from the test supervisor
- P - Partial Success -> The task was completed successfully but only after some guidance from the test supervisor
- F - Failure -> The task resulted in a bug or did not act as expected, which made it impossible to complete.
- NP - Not Performed -> The task was not performed.

Participants successfully completed most tasks. Regarding task 11 (The user is able to delete edges from the flowchart), there were difficulties in erasing edges from the story graph. As mentioned previously in Section 4.2.2.3, to delete an edge, a user had to first select the connection and then press the "Backspace" key. This turned out to be confusing for two users who thought that the "Delete" key was more intuitive and were therefore confused when they pressed it, but nothing happened. During the testing period (after User 3's session), the key was switched to the

"Delete" key, but then the same thing happened in reverse: User 4 attempted to delete edges using the "Backspace" key, but nothing happened.

Regarding task 15 (The user is able to add nodes on the dialog window), a bug was discovered when the user switched between multiple dialogue tabs that would sometimes revert the dialogue's state to what it was before the user opened the tab, which required the test supervisor's help to get around.

While attempting to complete task 18 (The user can calibrate the map correctly), six participants struggled to understand what was being asked of them and did not understand that they were calibrating a map at all, with two users believing that they were already inputting places, when they were only inserting the calibration anchors. The test supervisor had to intervene to explain the map calibration, and therefore, the result of this task ended up being a Partial Success for six participants.

Regarding task 21 (The user is able to add QR-code-triggered AR content to their story and preview it successfully), three users simply chose not to include QR-code-activated AR content in their stories.

5.2.1.2 Story Player

While participants used the Story Player, they were expected to complete the following tasks:

- Task 1 - The user is able to log in via Google Authentication
- Task 2 - The user is able to search for stories using the search bar
- Task 3 - The user is able to play a story from start to finish.
- Task 4 - The user is able to skip character dialogue
- Task 5 - The user is able to see AR content via Image Tracking.
- Task 6 - The user is able to see location-based AR content.
- Task 7 - The user is able to see AR content via QR-Code.
- Task 8 - The user is able to see their Story History on the Profile Tab.

The gathered data for the Story Player tasks is presented in Table 5.2.

Users Tasks	U1	U2	U3	U4	U5	U6	U7	U8	U9	U10
T1	S	S	S	S	S	S	S	S	S	S
T2	S	S	S	S	S	S	S	S	S	S
T3	S	S	S	S	S	S	S	S	S	S
T4	S	S	S	S	S	S	S	S	S	S
T5	S	S	S	S	PS	S	S	PS	S	S
T6	S	S	S	S	S	S	S	S	S	S
T7	NP	NP	NP	S	S	PS	S	PS	S	S
T8	S	S	S	S	S	S	S	S	S	S

Table 5.2: Test Sheet for the Story Player Tasks

The success was again noted for each task as:

- S - Success -> The task was completed successfully without guidance from the test supervisor
- P - Partial Success -> The task was completed successfully but only after some guidance from the test supervisor
- F - Failure -> The task resulted in a bug or did not act as expected, which made it impossible to complete.
- NP - Not Performed -> The task was not performed.

Task 7 (The user is able to see AR content via QR-Code) was not performed by users 1 through 3, as these participants did not complete task 21 of the Editor (The user is able to add QR-code-triggered AR content to their story and preview it successfully), neglecting to add any QR-code-triggered content to their stories. This task was also marked as a Partial Success for users 6 and 8, as these participants could not visualize their AR content with the QR code they had generated by themselves, requiring the test supervisor to position the device's camera in a specific position to trigger the content. This happened because the QR-code identifier that these participants chose ended up generating a QR code that worked poorly as a marker.

Task 5 (The user is able to see AR content via Image Tracking) was marked as a Partial Success for users 5 and 8 for the same reason. In their cases, the image chosen to trigger the AR content ended up being low in detail and possessing a poor resolution, both factors that contributed to their images being poor markers.

5.2.2 User Experience Survey

To better analyze the collected data, some characteristics of the volunteers were gathered, as mentioned in the previous section. Figure 5.1 displays the age distribution of participants, while Figure 5.2 displays the distribution of participants' educational levels.

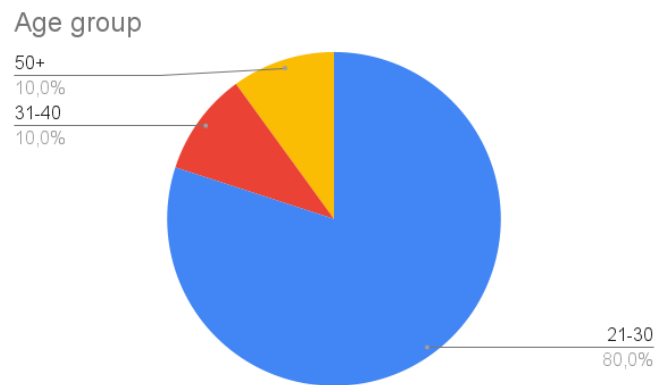


Figure 5.1: Test participant Age Distribution

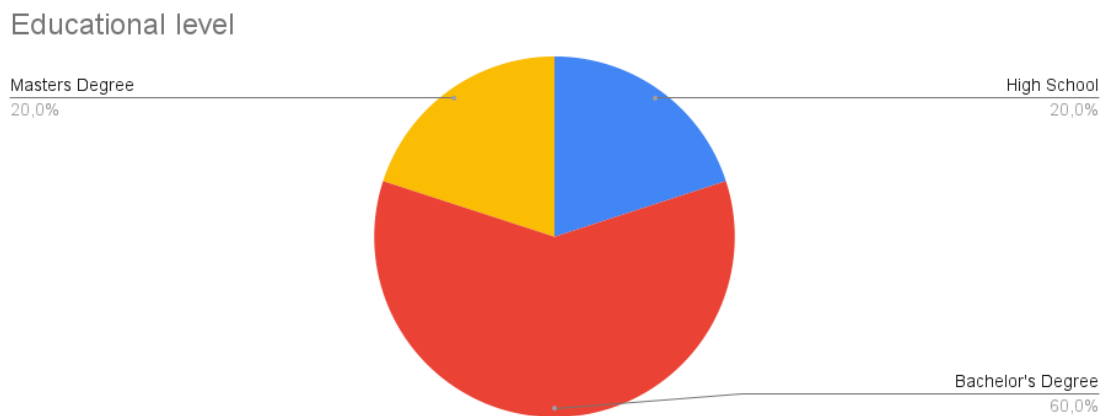


Figure 5.2: Test participant Educational Levels

No participants had interacted with editing software of this kind previously. The results of the User Experience Survey are presented in the appendix C. Figures C.1 through C.13 showcase the results of the Story Editor portion of the User Experience survey, while Figures C.14 to C.16 display the results to the Story Player portion of the survey.

5.3 Discussion

5.3.1 Think Aloud Protocol

This section details the discussion surrounding the results from the Think Aloud Protocol portion of user testing.

5.3.1.1 Story Editor

Considering the gathered data, the applications can be considered robust and functional, given that 83% of tasks were completed without any supervisor assistance. The tests proved to be important,

as the protocol allowed us to identify the users' thoughts, revealing key aspects that will now be discussed. Some users expressed some difficulty when using the Dialogue tab, specifically some confusion on how the multiple endings inside the dialogue related to the endings displayed on the Scene node in the Story tab. A participant attempted to paste YouTube links to an audio field with the expectation that the app would extract the sound from the video. A few participants expressed confusion about what they were editing when using the Inspector tab.

Some minor errors were detected during the tests. Image Scenes were displaying an "Image not found" error message before the user had added an image on the Inspector tab which was later fixed. An "Invalid coordinates" warning message on the Map tab was displaying with a green success color which was quickly resolved as well. Quiz Scene answer handles (the place from which users begin in order to draw the connections between Scenes) were originally not lined up with their corresponding text, which caused confusion for some users. This was later patched.

Participants also provided extra feedback: one participant suggested that the multiple tabs should be made draggable and repositionable. Multiple participants would have liked to have the possibility of setting background music for the whole duration of the experience. Two users suggested the inclusion of text formatting tools. One tester expressed that they would have liked to be able to adjust the positioning of the background images on Scenes. One user said they would have liked to be able to resize individual scenes on the graph. 2 users suggested the addition of a button to automatically format the story structure. Almost every tester expressed that adding CTRL+C (copy), CTRL+V (paste), and CTRL+Z (revert) functionality would drastically improve the editor's usability. One participant suggested the possibility of edges with editable colors. One participant proposed that clicking on a specific icon on the Scene node should open the Inspector on the corresponding section. Another participant suggested the addition of hotkeys for commonly executed actions. Another user suggested adding text-to-speech for the dialogues. One participant recommended including volume controls for audio content. Another participant suggested the implementation of implicit variables, that is, dynamic variables that take on different values depending on how players interact with the story. Some participants wanted a way for players to be able to take photos during the experience and then be able to share them on social media, which could be implemented as a "User Interaction" Scene.

5.3.1.2 Story Player

The Story Player can be deemed robust, with 75% of tasks being completed successfully by all participants. The Partial Success obtained on task 5 (The user is able to see AR content via Image Tracking) was obtained in cases where little care was taken when choosing a marker image. This may indicate that the Editor should be informing users better about what constitutes a good marker image. No additional feedback for the Story Player was provided, which can be attributed to it being a simple application with a relatively small scope.

5.3.2 User Experience Survey

The results from the User Experience Survey demonstrate that the Editor's usability is quite satisfactory, with the overall usability average score being 3.8 out of 5. Participants provided high ratings for various aspects of the app, indicating positive user experiences in several key areas. Specifically, users rated the story structure navigation (average score of 4.3/5), Inspector tab usability (average score of 3.7/5), character creation (average score of 4.3/5), story exporting (average score of 4.6/5), custom content integration (average score of 4.5/5), and AR integration (average score of 3.5/5) highly, suggesting that these features are well-designed and user-friendly. The app can also be considered responsive and fast, with participants rating these aspects highly in the satisfaction survey. The responsiveness of the app received an average score of 3.9 out of 5, while its speed was rated even higher, with an average score of 4.4 out of 5. These ratings indicate that users generally found the app to be efficient and quick in its operations, contributing positively to the overall user experience.

However, the survey also highlighted an area in need of improvement. The Map tab was identified as the least intuitive part of the app, corroborating findings from the earlier "Think Aloud" tests. The majority of participants gave a rating of "2" for the map calibration process (average score of 2.4), indicating significant challenges in understanding and using this feature. The Map tab's utility to annotate an experience's physical location did not fare much better, obtaining an average score of 3.1 from participants. This feedback suggests that while the overall app usability is strong, specific enhancements are required for the Map tab to ensure a more seamless and intuitive user experience.

As for the Story Player, the results from the survey show a high level of satisfaction with the product. Overall usability was rated with an average score of 4.1, indicating that users generally found the player easy to use. The information provided in the profile tab was particularly satisfactory to users, who rated it with an average score of 4.4. Additionally, the experience of playing the stories through the app was well-received, receiving an average score of 3.9. These ratings suggest that the story player was appreciated by users and performed effectively in delivering a positive user experience.

Overall, most of the participants showed interest and seemed to be satisfied with the app's potential, namely the AR component.

5.4 Improvements After User Testing

This section outlines changes made to the Story Editor application made as a result of the user testing process.

Tasks 15 (The user is able to add nodes on the dialog window) and 11 (The user is able to delete edges from the flowchart), which obtained some Partial Success results, were addressed after the period of testing. The bug that arose during the execution of task 15 was fixed, while the confusion that arose while deleting an edge was eliminated by simply assigning both "Delete"

and "Backspace" keys to the deletion of a connection. In order to address the Partial Successes of task 18 (The user can calibrate the map correctly), the instruction text on the Map tab was given some more relevance with the addition of a white background and border around it, but further refinements could be implemented in order to bring this component of the application to a more satisfactory level of usability. This is discussed further in Section 6.4.5.

The confusion that participants felt while using the Inspector tab arose from the fact that they could not see the Scene node being edited in real-time due to being too zoomed out in the Story tab. To address this, when a user now selects a Scene to edit, the application will zoom in and center the story graph on that Scene. This way, users can see their actions' immediate feedback.

Copy-paste functionality was added to the application. When a user selects a Scene and presses CTRL+C, it will appear selected, as can be seen in Figure 5.3. When the user presses CTRL+V, a new Scene with the same properties is created on the story graph. Additionally, selecting a Scene and pressing CTRL+X will delete it from the graph. The addition of these common shortcuts should make users more comfortable with using the application.

Quality-of-life changes were also made. Some test participants had some trouble figuring out that Scenes needed to be connected in order for transitions between them to happen. To prevent this, when a user adds their first Scene to the Story, it is automatically connected to the Begin Scene.

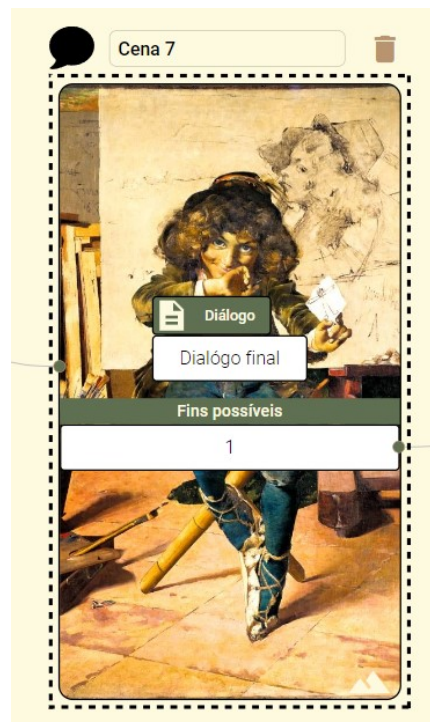


Figure 5.3: Scene node selected for copying

Chapter 6

Conclusions

The primary goal of this thesis was to develop an AR story editor that empowers users without programming backgrounds, such as historians, curators, and tour guides, to create and edit interactive multimedia narratives. By building on the foundations laid in the initial iteration of the FronTowns project, which focused on 3D historical recreations (Cardoso, 2023), this thesis sought to extend the capabilities of the system to include Augmented Reality features. The tool aimed to support the creation of AR-enabled visits to cultural heritage sites in collaborative environments. Our proposal addresses historians' critical need to craft narratives by leveraging the benefits of AR to embed these stories within cultural heritage locations and by providing a user-friendly authoring tool that caters to a broad spectrum of users without programming experience. This fusion of storytelling and technology fulfills their creative requirements, providing them with a robust tool that supports the high-fidelity prototyping process. It enables users to bring their imaginative narratives to life, seamlessly integrating various multimedia elements and AR features. As a result, it enhances their ability to craft immersive and engaging cultural heritage experiences that captivate and educate their audience.

6.1 Summary

To achieve these goals, a systematic review of existing literature and technologies was conducted. This review identified the current state of AR story editors, highlighting gaps and opportunities for improvement. Meetings with the stakeholders were instrumental in refining the project goals, ensuring they were aligned with their practical needs.

The Design Science Research (DSR) methodology was employed to guide the development process, which involved several key phases. Firstly, requirements were created based on the insights from the systematic review and stakeholder meetings. These included usability, accessibility, and functionality for non-technical users. Following this, a solution was proposed to meet these requirements. Then, the proposed solutions were developed using modern web technologies. The AR story editor and player were built using React, a popular JavaScript library for building

user interfaces, while the backend infrastructure was developed with NodeJS. Finally, the developed solutions were evaluated through user testing and feedback sessions. This iterative process allowed for continuous refinement, ensuring the final product met user needs and expectations.

6.2 Contributions

This section aims to answer the initial question: "How can co-authoring tools enhance the collaborative creation of AR narratives?". In order to provide an answer, it will begin by explaining how the two proposed objectives were achieved.

6.2.1 Determine the optimal degree of freedom that should be provided to users (historians, curators, and tour guides)

In designing the AR story editor for cultural heritage experiences, the degree of freedom afforded to users was a critical consideration in balancing creative autonomy with ease of use and consistency in the user experience. In the end, a high level of freedom was given to users. In terms of creative control, users have extensive creative control to craft unique and personalized narratives, including the ability to integrate various multimedia elements (images, videos, sound, text) and arrange them in the ways that best tell their story, like in the Metaverse Studio platform. Users are also able to customize interactions, such as triggers for AR content, being able to choose between geolocation (as is offered by the ARWay platform), and image recognition. Additionally, the capability for users to create branching story paths and multiple endings, like present in [Vrettakis et al. \(2019\)](#), reinforces the high degree of freedom offered by the Editor. The app also allows users to upload custom assets like defended by [Brett and Hargood \(2023\)](#) instead of relying on libraries of common assets, which elevates the editor's level of freedom.

Additionally, determining the optimal degree of freedom for users was validated by user testing. Through this process, it was concluded that the application offers a satisfyingly high degree of freedom, empowering users to tailor their narratives precisely to their creative visions and storytelling needs. This approach not only supported creative expression but also enhanced user engagement and storytelling capabilities within the application.

6.2.2 Design the user interface of the AR story editor to ensure usability for individuals without programming backgrounds.

Designing the user interface of the Story Editor to ensure usability for individuals without programming backgrounds involved several key principles and strategies. The goal was to create an intuitive, accessible, and efficient tool that supports users in creating engaging AR narratives with minimal technical barriers. The interface was designed with simplicity in mind, with the purpose of avoiding clutter and keeping the layout clean, with clear navigation and minimal distractions. By using consistent design elements, as recommended by [Not and Petrelli \(2019\)](#), such as icons

and colors, throughout the interface, it was hoped to create a predictable and easy-to-navigate environment. The implementation of a drag-and-drop interface, like the one implemented in [Batsaras and Xinogalos \(2023\)](#) and mentioned by [Göbel et al. \(2008\)](#) and [Kauhanen et al. \(2007\)](#), allowed users to create interactions and narrative flows through graphical representations rather than text-based coding as present in [Schenk et al. \(2021\)](#)'s previous tool. The application's attention to responsive feedback, as recommended by [Economou et al. \(2008\)](#) and [Brett and Hargood \(2023\)](#), provided immediate validation that helped users understand the impact of their actions right away. As [Winterbottom and Blake \(2004\)](#) proposed, error messages were made easily comprehensible for end-users while remaining non-intrusive, never getting in the way of the creation process.

The iterative design approach allowed for continuous UI refinements based on feedback. This ensured the tool remained relevant and user-friendly over time.

During user testing, participants without programming experience were able to seamlessly integrate AR components and diverse media elements, including video, audio, images, and 3D models, into their applications. This proves that the editor's interface facilitated straightforward editing, enabling users to configure AR overlays and multimedia content. The inclusion of the AR preview feature also contributed massively to usability by allowing users to nail the positioning of AR elements and see how the changes they made were reflected in the AR environment without needing to switch back and forth between the Player and the Editor.

By prioritizing simplicity and accessibility, the editor can empower historians, curators, and other non-technical users to create engaging and immersive AR experiences at cultural heritage sites without requiring advanced technical knowledge or alternatively. Alternatively, it can serve as a powerful tool for high-fidelity prototyping, allowing users to experiment with and refine their AR narratives, ensuring precision and effectiveness before final deployment.

6.2.3 How can co-authoring tools enhance the collaborative creation of AR narratives?

The Story Editor enhances the collaborative creation of AR narratives by providing a user-friendly interface and robust functionalities tailored for non-programmers, such as historians, curators, and tour guides. The tool facilitated seamless collaboration among users through intuitive features like drag-and-drop controls, customizable templates, and real-time editing capabilities. The implementation of the Story Editor enhanced collaborative creation by facilitating real-time collaboration, improving communication, and providing a structured yet flexible framework for project management. Users were able to use the application for high-fidelity prototyping but also as a way to create end-user-ready experiences. It enabled co-authors to contribute their expertise and creative input effectively, allowing them to integrate diverse multimedia elements such as images, videos, audio, and 3D models into AR experiences. The Editor can contribute significantly to the efficiency of the co-creation process by streamlining workflows, improving coordination, and enabling quick feedback. By facilitating the content creation process and promoting collaboration,

the authoring tool can empower teams to brainstorm ideas, refine storylines, and iterate on narratives collaboratively. This approach not only accelerates the development timeline but also ensures that AR narratives are rich in historical context and engaging for audiences visiting cultural heritage sites. Furthermore, the tool can foster creativity by bringing together diverse perspectives and inspiring team members through collaborative synergy. They can also support multimedia teams in accelerating application development as an initial iteration cycle leveraging the Editor has been completed. With this initial phase of design and ideation of AR experiences already addressed, the subsequent stages of development and testing (with stakeholders and users), followed by implementation and dissemination, can be carried out more efficiently. This efficiency is achieved because the final product is more likely to align with what was collaboratively created, ensuring that the developed solutions better meet the needs and expectations of the end users.

6.3 Limitations

The Story Editor currently does not allow users to personalize the interface to their liking. Allowing users to customize certain aspects of the interface to suit their preferences, such as theme colors, font sizes, and workspace layout, could enhance user comfort and efficiency. This is further elaborated in Section 6.4.1.

In retrospect, while still providing creative options, the tool should offer templates and guided workflows to assist users who might feel overwhelmed by too many choices. These templates could serve as starting points, which users could then customize to their needs.

An argument could be made that the upload of 3D models is a task too complex for the target audience of historians and museum curators, but unfortunately, this was the only way the feature could be integrated into the app due to time constraints. As mentioned in Section 6.4.5, an integration with Sketchfab API could possibly remedy this issue.

Additionally, it could be argued that the Map tab's calibration process also falls beyond the realm of achievability for the target audience, which may not be able to input the correct GPS coordinates in order for the process to succeed. While the 'Use my Position' button makes it so that users may simply walk to the location to input the GPS coordinates, this could still prove to be difficult. A better solution to this could be integrating with Open Street Maps as described further in Section 6.4.5, which would avoid having to input coordinates at all.

The Editor's collaborative functionality could be fostered to a higher degree if the Editor implemented real-time collaboration features in the same vein of Figma¹ for instance. Functionalities such as simultaneous editing with real-time updates and synchronization, complemented by in-app messaging, comments, and annotations, would go a long way in enhancing the collaborative aspect of the Editor. This is further described in Section 6.4.3.

One limitation of the project was a lack of extensive user testing. While valuable feedback was extracted from the sessions that were conducted, the limited number of participants and testing scenarios may not have fully captured the diverse range of user needs and potential issues.

¹Website: <https://www.figma.com/>

Additionally, the main age group of participants was between 21 and 30, an age range that typically possesses higher technology literacy compared to the rest of the population, which may compromise the validity of the tests when considering our initial objectives of assuring a user-friendly design for non-programmers. These constraints could affect the generalizability of the findings and the robustness of the editor's usability across different user groups and contexts.

6.4 Future Work

The development and implementation of a user-friendly AR story editor for cultural heritage experiences marks a significant advancement in democratizing AR content creation. However, the journey towards fully realizing the potential of this tool is ongoing. Future work should focus on several key areas to enhance the capabilities, usability, and impact of the AR story editor, ensuring it remains relevant and effective for a diverse range of users.

6.4.1 Workspace Customization

Workspace customization tools should be added to the Editor. When users spend a long time interacting with any given interface, the need to customize it arises naturally. Features such as tab repositioning, font settings, and icon size and style should be strongly considered as they would not only improve personalization but also increase the app's accessibility, allowing users to change the app's layout to better suit their visual preferences. Color scheme selection, such as offering a choice between light and dark themes, should be considered to reduce eye strain. This could be taken even further by enabling users to create their own color schemes for the interface to enhance visual comfort and personal style. Additionally, the implementation of keyboard shortcuts for common actions would allow users to customize inputs for various actions, making it easier to perform frequent actions quickly.

6.4.2 Tutorials

The inclusion of tutorials could significantly lower the learning curve of the Editor. Users could access tutorials and guides focused on specific tools/workflows directly within the workspace. Additionally, a contextual help feature that provides tips and information based on the current task or tool being used (toggled on or off according to user preference) could be implemented.

6.4.3 Collaboration

The introduction of real-time collaborative features such as simultaneous editing would allow multiple users to work on the same project simultaneously, with real-time updates and synchronization. This would imply implementing a robust version control system that would ensure that changes are tracked and conflicts are managed effectively. An in-app chat functionality should complement this system to enable seamless communication between collaborators, thus reducing

the need to switch between different platforms and keeping the focus on the task at hand. Furthermore, adding the possibility for users to leave comments and annotations directly on the story graph would facilitate discussion and feedback without disrupting the workflow. Additionally, collaboration with generative AI, namely image and text generation, can be integrated into the application. Generative AI can assist in creating realistic historical images, enriching narratives with compelling text, and providing adaptive content tailored to specific cultural heritage sites, allowing users to produce high-quality multimedia elements without extensive technical skills.

6.4.4 User suggestions

This subsection details possible additions to the app that were derived from either direct user feedback or by observing how users interacted with the Editor.

Firstly, the addition of text formatting tools was by far the most requested feature. Since users tried to add YouTube links to audio fields during testing, perhaps the integration of the backend with an API should be considered in order to extract the audio of YouTube videos as a .mp3 file. For further tester suggestions, please consult Section 5.3.

6.4.5 Usability

This subsection details features that would improve the usability of the Editor by users with little computer skills if included. Firstly, integration of the Map tab with OpenStreetMaps could be considered to avoid having users manually insert coordinates. Additionally, since OpenStreetMaps is unlikely to provide the same level of venue detail as the current solution of uploading a custom floor plan, perhaps the annotation capacities of the Map tab could be expanded to a sort of Paint-like environment, allowing users to draw on their map, although this requires more investigation to determine its feasibility.

Furthermore, to avoid having users manually upload 3D models, integration of the app with the Sketchfab API, which allows the download of models, could be considered, which would make it so that users simply had to add the ID of the model they desire.

Offering a library of ready-made templates and recipes would greatly simplify the creation process, allowing users to focus on customizing content rather than building from scratch. Additionally, adding a media library of shared resources would take some pressure of having to insert custom content for everything from users.

More extensive user usability testing, ideally incorporating the implementation of test heuristics, should be carried out in the future. Conducting tests with individuals who possess more know-how in the cultural area, such as historians and museum guides, would provide better insights and feedback. Their expertise and familiarity with cultural heritage content would enable them to evaluate the AR story editor more critically and offer suggestions that align more closely with the practical needs and challenges faced in their field. This targeted feedback would be invaluable in refining the tool to ensure it truly meets the requirements of its intended user base, leading to a more effective and relevant application.

6.4.6 Story Player

The Story Player could greatly benefit from some form of social media integration. Users might become more motivated to play and engage with the experiences if they are given the chance to publish their particular experience outcomes on social media. Initially, players of the stories would be able to see a minimap displaying their current location on the venue on the corner of the screen while playing an experience. Due to time constraints, this couldn't be integrated into the final version of the Player, but it would add great value to the Story Player if implemented.

Additionally, the addition of tags on stories was meant to allow the choice of an experience based on its content. A functionality to allow the filtering of experiences based on tags should be added to the Story Player.

6.4.7 Other

Images should be compressed and resized before being uploaded to the backend in order to save space and reduce load times. This would also have the added benefit of reducing data usage for the players of the experience.

Although this project focused on the application of AR-enhanced storytelling in a cultural context, these kinds of tools are very much applicable for educational purposes, so it would be interesting to do some additional research focusing on the possible applications of these kinds of tools in that area.

References

- Abawi, D. F., Reinhold, S., and Dörner, R. (2004). A toolkit for authoring non-linear storytelling environments using mixed reality. In *Technologies for Interactive Digital Storytelling and Entertainment: Second International Conference, TIDSE 2004, Darmstadt, Germany, June 24-26, 2004. Proceedings 2*, pages 113–118. Springer.
- Abderrahim, L. and Cigerci, F. M. (2018). A systematic review of research undertaken into the use of digital storytelling as a pedagogical tool in the language classroom. In *Oxford Conference Series| January 2018*, page 113.
- Batsaras, C. and Xinogalos, S. (2023). A comparative analysis of low or no-code authoring tools for location-based games. *Multimodal Technologies and Interaction*, 7(9):86.
- Brett, J. and Hargood, C. (2023). Authoring tools for mixed reality.
- Cardoso, J. A. L. (2023). Authoring tool for multiplatform interactive digital storytelling.
- Carmigniani, J. and Furht, B. (2011). *Augmented Reality: An Overview*, pages 3–46.
- De Paolis, L. T., Gatto, C., Corchia, L., and De Luca, V. (2023). Usability, user experience and mental workload in a mobile augmented reality application for digital storytelling in cultural heritage. *Virtual Reality*, 27(2):1117–1143.
- Economou, D., Gavalas, D., Kenteris, M., and Tsekouras, G. E. (2008). Cultural applications for mobile devices: Issues and requirements for authoring tools and development platforms. *SIGMOBILE Mob. Comput. Commun. Rev.*, 12(3):18–33.
- Erdem, A. T., Utku, B., Abaci, T., and Erdem, Ç. E. (2009). Advanced authoring tools for game-based training. In *Proceedings of the 2009 Summer Computer Simulation Conference*, pages 95–102. Citeseer.
- Ganapathi, S. P. and Gokhale, S. (1995). Development of an authoring system for engineering education for pc-dos applications.
- Göbel, S., Salvatore, L., and Konrad, R. (2008). Storytec: A digital storytelling platform for the authoring and experiencing of interactive and non-linear stories. In *2008 International Conference on Automated Solutions for Cross Media Content and Multi-Channel Distribution*, pages 103–110. Ieee.
- Green, D., Hargood, C., and Charles, F. (2018). Define "authoring tool": A survey of interactive narrative authoring tools.
- Green, D., Hargood, C., and Charles, F. (2021). Use of tools: Ux principles for interactive narrative authoring tools. *J. Comput. Cult. Herit.*, 14(3).

- Hevner, A. R. (2007). A three cycle view of design science research. *Scandinavian journal of information systems*, 19(2):4.
- Kauhanen, M., Eaket, C., and Biddle, R. (2007). Patterns for story authoring tools. In *EuroPLOP*, pages 19–36. Citeseer.
- Kelliher, A. and Slaney, M. (2012). Tell me a story. *IEEE MultiMedia*, 19:4.
- Kuhn, J. (2018). Minecraft: Education edition. *CALICO Journal*, 35(2):214–223.
- Lambert, J. (2013). *Digital storytelling: Capturing lives, creating community*. Routledge.
- Laurent, M., Monnier, S., Huguenin, A., Monaco, P.-B., and Jaccard, D. (2022). Design principles for serious games authoring tools. *International Journal of Serious Games*, 9(4):63–87.
- Massidda, M., Travaglini, L., and Pescarin, S. (2022). Interactive digital narrative authoring tools and hybrid experiences in cultural heritage: An integrated review. pages 31–35. Eurographics Association.
- Massidda, M., Travaglini, L., and Pescarin, S. (2023). Interactive Digital Narrative Authoring Tools and Hybrid Experiences in Cultural Heritage: An Integrated Review. In Bucciero, A., Fanini, B., Graf, H., Pescarin, S., and Rizvic, S., editors, *Eurographics Workshop on Graphics and Cultural Heritage*. The Eurographics Association.
- Moallem, J. D. and Raffae, W. L. (2020). A review of agency architectures in interactive drama systems. In *2020 IEEE Conference on Games (CoG)*, pages 305–311. IEEE.
- Nakevska, M., van der Sanden, A., Funk, M., Hu, J., and Rauterberg, M. (2017). Interactive storytelling in a mixed reality environment: The effects of interactivity on user experiences. *Entertainment Computing*, 21:97–104.
- Narawi, N. A. A., Abd Rahman, H., Mohamed Shaari, N. A., and Wan Hussin, W. Y. (2020). Interactive story graph structure for digital storytelling. *Malaysian Journal of Computing (MJoC)*, 5(2):619–634.
- Nisi, V. (2017). The changing panorama of interactive storytelling: a review from locative to transmedia. *DOC online - Revista Digital de Cinema Documentário*, pages 43–68.
- Not, E. and Petrelli, D. (2019). Empowering cultural heritage professionals with tools for authoring and deploying personalised visitor experiences. *User Modeling and User-Adapted Interaction*, 29(1):67–120.
- Quah, C. Y. and Ng, K. H. (2022). A systematic literature review on digital storytelling authoring tool in education: January 2010 to January 2020. *International Journal of Human-Computer Interaction*, 38(9):851–867.
- Riedl, M., Li, B., Ai, H., and Ram, A. (2011). Robust and authorable multiplayer storytelling experiences. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 7(1):189–194.
- Riedl, M. O. and Bulitko, V. (2012). Interactive narrative: An intelligent systems approach. *AI Magazine*, 34(1):67.

- Schenk, K., Lari, A., Church, M., Graves, E., Duncan, J., Miller, R., Desai, N., Zhao, R., Szafron, D., Carbonaro, M., and Schaeffer, J. (2021). Scriptease ii: Platform independent story creation using high-level patterns. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 9(1):170–176.
- Simon, H. A. (1996). *The sciences of the artificial*. MIT press.
- Spanoudakis, M., Nakou, A., Meliadou, E., Gouscos, D., and Meimaris, M. (2015). Digital words of wisdom? *Cultural Science Journal*, 8(2):108–130.
- Torres, A., Kapralos, B., Da Silva, C., Peisachovich, E., and Dubrowski, A. (2022). Moirai: A no-code virtual serious game authoring platform. In *Virtual Worlds*, volume 1, pages 147–171. MDPI.
- Vayanou, M., Katifori, A., Karvounis, M., Kourtis, V., Kyriakidi, M., Roussou, M., Tsangaris, M., Ioannidis, Y., Balet, O., Prados, T., et al. (2014). Authoring personalized interactive museum stories. In *Interactive Storytelling: 7th International Conference on Interactive Digital Storytelling, ICIDS 2014, Singapore, Singapore, November 3-6, 2014, Proceedings 7*, pages 37–48. Springer.
- Verkuyl, M., Dubois, N., Goldsworthy, S., Merwin, T., Willet, T., and Job, T. (2022). Virtual simulation: An educator’s toolkit (v. 1.9-february 27, 2023).
- Verzini, A. F. M., Tonelli, J. I., Challiol, C., Lliteras, A. B., and Gordillo, S. E. (2015). Authoring tool for location-aware experiences. In *Proceedings of the 2015 Workshop on Narrative & Hypertext, NHT ’15*, page 21–25, New York, NY, USA. Association for Computing Machinery.
- Vrettakis, E., Kourtis, V., Katifori, A., Karvounis, M., Lougiakis, C., and Ioannidis, Y. (2019). Narralive—creating and experiencing mobile digital storytelling in cultural heritage. *Digital Applications in Archaeology and Cultural Heritage*, 15:e00114.
- Winterbottom, C. and Blake, E. (2004). Designing a vr interaction authoring tool using constructivist practices. In *Proceedings of the 3rd international conference on Computer graphics, virtual reality, visualisation and interaction in Africa*, pages 67–71.

Appendix A

Informed Consent

The following text was read and signed digitally by all participants before engaging in the testing of the applications.

A.1 Consentimento Informado para Participação em Teste de Aplicação

Descrição do Projeto: O meu nome é Miguel Freitas, sou aluno do Mestrado em Engenharia Informática e Computação da Faculdade de Engenharia da Universidade do Porto, sob a orientação da Professora Maria van Zeller e coorientação do Professor António Coelho, ambos da Faculdade de Engenharia da Universidade do Porto. A investigação centra-se na criação de um editor de histórias com conteúdo de Realidade Aumentada que auxilie historiadores a criar experiências no contexto de visitas a sítios de património cultural.

Procedimento: Durante este teste, você será solicitado a utilizar a aplicação e completar uma série de tarefas. Irão ser coletados dados sobre como você interage com a aplicação. Também poderemos pedir que responda a questionários ou participe de entrevistas para fornecer feedback sobre sua experiência.

Confidencialidade: Todas as informações coletadas durante este teste serão mantidas estritamente confidenciais e serão usadas apenas para fins de pesquisa e desenvolvimento da aplicação. Nenhuma informação pessoal que possa identificá-lo será compartilhada com terceiros sem o seu consentimento explícito.

Voluntariedade: A sua participação é totalmente voluntária. Você pode optar por não participar ou retirar o seu consentimento a qualquer momento, sem qualquer prejuízo. Se decidir retirar-se do estudo, todos os dados coletados até aquele ponto serão excluídos.

Benefícios e Riscos: Embora não haja benefícios diretos para você ao participar deste estudo, a sua contribuição será valiosa para o desenvolvimento de uma aplicação melhorada. Não prevemos riscos significativos associados à sua participação, além dos inconvenientes menores associados ao uso normal de um software em desenvolvimento.

Contato: Se você tiver qualquer dúvida ou preocupação sobre este teste ou sobre seus direitos enquanto participante, por favor, entre em contato pelo e-mail [radiodomiguel70@gmail.com] ou pelo telefone [917377278].

Consentimento: Ao assinar em baixo, você indica que leu e compreendeu as informações fornecidas acima, que teve a oportunidade de fazer perguntas e que consente voluntariamente em participar deste teste. Declaro que aceito participar neste estudo e autorizo a gravação das interações, discussões e contribuições feitas por mim durante o teste, para efeitos de análise e tratamento dos dados para a investigação de mestrado. Compreendo que os dados obtidos serão tratados de forma a garantir o anonimato dos participantes, sendo utilizados apenas para os propósitos da investigação e destruídos após o seu tratamento e conclusão do mestrado do estudante, no período máximo de dois anos.

Appendix B

Low-fidelity Prototyping Example

The following images were taken during the use of the Story Editor application for low-fidelity prototyping purposes. These drawings were created in co-creation workshops at the Gallery of Biodiversity, conducted in February 2020. More info on the co-creation workshop can be found at <https://hdl.handle.net/10216/140610>.

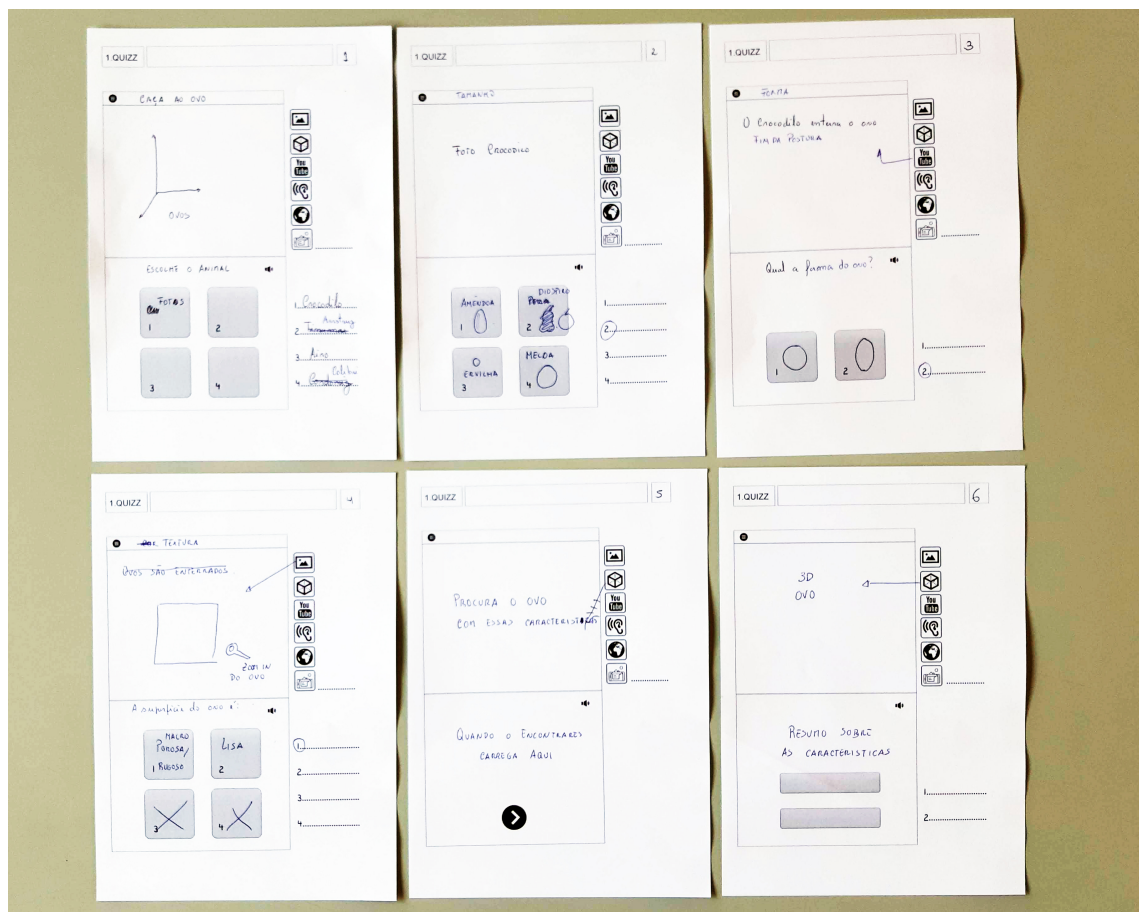


Figure B.1: All Screens in paper format

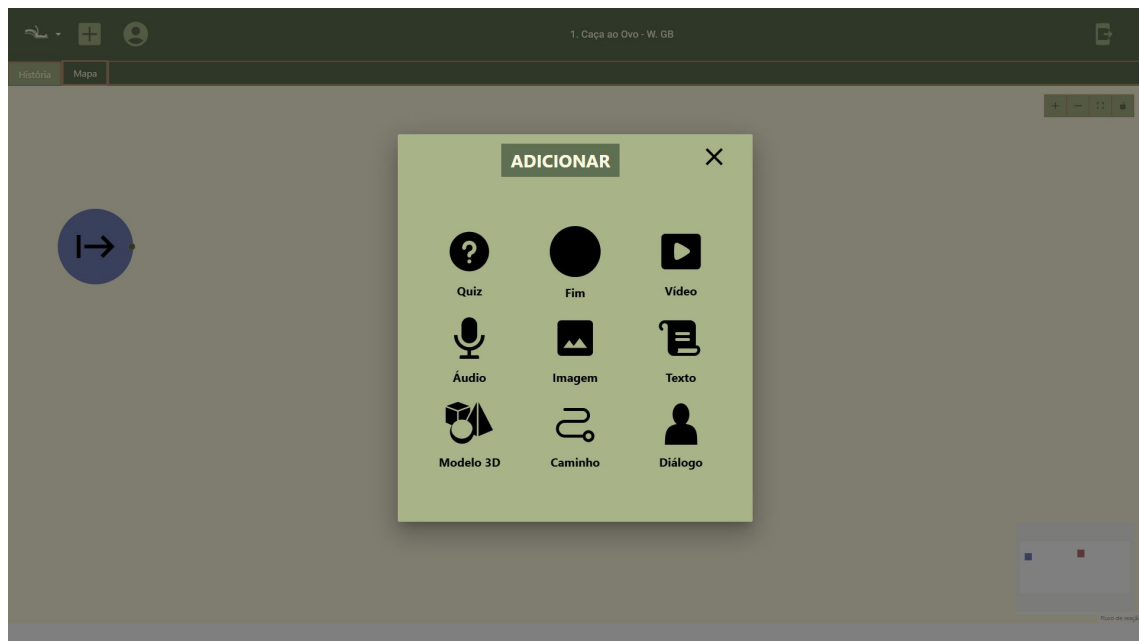


Figure B.2: Story creation process beginning

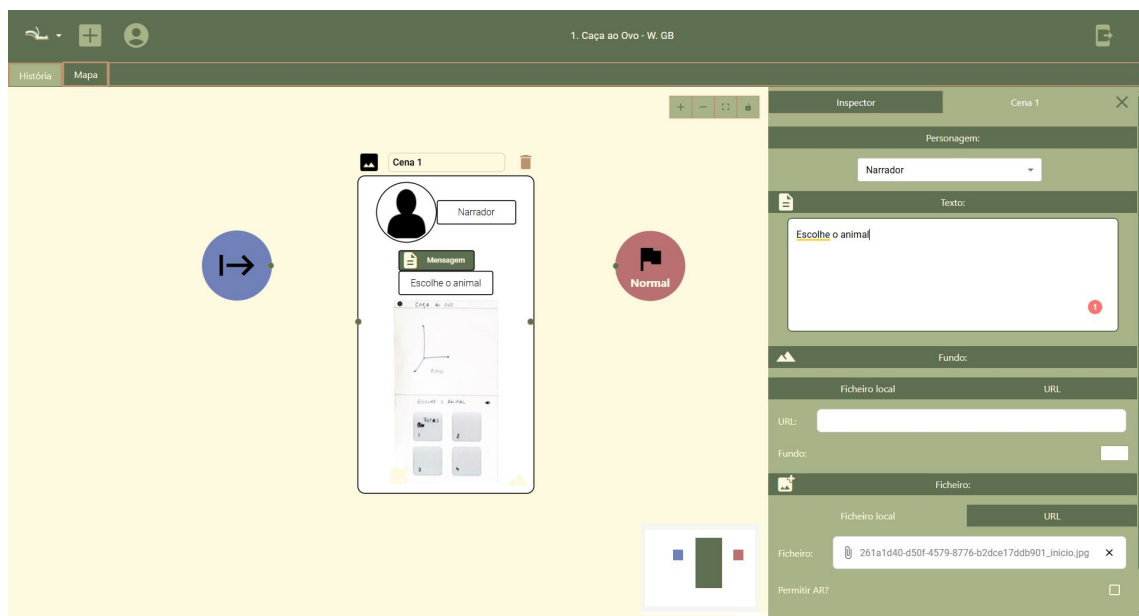


Figure B.3: First Scene in the Story Editor

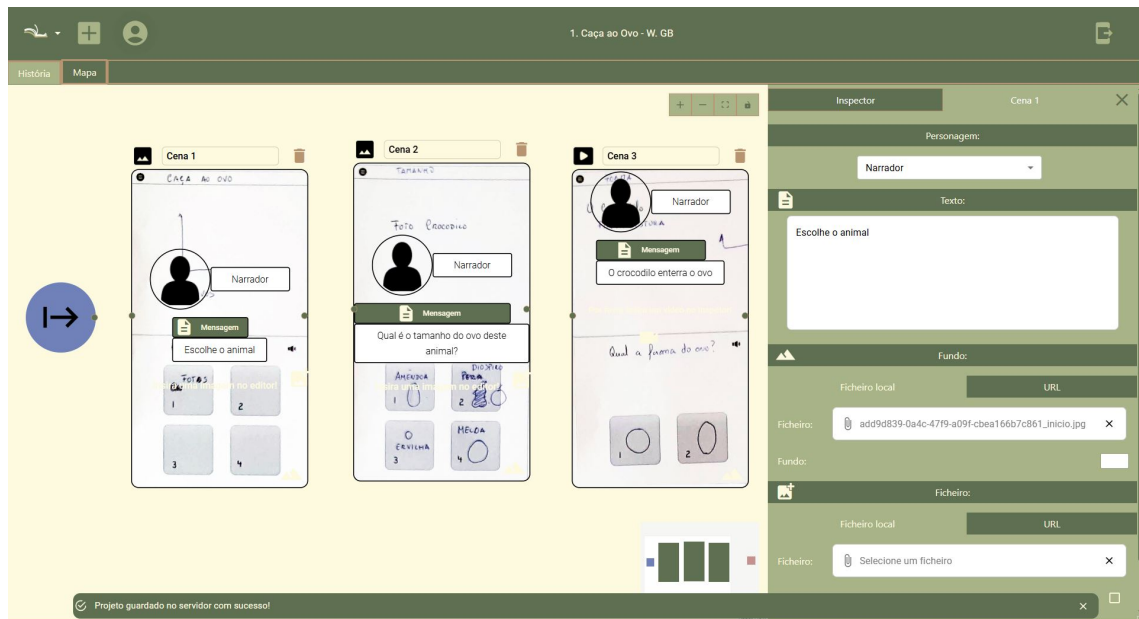


Figure B.4: Second Scene in the Story Editor

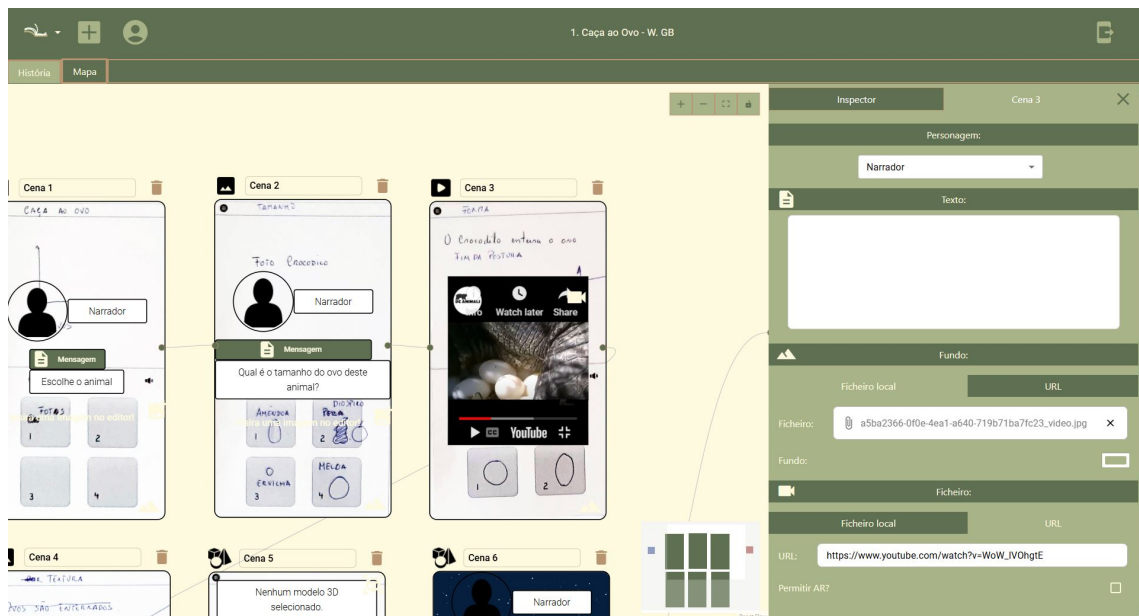


Figure B.5: Third Scene in the Story Editor

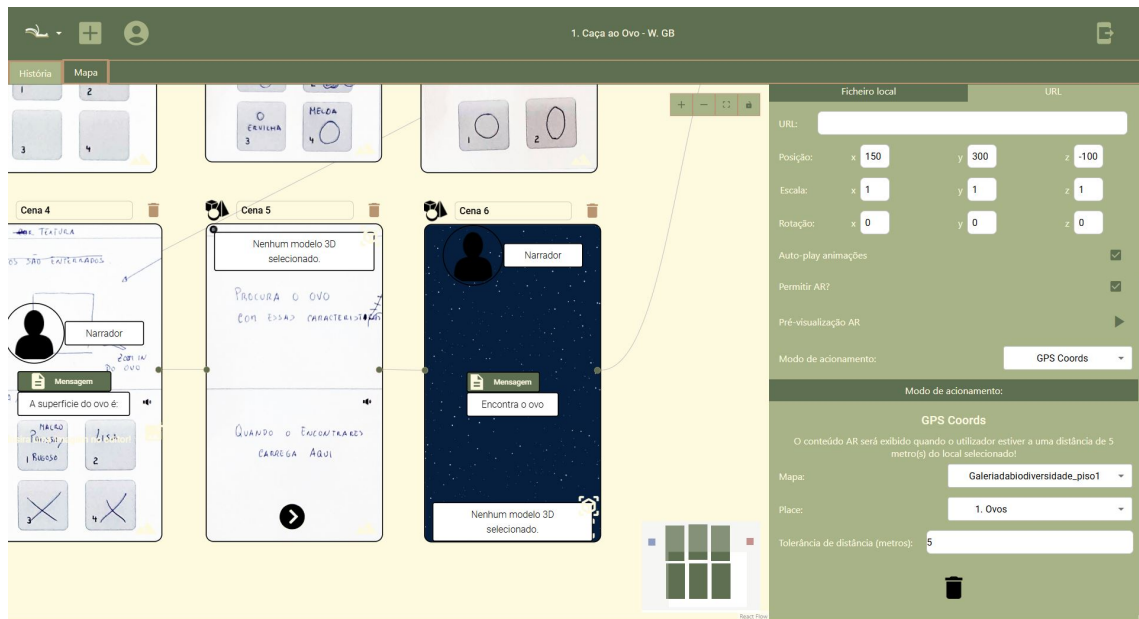


Figure B.6: Sixth Scene in the Story Editor

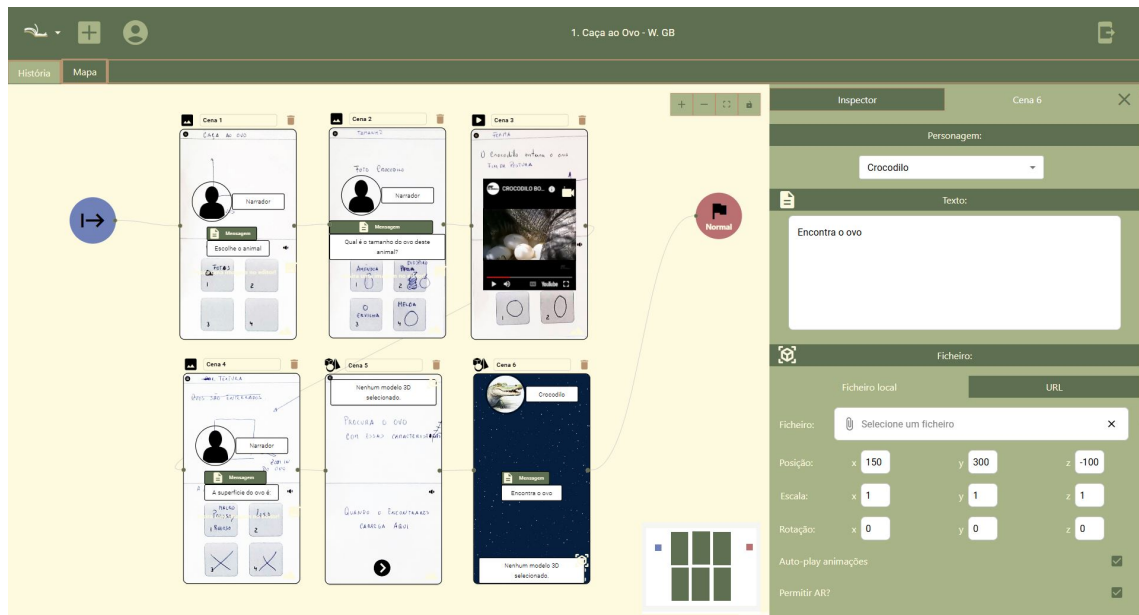


Figure B.7: Final State of the Story

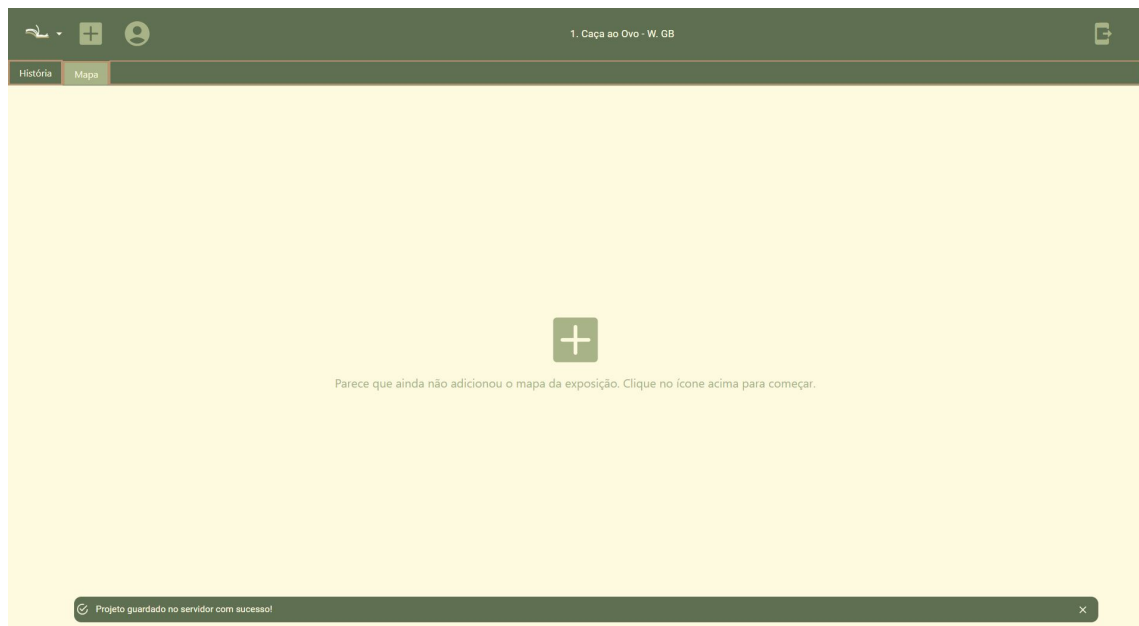


Figure B.8: Beginning of the Map Creation Process



Figure B.9: Placement of the Anchors

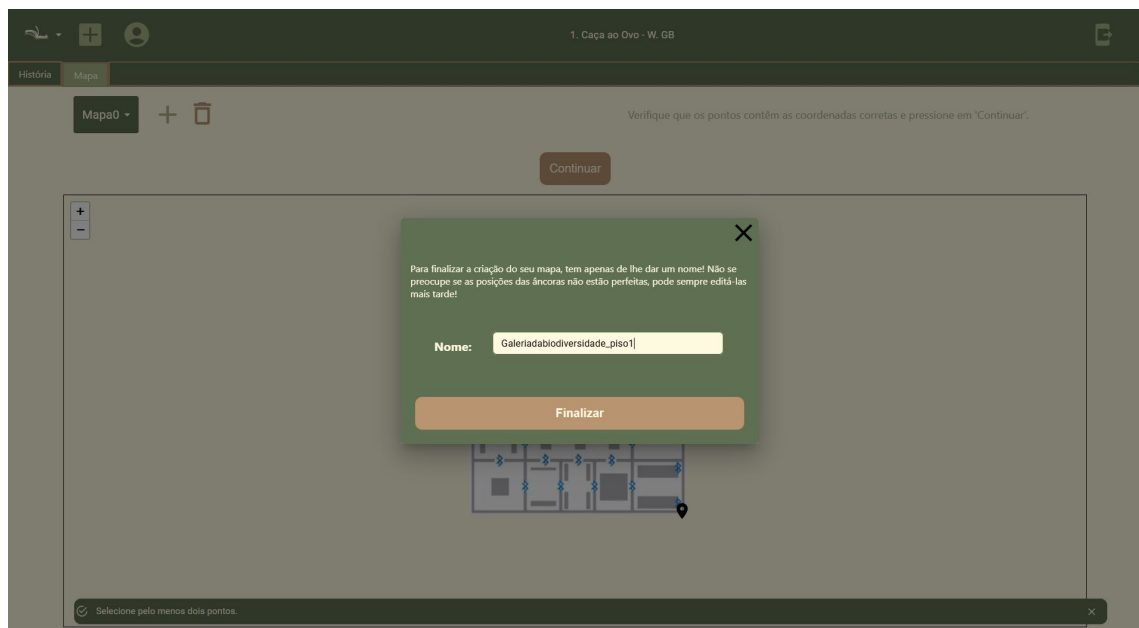


Figure B.10: Naming the map



Figure B.11: Final State of the map

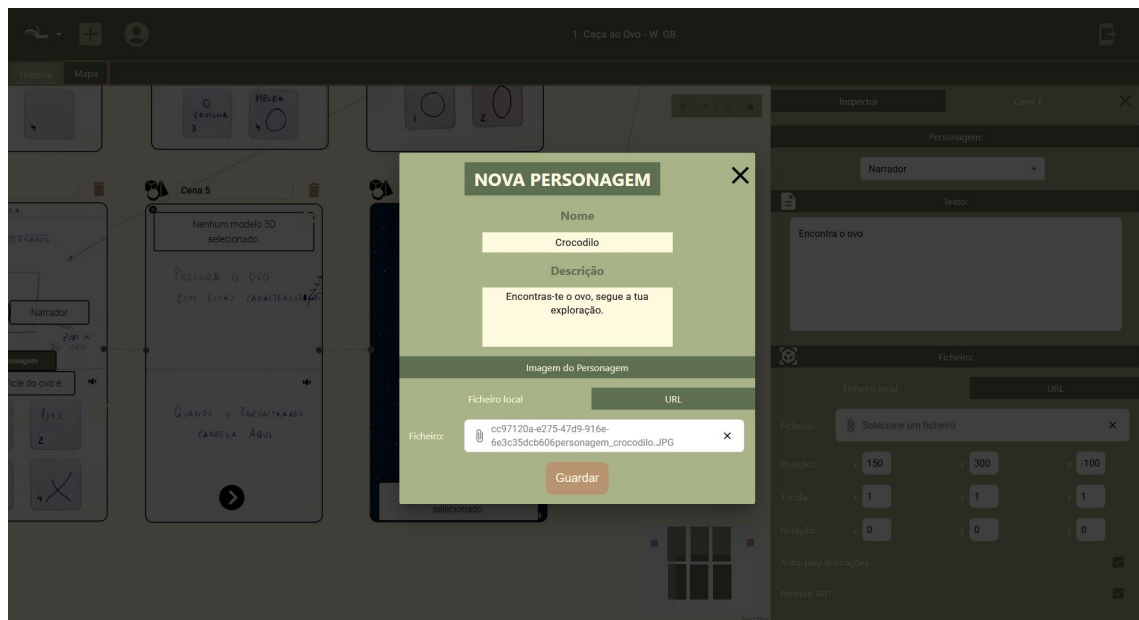


Figure B.12: Creating a new Character

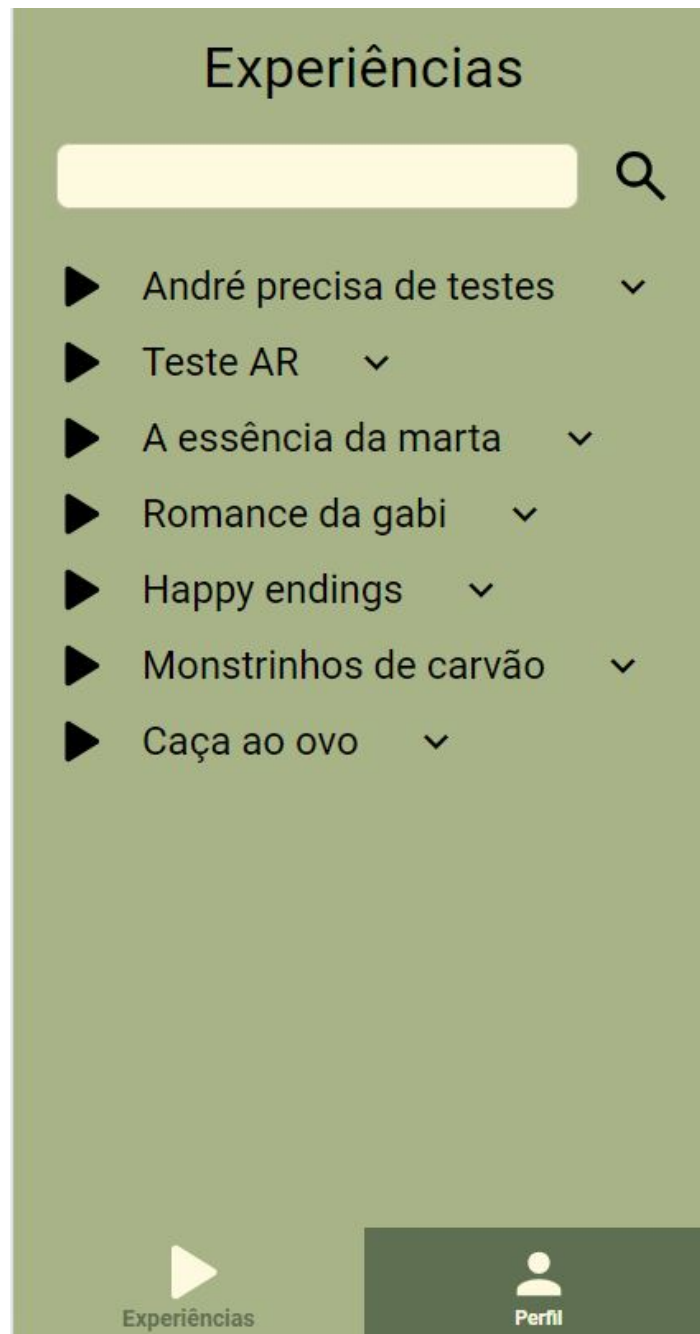


Figure B.13: List of Experiences in the Story Player

Appendix C

User Testing Results

This appendix contains the graphs displaying the answers participants provided in the user experience survey.

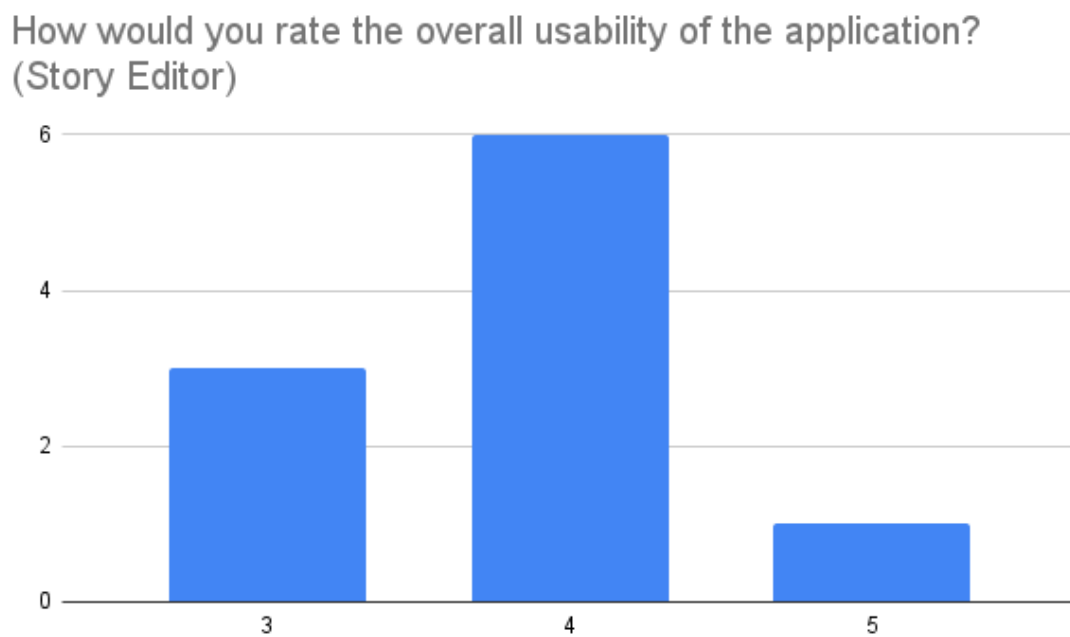


Figure C.1: Results for the "How would you rate the overall usability of the application?" question (Story Editor)

How difficult was it to navigate your story structure?

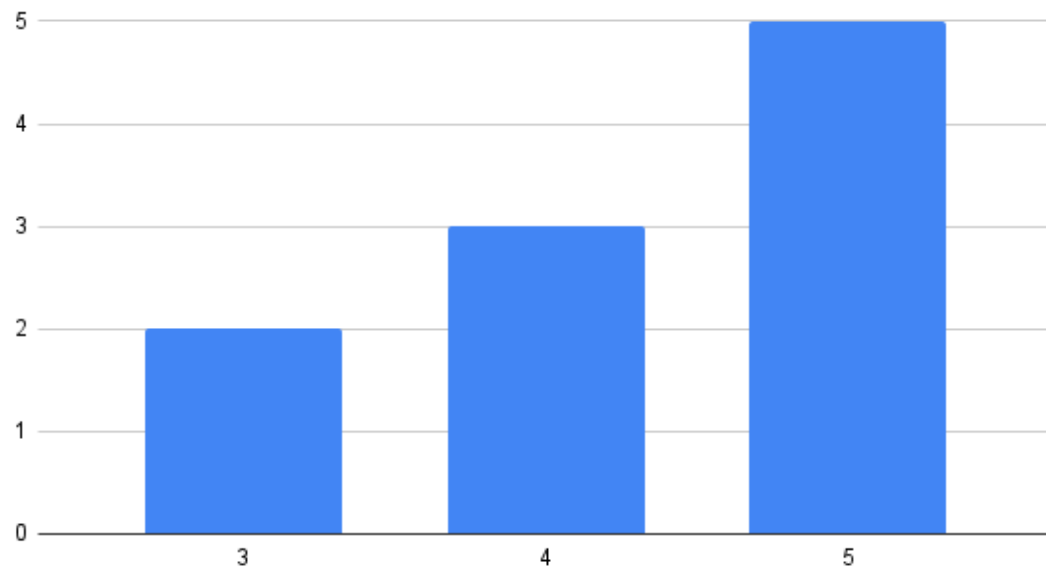


Figure C.2: Results for the "How difficult was it to navigate your story structure?" question

How difficult was it to use the "Inspector" tab to edit the content of your Scenes?

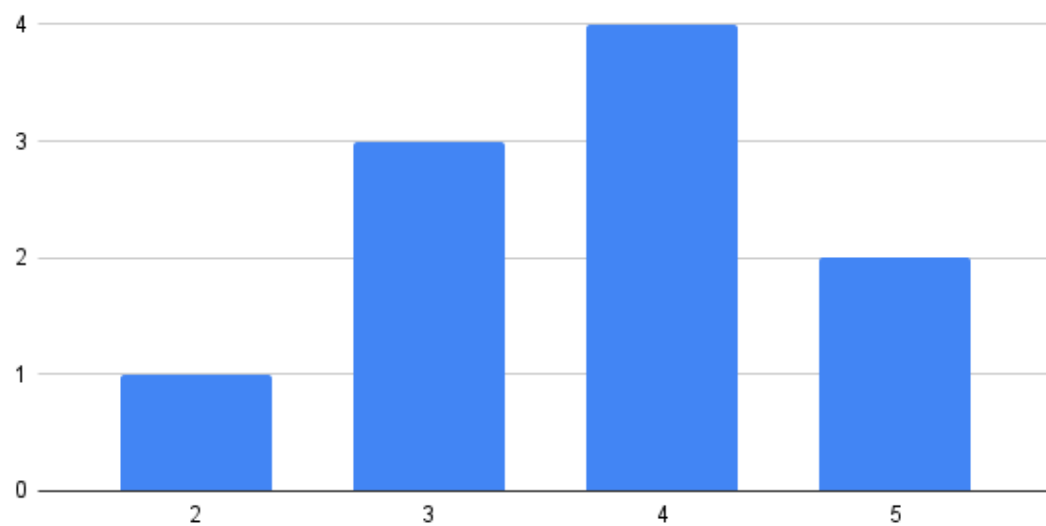


Figure C.3: Results for the "How difficult was it to use the "Inspector" tab to edit the content of your Scenes?" question

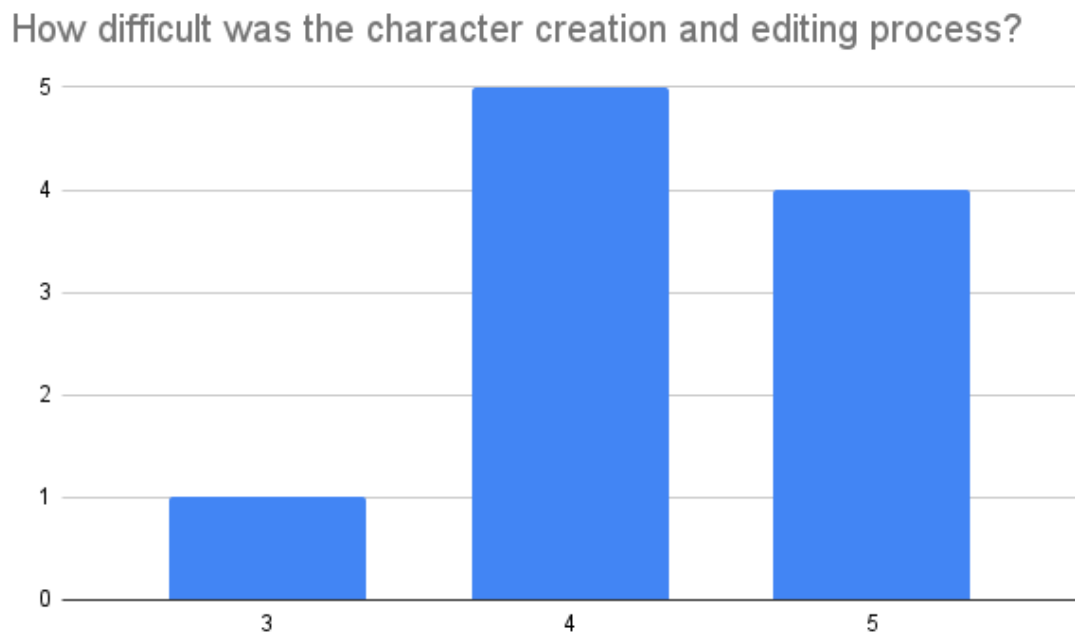


Figure C.4: Results for the "How difficult was the character creation and editing process?" question

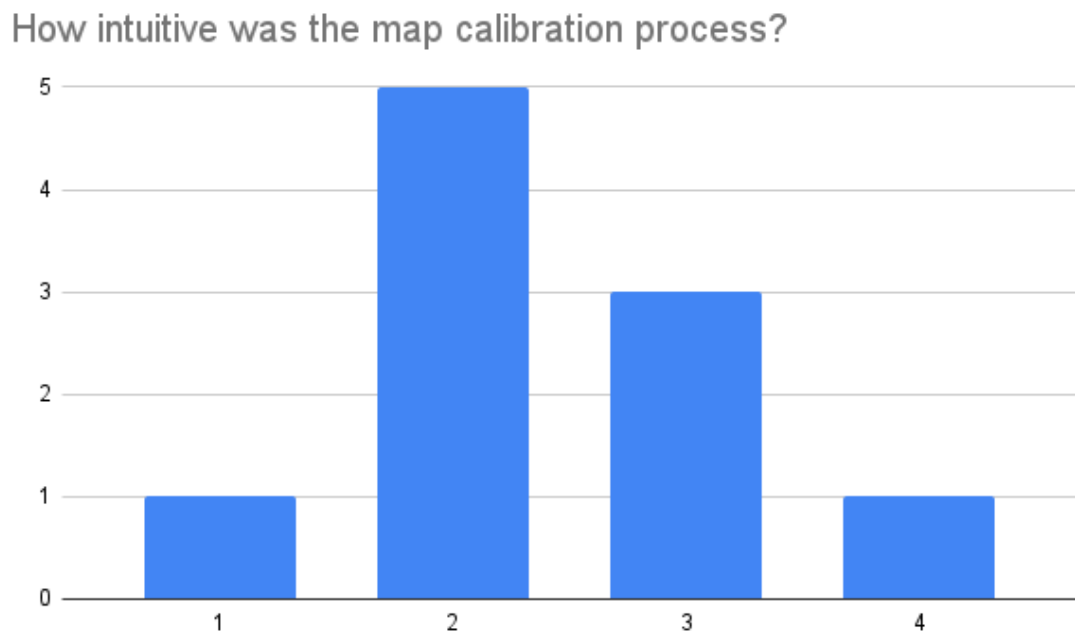


Figure C.5: Results for the "How intuitive was the map calibration process?" question

How useful was the "Map" tab to annotate the physical location of your experience?

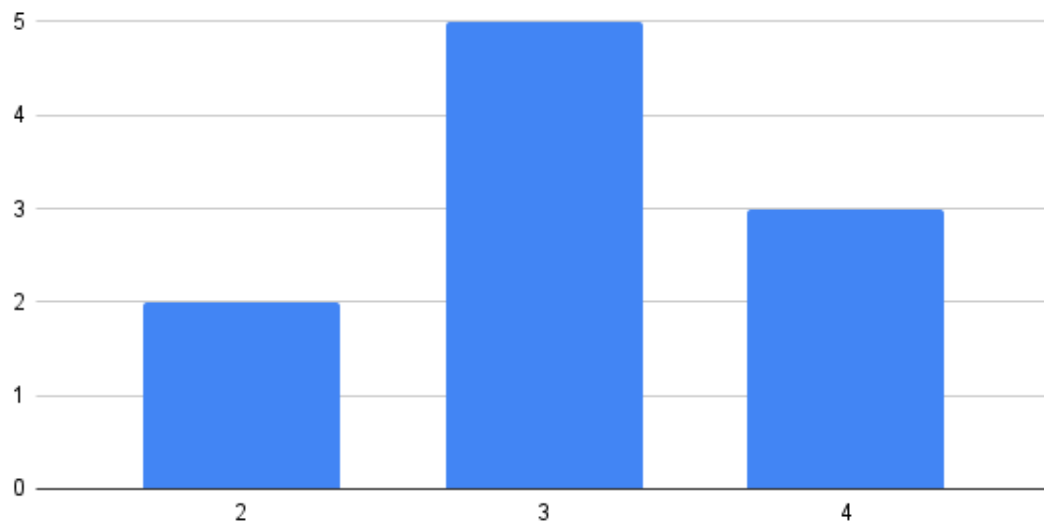


Figure C.6: Results for the "How useful was the "Map" tab to annotate the physical location of your experience?" question

How difficult was it to export your story?

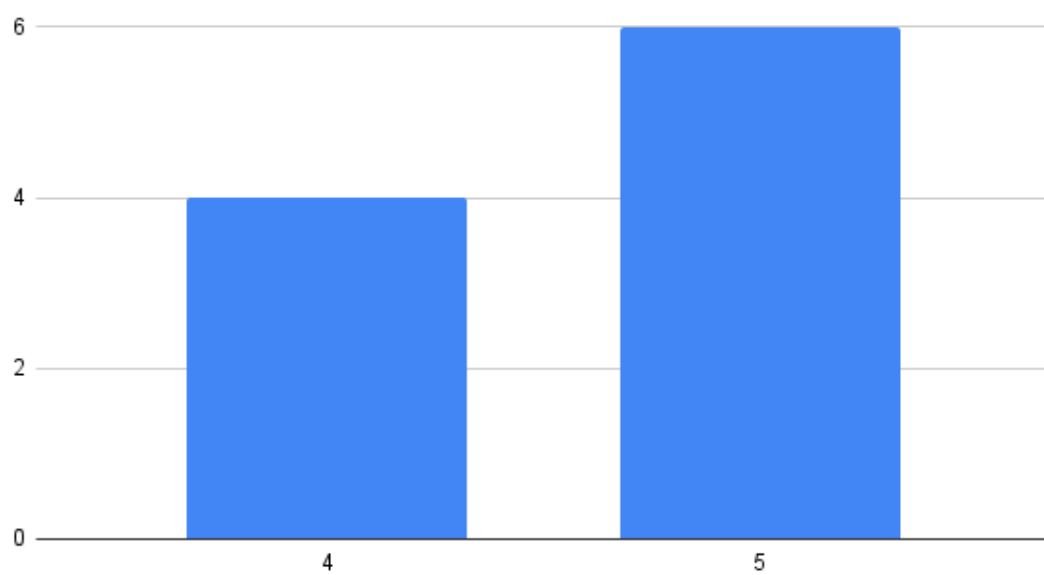


Figure C.7: Results for the "How difficult was it to export your story?" question

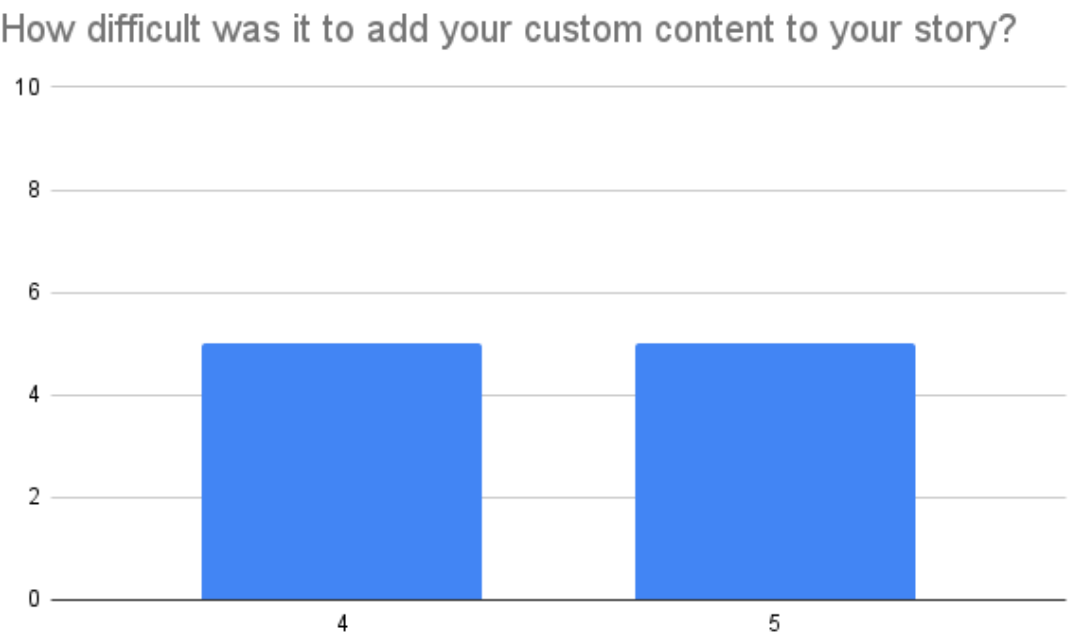


Figure C.8: Results for the "How difficult was it to add your custom content to your story?" question

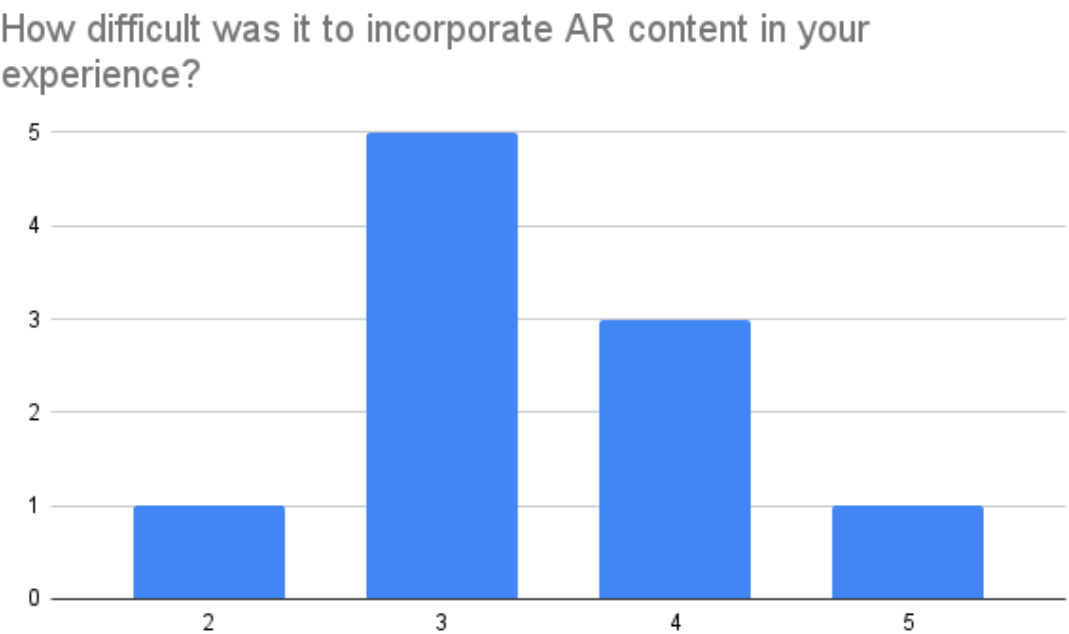


Figure C.9: Results for the "How difficult was it to incorporate AR content in your experience?" question

How clear were the explanations on how the different trigger modes for AR work?

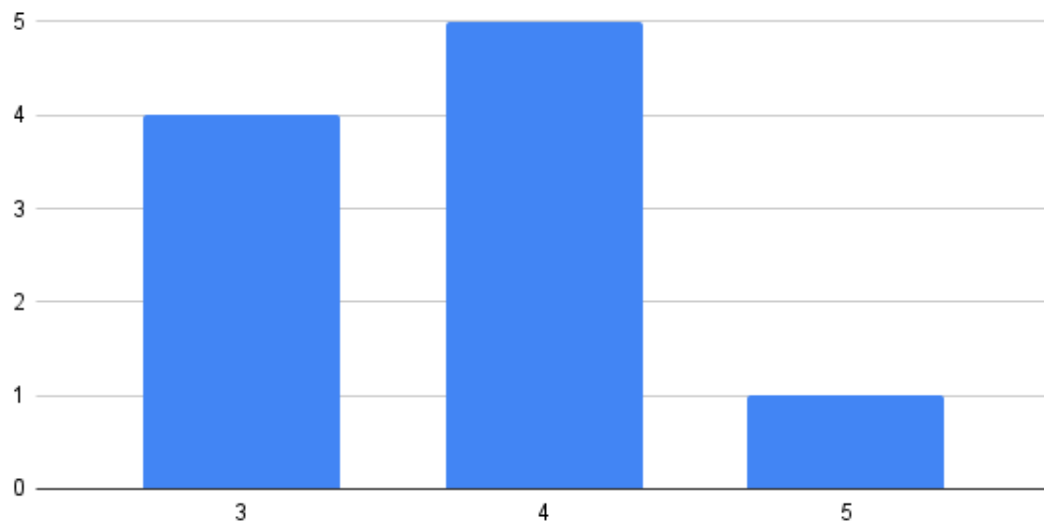


Figure C.10: Results for the "How clear were the explanations on how the different trigger modes for AR work?" question

How clear were the export warning/error messages?

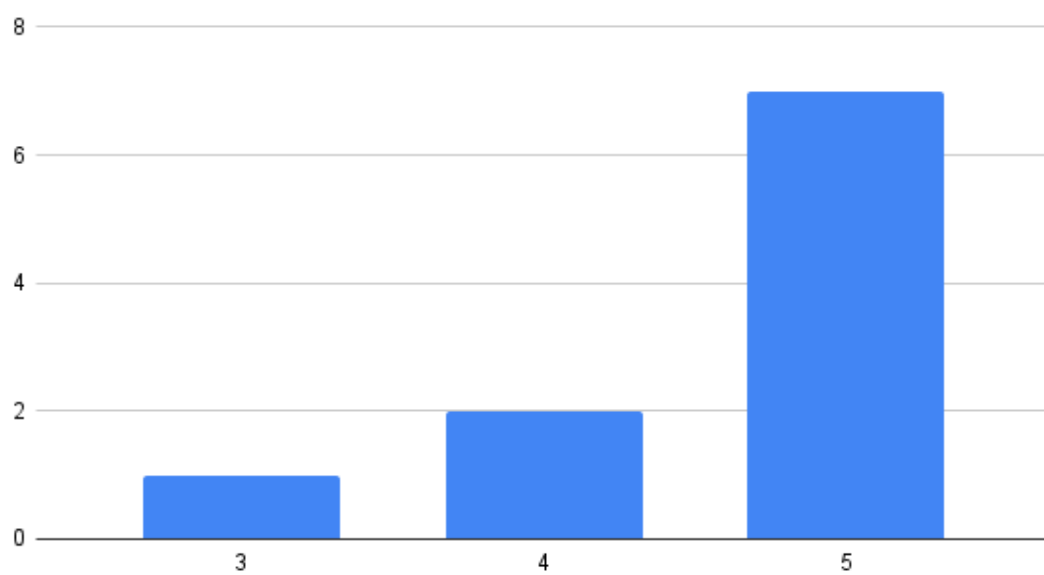


Figure C.11: Results for the "How clear were the export warning/error messages?" question

How would you rate the application's performance (Story Editor) in terms of responsiveness?

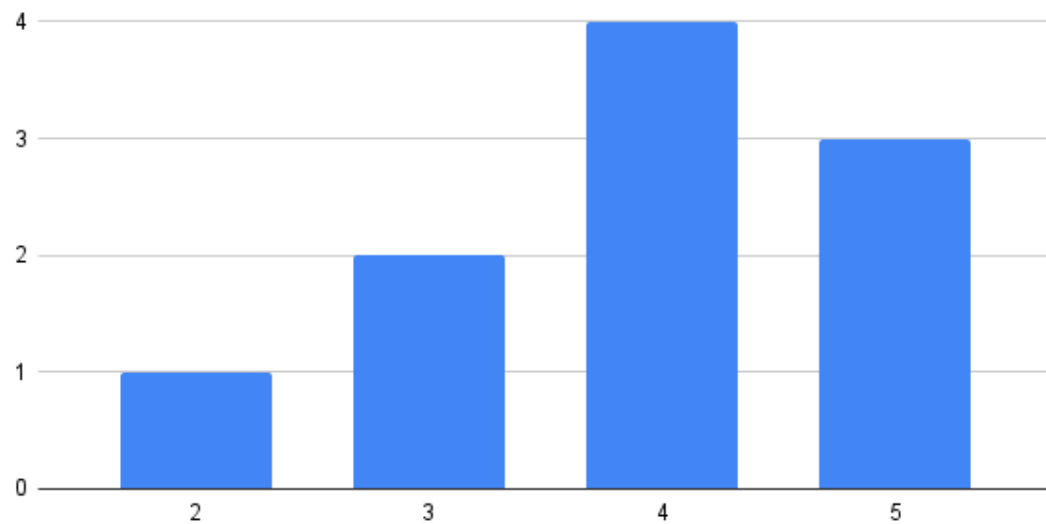


Figure C.12: Results for the "How would you rate the application's performance in terms of responsiveness?" question

How would you rate the application's performance (Story Editor) in terms of speed?

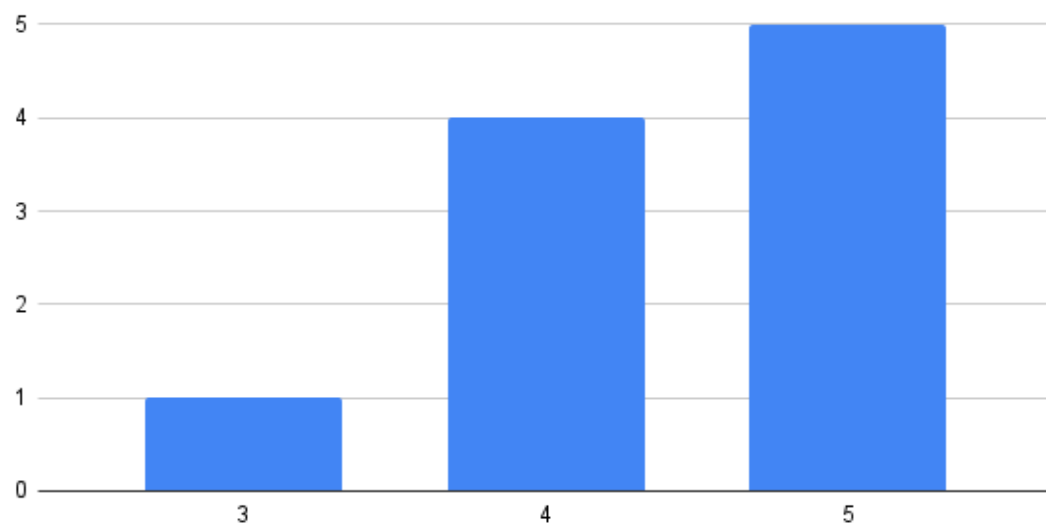


Figure C.13: Results for the "How would you rate the application's performance in terms of speed?" question

How would you rate the overall usability (Story Player) of the application?

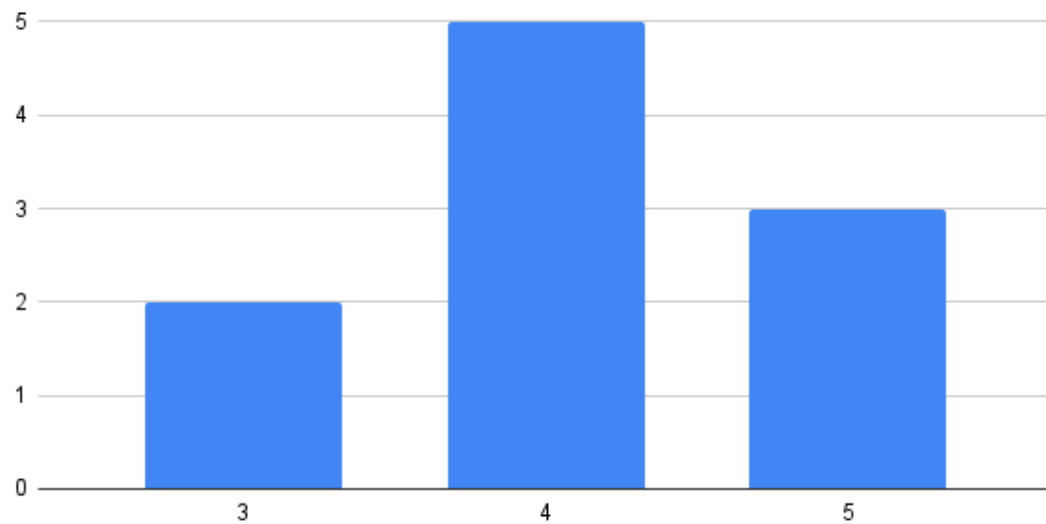


Figure C.14: Results for the "How would you rate the overall usability of the application?" question (Story Player)

How satisfied were you with the content displayed in the Profile tab?

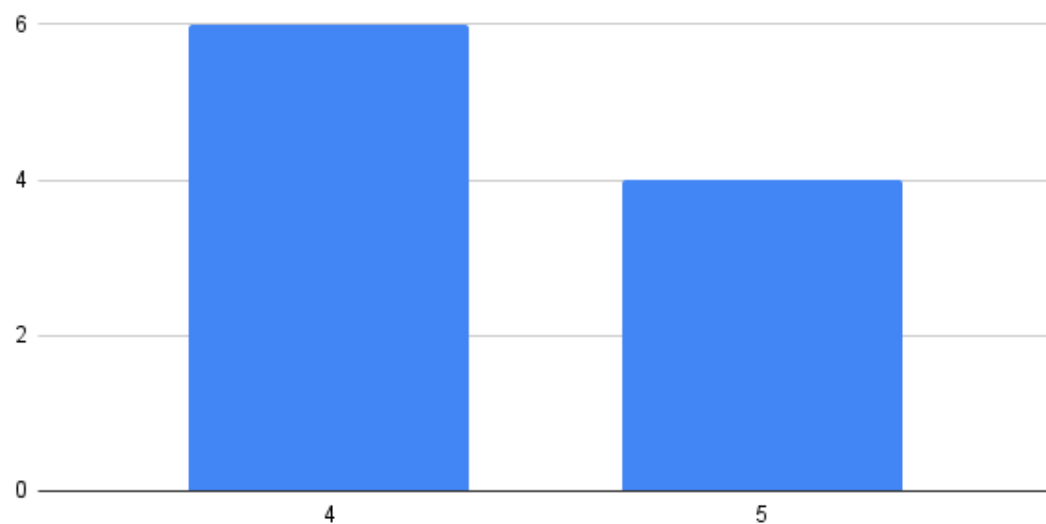


Figure C.15: Results for the "How satisfied were you with the content displayed in the Profile tab?" question

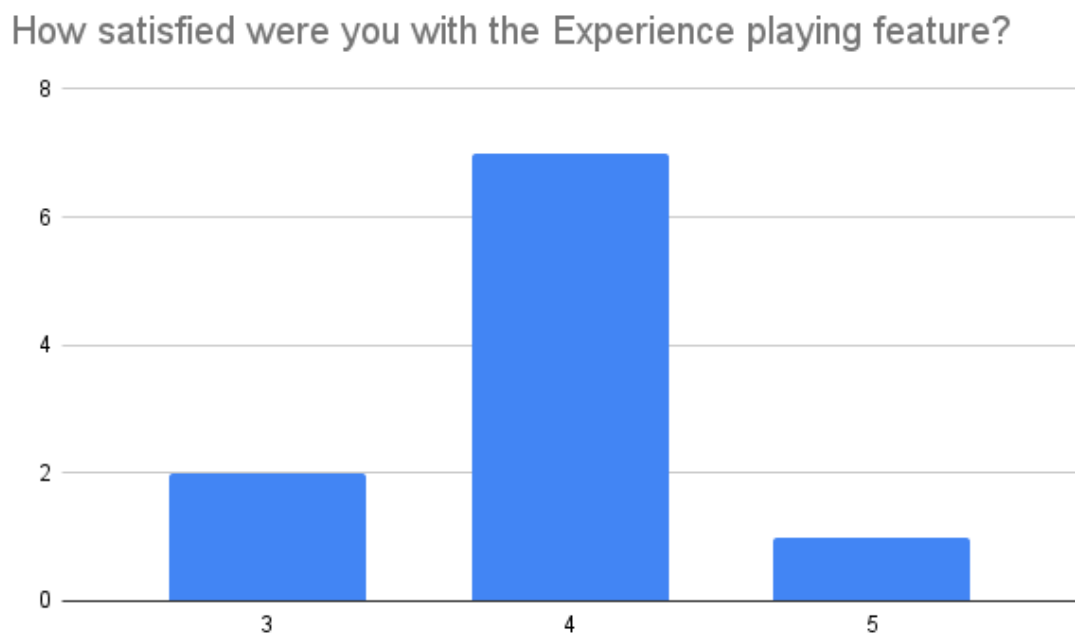


Figure C.16: Results for the "How satisfied were you with the Experience playing feature?" question